VELLORE INSTITUTE OF TECHNOLOGY, CHENNAI

B.Tech. Computer Science and Engineering with Specialization in Artificial Intelligence and Robotics

School of Computer Science and Engineering (SCOPE)

PROJECT REPORT

FINSIGHT

Cloud-Based Personal Finance Intelligence & Expense Analytics Platform

Understand Your Spending. Make Smarter Financial Decisions.

Submitted by

Aakar Gupta

Registration Number: 24BRS1321

Academic Project Report

Domain: Cloud Computing, FinTech, Personal Finance Management and Data Analytics

Faculty Guide: Dr. Pradeep K (SCOPE)

Programme: B.Tech. Computer Science and Engineering (AI and Robotics)

Department: School of Computer Science and Engineering (SCOPE)

Academic Year: 2026-27

Date of Submission: 16-10-2026

Vellore Institute of Technology, Chennai

VELLORE INSTITUTE OF TECHNOLOGY, CHENNAI

School of Computer Science and Engineering (SCOPE)

CERTIFICATE

This is to certify that the project report entitled **"FinSight — Cloud-Based Personal Finance Intelligence & Expense Analytics Platform"** is a record of the project work carried out by **Aakar Gupta (Registration Number 24BRS1321)**, a student of the B.Tech. Computer Science and Engineering with Specialization in Artificial Intelligence and Robotics, School of Computer Science and Engineering (SCOPE), under my guidance and supervision during the academic year 2026-27, in partial fulfilment of the requirements of the B.Tech. programme at Vellore Institute of Technology, Chennai.

The project designs and implements a personal finance tracking and analytics web application with an explainable financial intelligence engine, anomaly detection, receipt OCR, a What-If simulator and a rule-based finance assistant, together with its serverless deployment on Amazon Web Services.

Faculty Guide: Dr. Pradeep K

Department: School of Computer Science and Engineering (SCOPE)

Programme: B.Tech. CSE (AI and Robotics)

Date: 16-10-2026

Dr. Pradeep K
Faculty Guide
School of Computer Science and Engineering (SCOPE)
Vellore Institute of Technology, Chennai

TABLE OF CONTENTS

ABSTRACT xi

1. INTRODUCTION 1

1.1 Background 1

1.2 Problem Statement 1

1.3 Motivation 2

1.4 Objectives 2

1.5 Scope 3

1.6 Proposed Solution 3

1.7 Technology Overview 3

1.8 System Architecture 4

1.9 Functional Requirements 5

1.10 Non-Functional Requirements 6

2. LITERATURE REVIEW 7

2.1 Personal Finance Management and Tracking Behaviour 8

2.2 Anomaly and Outlier Detection 8

2.3 OCR and Receipt Understanding 9

2.4 Interpretability and Explainable Decision Support 9

2.5 Serverless Cloud Computing 9

2.6 Design Decisions Derived from the Literature 9

3. GAPS IDENTIFIED AND LIMITATIONS 11

3.1 Gaps in Existing Approaches 11

3.2 Identified System Requirements 11

3.3 Technical Limitations 11

3.4 Security and Privacy Limitations 12

4. MODULES 13

4.1 Authentication and User Management 13

4.2 Expense Management 15

4.3 Category and Payment-Method Management 18

4.4 Budget Management and Alerts 19

4.5 Financial Dashboard 20
4.6 Spending Analytics 22
4.7 Personal Finance Intelligence Engine 24
4.8 Financial Anomaly Detection 24
4.9 Smart Receipt Scanner 25
4.10 What-If Financial Simulator 27
4.11 Conversational Finance Assistant 27
4.12 Savings Goals 28
4.13 Receipt Storage 29
4.14 Backend API 30
4.15 Local Database (SQLite) 31
4.16 Cloud Database (Amazon DynamoDB) 32
4.17 AWS Authentication and Authorization 32
4.18 Cloud Monitoring and Error Handling 32
4.19 Module Interaction and Data Flow 33
4.20 Data Model and Entity Relationship Diagram 34
5. SYSTEM DESIGN AND USER WORKFLOW 35
5.1 Primary User Workflow 35
5.2 Public Landing Page 35
5.3 Responsive Design 36
5.4 Requirements Traceability 38
6. IMPLEMENTATION DETAILS 39
6.1 Frontend Architecture 39
6.2 Backend Architecture 39
6.3 REST API Design 39
6.4 Database Design 39
6.5 Authentication 39
6.6 Authorization 40
6.7 Expense Management 40
6.8 Financial Analytics 40

6.9 Financial Intelligence 40
6.10 Anomaly Detection 40
6.11 Receipt OCR 40
6.12 What-If Simulation 41
6.13 Rule-Based Finance Assistant 41
6.14 Savings Goals 41
6.15 AWS Integration 41
6.16 Error Handling 41
6.17 Logging and Monitoring 42
6.18 Security Considerations 42
6.19 Local Execution versus AWS Deployment 42
6.20 User Interface Animations 42
7. ALGORITHMS AND FINANCIAL INTELLIGENCE 43
7.1 Exact Money Arithmetic 43
7.2 Spending Aggregation and Period Comparison 43
7.3 Budget Utilization, Thresholds and Projection 44
7.4 Insight Generation Rules 45
7.5 Recurring Payment Detection 46
7.6 Anomaly Detection 46
7.7 OCR Extraction and Receipt Parsing 48
7.8 What-If Simulation 49
7.9 Rule-Based Natural-Language Assistant 50
7.10 Savings Goal Metrics 51
8. TESTING AND EVALUATION 53
8.1 Testing Strategy 53
8.2 Automated Test Results 53
8.3 Representative Test Cases 54
8.4 Live Testing on AWS 55
8.5 Problems Found During Deployment and Their Fixes 61
8.6 Evaluation and Limitations 61

9. AWS ARCHITECTURE AND DEPLOYMENT 62
9.1 Deployed Architecture 62
9.2 Frontend Hosting Decision 63
9.3 Resource Mapping 65
9.4 Deployment Workflow 65
9.5 Deployed Resources in the AWS Management Console 66
9.6 Resource Views Rendered from AWS API Responses 71
9.7 Monitoring 73
9.8 Cost Analysis 73
9.9 Cleanup and Rollback 75
10. SECURITY AND PRIVACY 76
11. CONCLUSION 78
12. FUTURE WORK 79
REFERENCES 80

LIST OF FIGURES

Figure 1: High-level system architecture of FinSight 5
Figure 2: Sign-in and registration pages (dark theme) 13
Figure 3: Profile page (cropped to the content area) 14
Figure 4: Settings page: preferences, appearance, categories, demo data and data deletion (cropped) 15
Figure 5: Transactions page with summary cards, search and sortable table 16
Figure 6: Add-expense form (cropped to the content area) 17
Figure 7: Edit-expense form with the Delete option (cropped to the content area) 17
Figure 8: Expense detail dialog with Delete and Edit actions 18
Figure 9: Expense filtering by text, category and payment method (14 matching transactions) 19
Figure 10: Budgets page: overall budget, alerts, category budgets and history 20
Figure 11: Dashboard in the dark theme (synthetic demo data, labelled “Demo”) 21
Figure 12: Dashboard in the light theme 22
Figure 13: Analytics page (three-month range) 23
Figure 14: Financial insights with an expanded “Why am I seeing this?” evidence panel 24
Figure 15: Anomaly alerts with reasons, comparison values and review actions 25
Figure 16: Receipt scanner after genuine Tesseract OCR of a generated sample receipt: extracted fields with confidence and the review form 26
Figure 17: What-If simulator comparing two scenarios (results marked “Hypothetical”) 27
Figure 18: Finance assistant answering two questions with tabular evidence 28
Figure 19: Savings goals with progress, required monthly saving and contribution history 29
Figure 20: Receipt upload panel (local storage mode) 30
Figure 21: Authentication and authorization sequence (Cognito mode, as deployed) 32
Figure 22: Module interaction diagram 33
Figure 23: Level-1 data flow diagram 34
Figure 24: Entity relationship diagram of the logical data model 34
Figure 25: Primary user workflow 35
Figure 26: Public landing page 36
Figure 27: Mobile layout at 390 px: dashboard (left) and navigation drawer (right) 37
Figure 28: Example: category-wise spending for the current month (synthetic demo data) 44
Figure 29: Example: monthly spending against the overall budget (synthetic demo data) 44

Figure 30: Example: budget utilization by budget (synthetic demo data) 45
Figure 31: Anomaly detection flow for a new or edited expense 48
Figure 32: Example: modified z-scores of Shopping transactions (synthetic demo data) 48
Figure 33: Smart receipt scanner pipeline..... 49
Figure 34: Example: hypothetical monthly savings of three What-If scenarios (synthetic demo data)..... 50
Figure 35: Example: savings goal progress (synthetic demo data)..... 52
Figure 36: Backend automated tests per module (all 131 passed)..... 54
Figure 37: Live dashboard on AWS after Cognito sign-in (synthetic test user; data from DynamoDB)..... 57
Figure 38: Live receipt scanner: image uploaded to the private S3 bucket and read by the Tesseract container function, with fields awaiting review..... 58
Figure 39: Live application at 390 px: dashboard (left) and navigation drawer (right).... 59
Figure 40: About the author dialog on the live landing page, opened from the information button (dark theme)..... 60
Figure 41: Live landing page footer with the “Created by Aakar Gupta” credit, which opens the same dialog 60
Figure 42: Deployed AWS architecture of FinSight (FrontendHosting=apigateway) 63
Figure 43: CloudFormation events: FrontendDistribution CREATE_FAILED, rollback and deletion of the failed attempt (AWS Console; account ID masked)..... 64
Figure 44: AWS SAM deployment workflow as executed..... 66
Figure 45: CloudFormation: the three FinSight stacks and the finance-intelligence-prod overview (UPDATE_COMPLETE) (AWS Console; account ID masked) 66
Figure 46: CloudFormation outputs: WebUrl and ApiUrl on one origin, FrontendHosting=apigateway, table, bucket and Cognito identifiers (AWS Console; account ID masked) 67
Figure 47: API Gateway: the CognitoJwt authorizer attached to the API routes (JWT Auth) (AWS Console; account ID masked)..... 67
Figure 48: Lambda: the nine FinSight functions (eight zip packages on Python 3.13 and the OCR container image) (AWS Console; account ID masked)..... 68
Figure 49: DynamoDB: table finance-intelligence-prod, on-demand capacity, PK/SK keys, point-in-time recovery off (AWS Console; account ID masked)..... 68
Figure 50: Cognito: user pool finance-intelligence-prod on the Lite feature plan (AWS Console; account ID masked) 69
Figure 51: S3: receipts bucket permissions with Block all public access turned on (AWS Console; account ID masked) 69
Figure 52: ECR: the three OCR container image versions; the newest is used by the receipts function (AWS Console; account ID masked) 69

Figure 53: CloudWatch Logs: receipts-function log events with method, path, status and duration only (AWS Console; account ID masked) 70
Figure 54: CloudWatch: ApiServerErrorsAlarm (5xx ≥ 5 in 5 minutes) in state OK (AWS Console; account ID masked) 70
Figure 55: IAM: one execution role per FinSight function, created by the stack (AWS Console; account ID masked) 71
Figure 56: Deployed Lambda functions (eight API functions and WebFunction) and their shared non-secret environment (rendered from live AWS API responses — not a Console screenshot) 72
Figure 57: API Gateway HTTP API: Cognito JWT authorizer and the 25 routes with their authorization type (rendered from live AWS API responses — not a Console screenshot) 73
Figure 58: Billing and Cost Management: credits summary on 16 October 2026 (AWS Console; account ID masked) 74

LIST OF TABLES

Table 1: Technology stack and rationale....4
Table 2: Functional requirements and implementation status....5
Table 3: Non-functional requirements....6
Table 4: Literature review summary....7
Table 5: Module specification — Authentication and User Management....13
Table 6: Module specification — Expense Management....16
Table 7: Module specification — Category and Payment-Method Management....18
Table 8: Module specification — Budget Management and Alerts....19
Table 9: Module specification — Financial Dashboard....21
Table 10: Module specification — Spending Analytics....23
Table 11: Module specification — Personal Finance Intelligence Engine....24
Table 12: Module specification — Financial Anomaly Detection....25
Table 13: Module specification — Smart Receipt Scanner....26
Table 14: Module specification — What-If Financial Simulator....27
Table 15: Module specification — Conversational Finance Assistant....28
Table 16: Module specification — Savings Goals....29
Table 17: API summary (paths relative to /api/v1)....30
Table 18: SQLite schema....31
Table 19: User workflow steps and corresponding screens....35
Table 20: Requirements traceability matrix....38
Table 21: Local execution compared with the AWS deployment....42
Table 22: Automated and scripted test results....53
Table 23: Representative test cases....54
Table 24: Live AWS test results by area (64 passed, 0 failed)....56
Table 25: Problems found during deployment and live testing....61
Table 26: AWS resource mapping and deployment status....65
Table 27: Security controls and their verification status....76

ABSTRACT

Personal expense tracking is easy to start and hard to sustain. Spreadsheets and basic expense apps record transactions but rarely explain them: users must work out for themselves whether a month is unusually expensive, which category is growing, whether a budget will hold until month-end or how a spending change would affect a savings goal. Many tools also depend on bank connections or paid AI services, which raise privacy and cost concerns for a student-scale deployment.

FinSight is a web-based personal finance intelligence and expense analytics platform built to close this gap. It records expenses with exact integer arithmetic, supports search, filtering, sorting, pagination and CSV export, manages monthly and category budgets with threshold alerts and month-end projections, and tracks savings goals and contributions. Five analytical modules run on the application's own deterministic logic: a Personal Finance Intelligence Engine that produces insights labelled as fact, estimate, projection or recommendation together with their evidence; anomaly detection based on robust statistics (median and median absolute deviation) with a human review workflow; a receipt scanner that uses the open-source Tesseract OCR engine and a rule-based parser followed by mandatory human review; a What-If simulator for hypothetical spending changes; and a rule-based conversational finance assistant.

The implementation uses React 19, TypeScript, Vite, Tailwind CSS and Recharts on the frontend and Python with FastAPI on the backend. A storage-independent repository layer runs on SQLite locally and on Amazon DynamoDB in the cloud. The AWS SAM template was deployed to the ap-south-1 (Mumbai) region: API Gateway (HTTP API) with an Amazon Cognito JWT authorizer, eight Lambda functions for the API (one a container image for OCR), one DynamoDB table, a private S3 receipts bucket and CloudWatch. Because the new account could not yet create CloudFront distributions, a static-file Lambda function serves the interface on the same HTTP API.

131 backend and 20 frontend tests pass, as do the TypeScript check, the production build, cfn-lint and sam validate. On the live deployment, all 64 API, authentication, storage and OCR checks and all 27 browser checks passed. FinSight provides decision support only; it is not a source of professional financial advice and does not connect to bank accounts.

Keywords: personal finance management, expense analytics, explainable insights, anomaly detection, OCR, serverless computing, AWS.

1. INTRODUCTION

1.1 Background

Managing personal finances well depends on knowing where money goes. Research on financial literacy treats financial knowledge as a form of human capital and links it to better saving, borrowing and planning decisions [1]. In practice, however, the information needed for good decisions is scattered across payment apps, bank statements, paper receipts and memory. Ethnographic work on how people track money shows that individuals combine many tools and that money carries strong emotional weight, which makes consistent tracking difficult [2].

Digital payments have made individual purchases faster and smaller, so a typical month contains many low-value transactions that are easy to overlook. Tools that merely store these records leave the user to perform the analysis. A useful personal finance system therefore needs to record transactions reliably **and** turn them into understandable, verifiable feedback about budgets, trends and goals.

1.2 Problem Statement

Users who try to manage their spending face several recurring difficulties:

- **Fragmented records.** Expenses are entered in different places (notes, spreadsheets, payment histories) and receipts are rarely linked to entries.
- **Limited understanding of patterns.** Totals alone do not show which categories are growing, whether a month is unusual or how this month compares with the same period last month.
- **Late budget feedback.** Users usually discover an overspent budget after the month ends instead of receiving a projection while there is still time to act.
- **Opaque "insights".** Automated advice is often presented without the calculation or data behind it, which makes it hard to trust or check.
- **Manual data entry.** Typing receipt details is tedious; automated extraction is error-prone and must be reviewed.
- **Disconnected goals.** Savings targets are rarely connected to spending behaviour, so users cannot see how a spending change affects a goal.
- **Privacy and cost.** Bank aggregation and paid AI services raise privacy questions and recurring costs that are hard to justify for an individual or a student project.

The problem addressed by this project is to design and build a secure, low-cost application that records expenses accurately and explains spending behaviour with transparent, evidence-based analytics, while keeping the user in control of every automated result.

1.3 Motivation

An integrated platform can connect activities that are usually separate: recording an expense immediately updates budgets, analytics, anomaly checks and insights; a scanned receipt becomes a reviewed expense; and a hypothetical spending change can be evaluated against a real savings goal. Explainability is a central motivation. Every insight in FinSight states whether it is a measured fact, an estimate, a projection or a recommendation and shows its calculation, period and supporting transactions. This follows the argument that interpretability should be defined and evaluated deliberately rather than assumed [12]. A further motivation is to practise cloud architecture under a realistic constraint: a serverless design that costs little when idle and protects private financial data by construction [10], [11].

1.4 Objectives

The project objectives are stated so that each can be checked against the implementation and the test evidence in Chapter 8:

- **O1:** Record, edit, delete, search, filter, sort, paginate and export expenses, storing money as integer minor units (paise) so that no floating-point rounding errors occur.
- **O2:** Support an overall monthly budget and category budgets with utilization, configurable alert thresholds (default 75%, 90% and 100%, one alert per threshold per month), month-end projections and history.
- **O3:** Provide a dashboard and an analytics page computed only from the user's recorded data, including period-over-period comparison of like-for-like date ranges.
- **O4:** Generate financial insights that are labelled by kind and carry their evidence (period, formula, values and transactions).
- **O5:** Flag unusual transactions with robust statistical rules, record the reason and comparison values, and let the user review alerts without the system ever modifying transactions.
- **O6:** Extract merchant, date, total, tax and line items from receipt images with open-source OCR and require human confirmation before an expense is created.
- **O7:** Evaluate hypothetical spending reductions and their effect on budgets and goals without changing stored records.
- **O8:** Answer common spending questions in natural language with a deterministic, rule-based assistant that uses only the user's own data.
- **O9:** Track savings goals, contributions, required monthly saving and progress status.
- **O10:** Enforce authentication on every private endpoint and isolate each user's data, verified by automated security tests.

- **O11:** Deploy the application to AWS as a cost-conscious serverless stack defined as infrastructure as code, and verify it with linting, tests against mocked AWS services and live end-to-end tests.

1.5 Scope

In scope: a responsive single-page web application with light and dark themes; local email/password authentication and an implemented Amazon Cognito mode; expense, category, payment-method, budget and goal management; dashboard and analytics; the five analytical modules described above; private receipt storage; demo data that is clearly labelled and removable; a FastAPI REST API; SQLite and DynamoDB repositories; and an AWS SAM template for deployment.

Out of scope or not supported: income tracking and net-savings calculations, bank-account synchronisation, investment portfolios, real-time market data, automatic multi-currency conversion, professional financial advice and fraud determination. CloudFront content delivery is defined in the template but was not deployed, because the AWS account had not yet been verified for CloudFront (Chapter 9).

1.6 Proposed Solution

FinSight is organised as a React single-page application that calls a versioned JSON API (/api/v1). The backend follows a layered design (routes → services → repository): HTTP concerns live in route modules, all business logic in storage-independent services, and persistence behind a repository interface with two implementations. The same code therefore runs locally (Uvicorn, SQLite, a private receipts folder and local JWT authentication) and in the deployed AWS configuration (API Gateway, Lambda through the Mangum adapter, DynamoDB, private S3 and Cognito). Analytical features are deterministic and explainable, and every automated result is shown with its basis and can be reviewed by the user.

1.7 Technology Overview

Table 1 summarises the technologies verified in the repository and the reasons for choosing them. Versions are those declared in the project's dependency files and observed in the development environment.

Table 1: Technology stack and rationale

Layer	Technology (verified)	Alternatives considered	Reason for choice
Frontend UI	React 19, TypeScript, Vite 8	Angular, Vue, server-rendered pages	Component model, strong typing, fast builds; static output suits S3 + CloudFront hosting [23]
Styling	Tailwind CSS v4 with design tokens	CSS modules, component libraries	Consistent light/dark theming from one token set; small CSS output
Charts	Recharts 3	Chart.js, D3	Declarative React charts with tooltips and accessible markup
Forms	React Hook Form + Zod	Manual state	Client-side validation mirroring server rules
Backend	Python 3, FastAPI, Pydantic v2	Flask, Django, Node.js	Typed request validation, automatic error handling, async-capable; runs on Lambda through Mangum [22], [13]
Local database	SQLite (WAL mode)	PostgreSQL	Zero-administration file database for development and tests
Cloud database	Amazon DynamoDB (on-demand)	Amazon RDS, Aurora	Serverless, pay-per-request, no idle instance cost; per-user partition key [16]
OCR	Tesseract 5 via pytesseract, Pillow	Amazon Textract, Google Vision	Open source with no per-page fees; runs inside the application [7], [24]
Authentication	Local scrypt + HS256 JWT; Amazon Cognito (RS256) in AWS mode	Custom OAuth server	Managed identity in the cloud; simple self-contained mode for development [18]
Infrastructure as code	AWS SAM / CloudFormation	Terraform, manual console	Native serverless resources, change-set review before deployment [14]

1.8 System Architecture

Figure 1 shows the high-level architecture. The presentation layer is a React SPA; the application layer is a FastAPI service whose domain services do not depend on HTTP or storage; and the data layer is selected by configuration (DATA_BACKEND and RECEIPT_STORAGE). The left profile at the bottom is the local configuration, and the right profile is the AWS configuration, which was tested against mocked AWS services and then deployed and tested live. The detailed AWS deployment architecture appears in Figure 42 (Chapter 9).

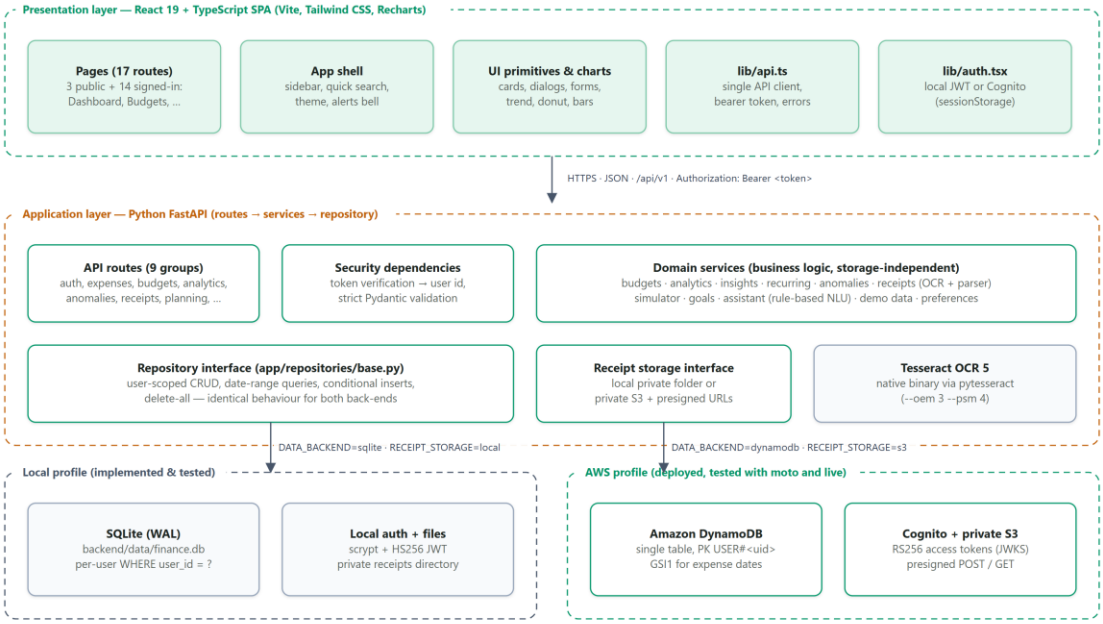


Figure 1: High-level system architecture of FinSight

1.9 Functional Requirements

Table 2: Functional requirements and implementation status

ID	Requirement	Module(s)	Status
FR1	Register, sign in, sign out; token-protected API	Authentication	Implemented; local mode tested; Cognito mode tested live on AWS
FR2	Create, view, edit and delete expenses with validation	Expense management	Implemented and tested
FR3	Search, filter (date, category, payment method, amount), sort, paginate, export CSV	Expense management	Implemented and tested
FR4	Built-in and custom categories; payment methods	Categories	Implemented and tested
FR5	Overall and category budgets; versions per month; history	Budgets	Implemented and tested
FR6	Threshold alerts, one per threshold per budget per month	Budgets	Implemented and tested
FR7	Dashboard: totals, change, budget, trend, categories, activity	Dashboard	Implemented; browser-verified
FR8	Analytics over preset and custom date ranges	Analytics	Implemented; browser-verified
FR9	Labelled, evidence-based financial insights	Intelligence engine	Implemented and tested
FR10	Anomaly detection with review states	Anomaly detection	Implemented and tested
FR11	Receipt upload, OCR extraction, review and confirmation	Receipt scanner	Implemented; real Tesseract OCR tested locally and in the Lambda container
FR12	What-If scenarios and required-savings calculation	Simulator	Implemented and tested

ID	Requirement	Module(s)	Status
FR13	Natural-language questions about spending, budgets and goals	Assistant	Implemented and tested
FR14	Savings goals, contributions, withdrawals, status	Goals	Implemented and tested
FR15	Profile, preferences, theme, demo data, delete all data	Profile & settings	Implemented; browser-verified

1.10 Non-Functional Requirements

Table 3: Non-functional requirements

Attribute	Requirement	How it is addressed	Evidence / status
Security	Only authenticated users reach private data; one user cannot read another's data	Token verification on every private route; user id only from the token; per-user queries	11 security tests pass; JWT authorizer and isolation tested live
Correctness	Exact money arithmetic	Integer minor units; Decimal ratios	Money and date tests pass
Maintainability	Business logic independent of storage and transport	Routes → services → repository; two repositories share one test suite	Full API suite also runs on mocked DynamoDB
Performance	Responsive pages for personal data volumes	Per-user date-range queries; aggregation in the service layer	No load test; live warm requests 15–130 ms, cold starts about 3 s
Reliability	Clear failure states; no fabricated data	Uniform error envelope; UI error states; backend-down screen	Verified in the browser locally and on AWS (0 console errors)
Usability	Consistent, responsive interface in two themes	Shared design tokens and primitives; mobile drawer	Checked at 1440 px and 390 px in both themes
Scalability	Scale without servers to manage	Lambda, HTTP API and on-demand DynamoDB	Deployed serverless; not load-tested
Cost awareness	Minimal idle cost	No VPC/NAT/EC2/RDS; log retention; throttling; Budgets alert	Estimated at about US$0.05–0.10 per month (Chapter 9)

2. LITERATURE REVIEW

The review covers five areas that shaped FinSight: personal finance behaviour and tracking, anomaly and outlier detection, OCR and receipt understanding, interpretability, and serverless cloud computing. Sources were selected for relevance and verifiability; bibliographic details were checked against publisher or repository records. Table 4 compares the works. Where a source reports no comparable quantitative measurements, the comparison is qualitative and this is stated explicitly; no performance figures are introduced that the sources do not report.

Table 4: Literature review summary

Serial Number	Journal Name and Year	Model/System Name	Parameters or Evaluation Criteria	Algorithms or Approach Comparison	Limitations
1	Journal of Economic Literature, 2014 [1]	Financial literacy as human-capital investment (Lusardi & Mitchell)	Survey evidence on knowledge and behaviour; no system cost	Theoretical and empirical review; not an algorithmic comparison	Explains why feedback matters but proposes no tool; mainly U.S. evidence
2	ACM CHI, 2014 [2]	Qualitative study of personal finance tracking (Kaye et al.)	Interviews with 14 participants	Contrasts tools people actually use (spreadsheets, apps, memory) and their emotional context	Small qualitative sample; design implications, not a validated system
3	ACM Computing Surveys, 2009 [3]	Taxonomy of anomaly detection (Chandola, Banerjee & Kumar)	Data requirements, labels needed, computational complexity per family	Classification, nearest-neighbour, clustering, statistical, information-theoretic and spectral methods	Survey; no single recommended method; complexity varies by technique
4	Statistical Science, 2002 [4]	Statistical fraud detection review (Bolton & Hand)	Supervised vs unsupervised settings; data availability	Supervised classifiers need labelled fraud; unsupervised outlier/peer-group methods flag deviations for review	Stresses that flagged cases need human investigation; domain data is scarce
5	Journal of Experimental Social Psychology, 2013 [5]	Median absolute deviation (MAD) outlier rule (Leys et al.)	Robustness to extreme values; simple computation	MAD vs mean ± 3 standard deviations: mean and SD are themselves distorted by outliers	Assumes roughly symmetric data; MAD can be zero for repeated identical values
6	ASQC Quality Press, 1993 [6]	Modified z-score (Iglewicz & Hoaglin)	Threshold 3.5 on $0.6745 \cdot (x - \text{median})/\text{MAD}$	Robust alternative to classical z-scores for small samples	Book-length treatment; threshold is a convention, not a guarantee
7	ICDAR, 2007 [7]	Tesseract OCR engine architecture (Smith)	Open source; runs on CPU; no per-page fee	Line finding, feature extraction and adaptive classifier described; no comparative benchmark	Accuracy depends on image quality and layout; no semantic field extraction

Serial Number	Journal Name and Year	Model/System Name	Parameters or Evaluation Criteria	Algorithms or Approach Comparison	Limitations
8	ICDAR, 2019 [8]	SROIE scanned-receipt benchmark (Huang et al.)	1,000 receipt images; tasks: text localisation, OCR, key information extraction	Compares many competition systems on a shared protocol	Reports a remaining performance gap in key information extraction; English printed receipts
9	ACM SIGKDD, 2020 [9]	LayoutLM: joint text-and-layout pre-training (Xu et al.)	Large-scale pre-training on document images; GPU training	Reports gains over text-only models on form and receipt understanding	Needs training data, GPU resources and model hosting; higher cost and complexity
10	UC Berkeley Technical Report, 2019 [10]	Serverless computing view (Jonas et al.)	Pay-per-use billing, autoscaling, no server administration	Contrasts serverless functions with serverful cloud computing	Notes limits for state, communication and long-running work
11	CIDR, 2019 [11]	Critique of first-generation FaaS (Hellerstein et al.)	Function lifetimes, I/O and data movement	Argues FaaS is a step forward for operations but weak for data-centric workloads	Position paper; platform limits have evolved since 2019
12	arXiv preprint, 2017 [12]	Framework for evaluating interpretability (Doshi-Velez & Kim)	Application-, human- and function-grounded evaluation	Defines evaluation levels rather than comparing algorithms	No single metric; human evaluation is costly

2.1 Personal Finance Management and Tracking Behaviour

Lusardi and Mitchell [1] review evidence that financial knowledge is associated with better financial outcomes, which motivates tools that make spending information visible and understandable. Kaye et al. [2] show that tracking is fragmented across tools and shaped by emotion and household context. Together these works suggest that a tracking tool should reduce the effort of recording (for example by receipt scanning), present information calmly and transparently, and avoid judgemental language. FinSight adopts neutral wording such as “worth a look” for anomalies and labels projections explicitly as uncertain.

2.2 Anomaly and Outlier Detection

Chandola et al. [3] classify anomaly detection techniques and note that the right family depends on the availability of labels, the nature of the data and computational cost. Bolton and Hand [4] distinguish supervised fraud detection, which needs labelled fraud cases, from unsupervised methods that flag unusual behaviour for human investigation. A personal finance app has no labelled fraud data and very small per-user samples, so supervised models are unsuitable. Leys et al. [5] argue that the mean ± 3 standard deviations rule is distorted by the very outliers it should detect and recommend the

median absolute deviation; Iglewicz and Hoaglin [6] define the modified z-score with the 3.5 threshold. FinSight implements this modified z-score per category, adds a ratio-to-median condition to avoid flagging small absolute deviations, handles the MAD = 0 case explicitly, and keeps every alert reviewable, consistent with the human-in-the-loop practice described in [4].

2.3 OCR and Receipt Understanding

Smith [7] describes the architecture of the Tesseract engine, which recognises text but does not identify semantic fields. The SROIE competition [8] shows that key information extraction from receipts remains harder than text recognition. Layout-aware deep models such as LayoutLM [9] report better field extraction but require large training corpora, GPUs and model hosting. Given the cost constraints of this project, FinSight combines Tesseract with a transparent rule-based parser, attaches a confidence score to every field and makes human review mandatory, accepting lower automation in exchange for zero inference cost and full user control.

2.4 Interpretability and Explainable Decision Support

Doshi-Velez and Kim [12] argue that interpretability must be defined in terms of its use and evaluated at application, human or function level. FinSight applies a function-grounded form of transparency: each insight carries its kind (fact, estimate, projection, recommendation or data gap), period, formula, input values and supporting transactions, so the user can verify the statement directly. The project does not claim a human-subject evaluation of interpretability.

2.5 Serverless Cloud Computing

Jonas et al. [10] present serverless computing as a model that removes server administration and bills only for use, while noting limitations for state and communication. Hellerstein et al. [11] criticise first-generation function platforms for data-intensive work. FinSight's workload — short, user-scoped API requests over small datasets — fits the strengths described in [10] and avoids the weaknesses in [11]: each request reads one user's partition, and the only long-running task (OCR, up to 60 seconds) runs in its own function.

2.6 Design Decisions Derived from the Literature

- Use robust statistics (median, MAD, modified z-score) rather than mean-based rules or supervised models for anomaly detection [3]–[6].
- Treat every flagged transaction as a prompt for review, never as proof of fraud [4].

- Use open-source OCR with explicit confidence and mandatory review instead of paid or GPU-hosted extraction [7]–[9].
- Expose the evidence behind every insight to support verification [12].
- Adopt a serverless, per-user partitioned architecture that matches short, independent requests [10], [11].

How FinSight differs. The reviewed works address individual problems. FinSight integrates them into one application in which recording, budgeting, explanation, anomaly review, receipt capture, simulation and goal tracking share the same user-scoped data and every automated output is explainable and reviewable. Its contribution is integration and transparency, not a new detection algorithm.

3. GAPS IDENTIFIED AND LIMITATIONS

3.1 Gaps in Existing Approaches

- **Fragmented tracking.** Users split records across tools [2]; receipts, budgets and goals are seldom connected to individual expenses.
- **Limited contextual analysis.** Totals are reported without like-for-like comparisons, category context or recurring-payment detection.
- **Late budget feedback.** Overspending is recognised after the fact rather than projected during the month.
- **Low interpretability.** Automated advice rarely shows its basis, although interpretability needs explicit definition and evaluation [12].
- **Unsuitable anomaly methods.** Supervised fraud models need labels unavailable to individuals [4]; mean-based rules are not robust [5].
- **Costly document understanding.** High-accuracy receipt extraction relies on large models and hosting [9], and remains imperfect [8].
- **Privacy and deployment constraints.** Bank aggregation and paid AI services add data exposure and recurring cost.

3.2 Identified System Requirements

These gaps translate into the capabilities FinSight attempts to provide: a single user-scoped data model that links expenses, receipts, budgets and goals; like-for-like comparisons and projections during the month; insights with explicit kind and evidence; robust, reviewable anomaly rules that never modify data; low-cost OCR with mandatory confirmation; hypothetical simulation separated from real records; and a deployment design with no idle servers. Table 20 (Chapter 5) traces these requirements to modules and evidence.

3.3 Technical Limitations

- **OCR errors.** Tesseract misreads characters on blurred, skewed, low-contrast or handwritten receipts; PDF receipts are not supported; fields are left blank with warnings when not found.
- **Image-quality dependence.** Pre-processing (rotation, grayscale, scaling, contrast, sharpening) helps but cannot recover information that is not in the image.
- **Rule-based guidance.** Insights and assistant answers cover predefined patterns and 14 intents; unusual phrasing or unsupported topics (income, investments) are answered with an explanation instead of a result.
- **Anomaly detection limits.** Rules need minimum history (5 transactions per category, 10 overall); new users receive few alerts; alerts indicate unusual spending, not fraud.

- **Data dependence.** All analytics depend on the completeness and accuracy of what the user records; FinSight has no bank feed.
- **Cloud verification gaps.** CloudFront delivery was not deployed; Cognito e-mail confirmation codes were not tested through a real inbox; access-token revocation after sign-out takes effect within 15 s and is skipped if Cognito is unavailable.
- **No performance evaluation.** No load test or user study was conducted; only individual live request timings were observed, and no accuracy figure is claimed for OCR on real receipts.
- **Cloud cost.** The account is on the AWS Free plan with promotional credits; Cost Explorer showed US$0.00 month-to-date, but its data lag by up to 24 hours, so the running cost is still an estimate.
- **External services.** In AWS mode the application depends on Cognito availability and its default e-mail sender quota.

3.4 Security and Privacy Limitations

Implemented controls are listed in Table 27 (Chapter 10). Remaining limitations include: no multi-factor authentication in local mode; an in-memory login rate limiter that resets on restart; tokens kept in sessionStorage (readable by injected script, mitigated by React's output escaping); no malware scanning or web application firewall; OCR text stored with the receipt record without separate encryption; and, in Cognito mode, “delete all data” removes application data but not the Cognito user. The project has not undergone an independent security audit and makes no compliance claims.

4. MODULES

This chapter describes every module in a consistent format: purpose, inputs, processing, outputs, technologies, API endpoints, error handling, security considerations and limitations. Endpoint paths are relative to /api/v1. Screenshots are genuine captures of the locally running application using a fictional demo account and FinSight's synthetic demo data, which the interface labels “Demo”.

4.1 Authentication and User Management

This module identifies the user, issues or verifies tokens and stores per-user preferences. Table 5 specifies the module and Figure 2 shows its user interface.

Table 5: Module specification — Authentication and User Management

Aspect	Description
Purpose	Register users, sign them in and out, and derive the user identity for every request.
Inputs	Name, e-mail, password (local mode); Cognito tokens (AWS mode); profile and preference changes.
Processing	Local: scrypt password hashing (N = 2^14, r = 8, p = 1, 16-byte salt), HS256 JWT with required claims and pinned algorithm, login rate limit (5 failures per e-mail and IP per 5 minutes). Cognito: RS256 signature check against the pool's JWKS, issuer, expiry, token_use = access and client_id.
Outputs	Access token; user context (user id, preferences); profile data.
Technologies	FastAPI dependencies, PyJWT, hashlib.scrypt, Amazon Cognito (AWS mode), React auth provider (lib/auth.tsx).
Endpoints	POST /auth/register, POST /auth/login, POST /auth/change-password (local mode); GET /me, PUT /preferences; GET /config (public).
Error handling	Generic “Incorrect email or password”; 401 for missing, invalid or expired tokens; 429 after repeated failures; field-level validation errors.
Security	User id taken only from the verified token; passwords never stored or logged in plaintext; tokens in memory and sessionStorage, never in URLs.
Limitations	No MFA; sign-out revocation is checked with Cognito GetUser and takes effect within 15 s (fail-open if Cognito is unavailable).



Figure 2: Sign-in and registration pages (dark theme)

Profile and settings (Figures 3 and 4) manage the display name, password, currency, time zone, alert thresholds, anomaly sensitivity, theme, custom categories, demo data and deletion of all data.

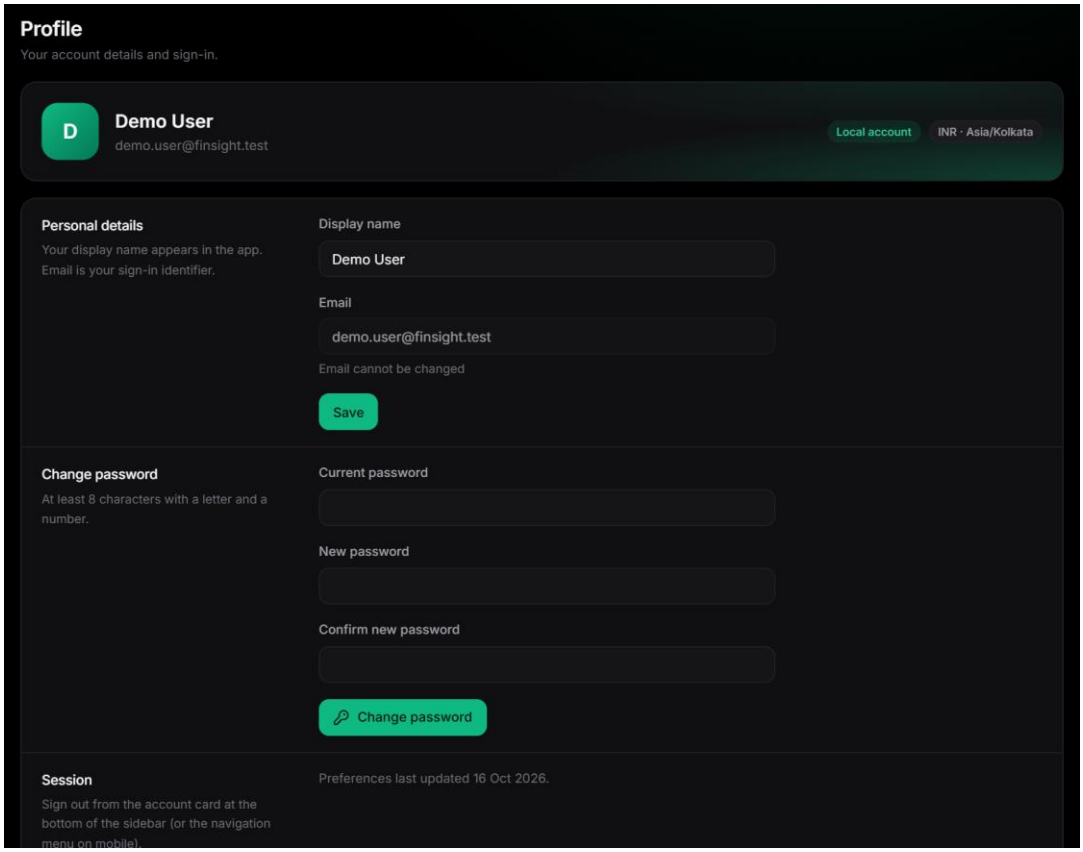


Figure 3: Profile page (cropped to the content area)

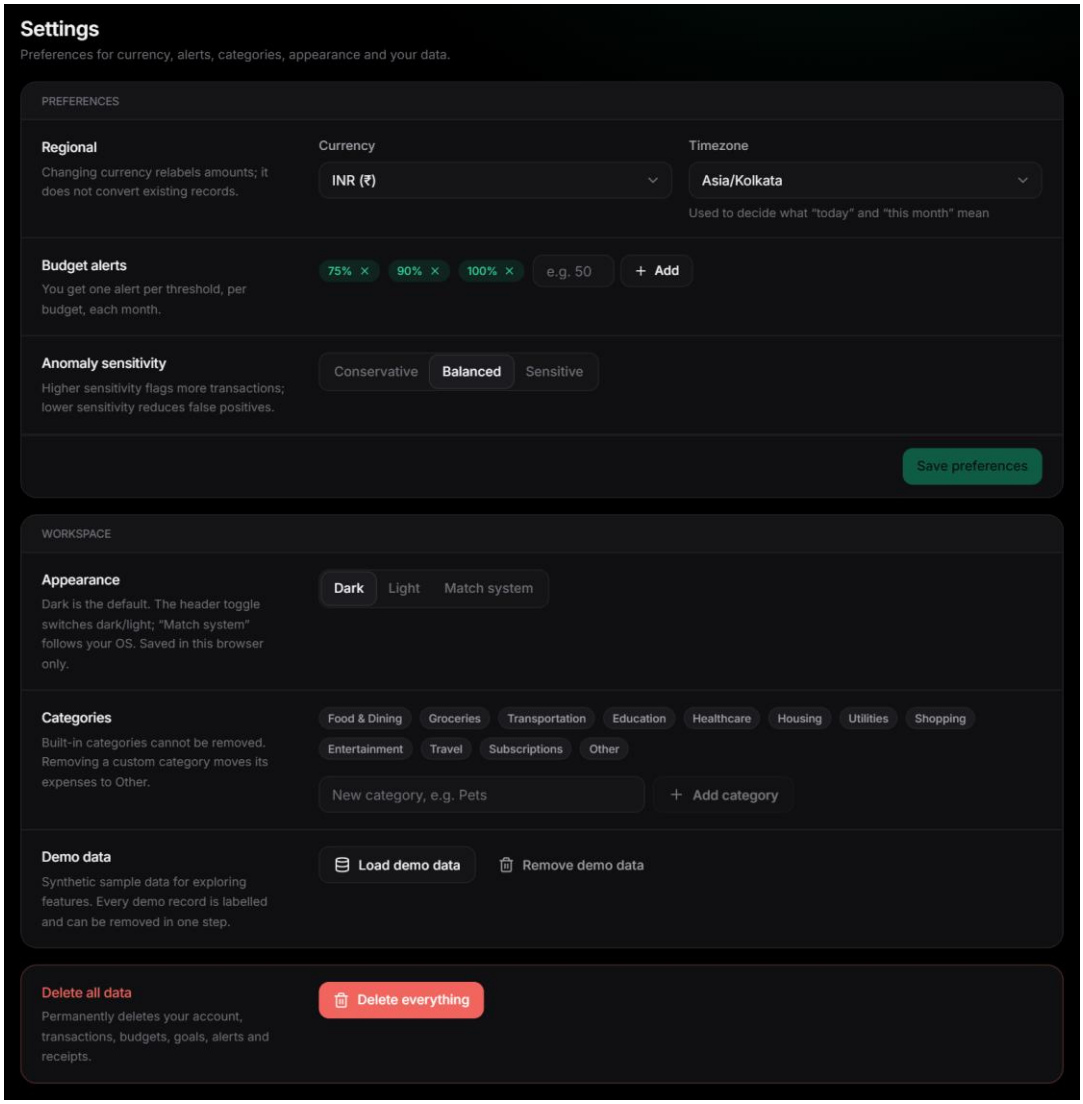


Figure 4: Settings page: preferences, appearance, categories, demo data and data deletion (cropped)

4.2 Expense Management

The core module records spending and keeps budgets, alerts and analytics consistent after every change. Table 6 specifies the module and Figure 5 shows its user interface.

Table 6: Module specification — Expense Management

Aspect	Description
Purpose	Create, read, update, delete, search, filter, sort, paginate and export expenses.
Inputs	Amount (string parsed to integer paise), date (not in the future, not before 2000), category, description, optional merchant, payment method and notes.
Processing	Strict validation (unknown fields rejected); persistence through the repository; after a change the budget threshold check and anomaly re-evaluation run, and their results are returned to the client.
Outputs	Expense record; paged list with total count and total amount over all matches; side-effect report (new alerts, threshold crossings).
Technologies	FastAPI, Pydantic, services/expenses.py; React Transactions, ExpenseEditor and ExpenseForm pages.
Endpoints	GET/POST /expenses; GET/PUT/DELETE /expenses/{id}.
Error handling	422 with field errors; 404 for missing or other users' ids; confirmation dialog before deletion.
Security	Every query is scoped to the authenticated user; CSV export is generated in the browser from the user's own data.
Limitations	Single currency per account; no income records.

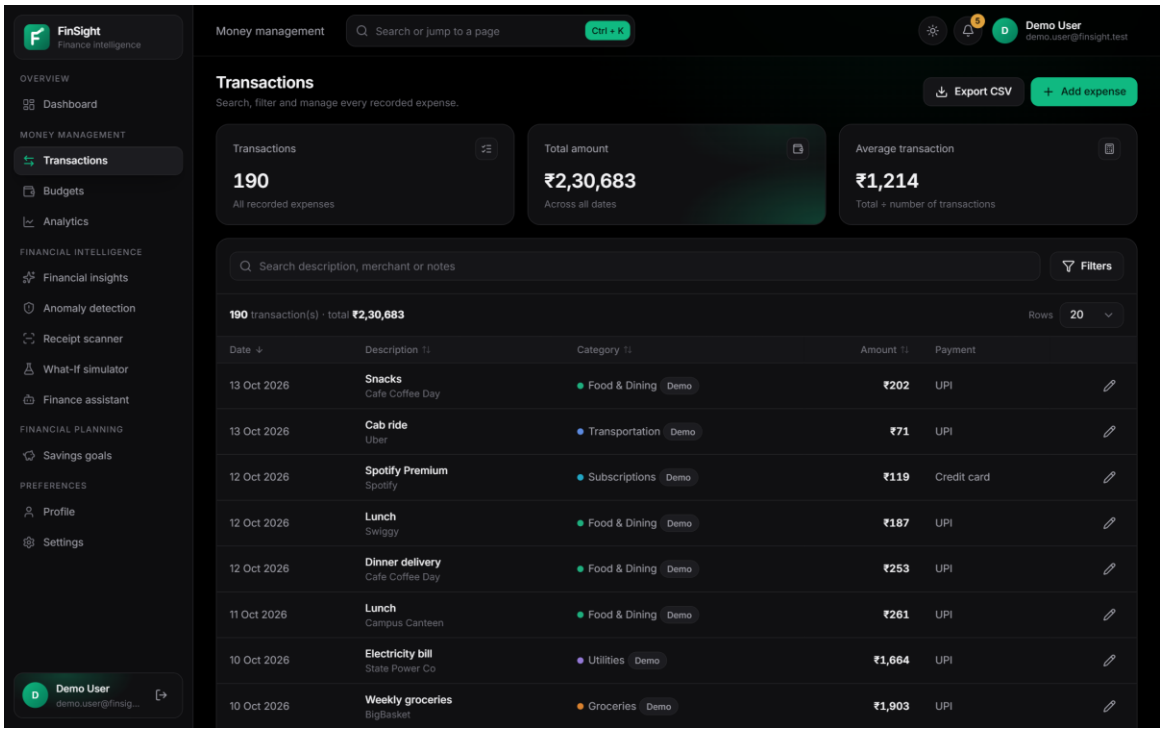


Figure 5: Transactions page with summary cards, search and sortable table

Figures 6 and 7 show the creation and editing forms (cropped to the form area); deleting an expense from its detail dialog (Figure 8) requires confirmation.

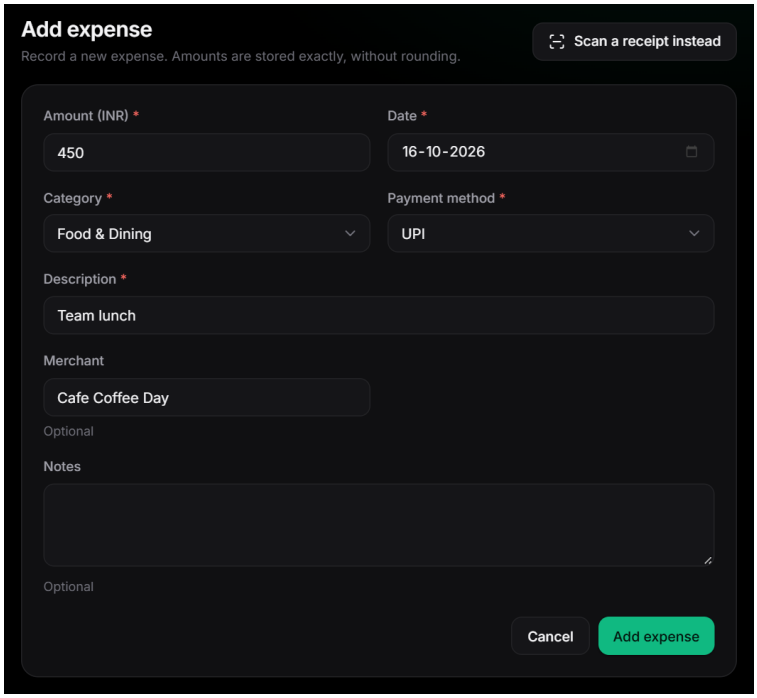


Figure 6: Add-expense form (cropped to the content area)

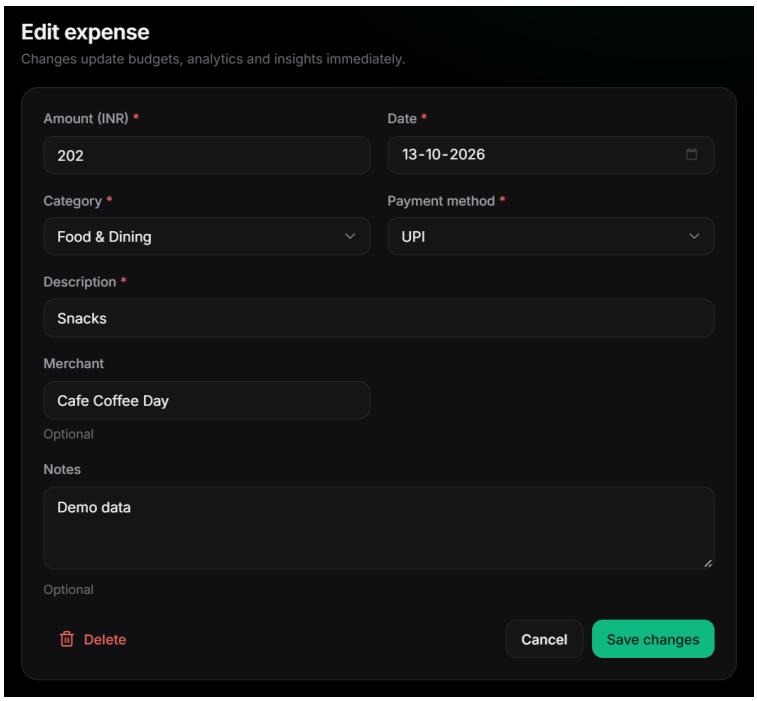


Figure 7: Edit-expense form with the Delete option (cropped to the content area)

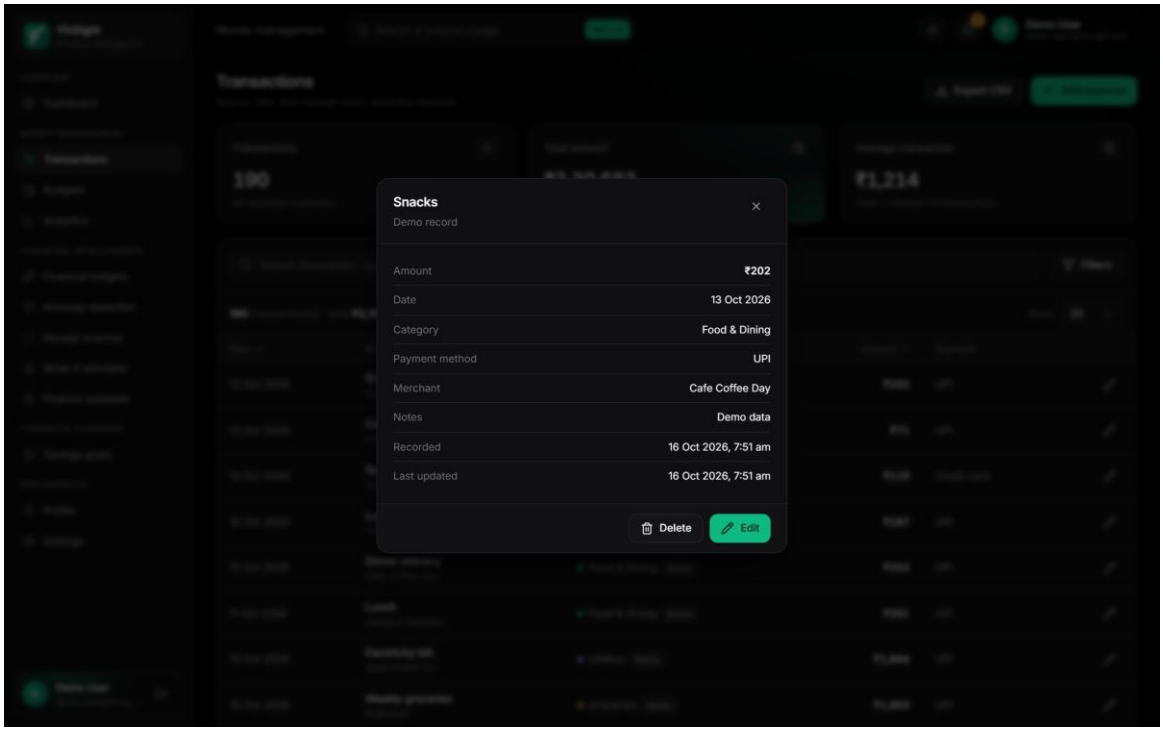


Figure 8: Expense detail dialog with Delete and Edit actions

4.3 Category and Payment-Method Management

Categories and payment methods give structure to expenses and drive category budgets, analytics and anomaly baselines. Table 7 specifies the module and Figure 9 shows its user interface.

Table 7: Module specification — Category and Payment-Method Management

Aspect	Description
Purpose	Provide 12 built-in categories and fixed payment methods, and allow custom categories.
Inputs	Custom category name; category and payment-method selections on expenses and filters.
Processing	Custom categories are stored in the user's preferences; removing one moves its expenses to “Other” and removes its budget.
Outputs	Reference lists for forms and filters; consistent category colours in charts.
Technologies	domain/categories.py, services/preferences.py, React Settings page.
Endpoints	GET /reference; POST /categories; DELETE /categories/{id}.
Error handling	Duplicate or blank names rejected; built-in categories cannot be removed.
Security	Custom categories are private to the user.
Limitations	Categorisation is manual (the receipt scanner only suggests a category).

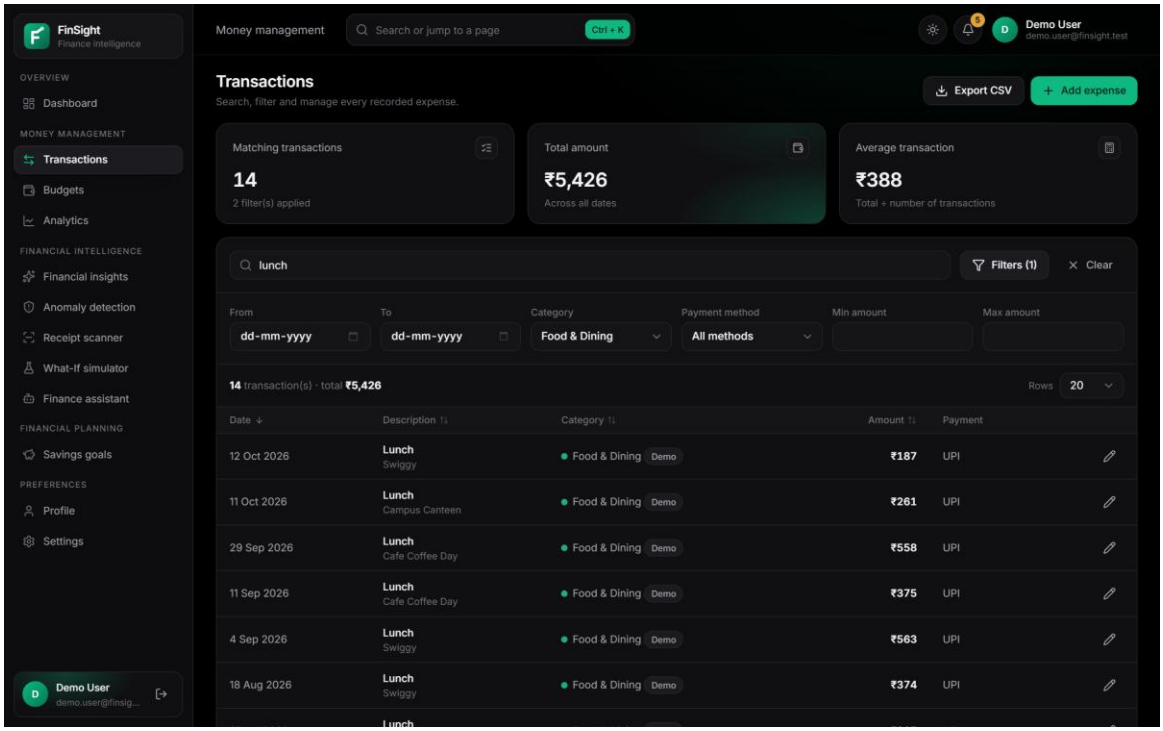


Figure 9: Expense filtering by text, category and payment method (14 matching transactions)

4.4 Budget Management and Alerts

Budgets turn recorded spending into early warnings. Table 8 specifies the module and Figure 10 shows its user interface.

Table 8: Module specification — Budget Management and Alerts

Aspect	Description
Purpose	Manage an overall monthly budget and category budgets, report utilization and send threshold alerts.
Inputs	Budget amount and optional category; user thresholds (default 75%, 90%, 100%).
Processing	Budgets are versioned by month (edits apply from the current month). Utilization = spent ÷ budget; status ok / warning / at limit / exceeded; projection = spent ÷ days elapsed × days in month (current month, after 3 days); alert events inserted only if absent for (user, month, budget, threshold).
Outputs	Budget overview, unbudgeted spending, alert events, six- or twelve-month history.
Technologies	services/budgets.py; Budgets page and dashboard budget card.
Endpoints	GET/POST /budgets; GET /budgets/history; PUT/DELETE /budgets/{id}.
Error handling	409 for a duplicate budget; past months read-only.
Security	User-scoped; alerts deduplicated with conditional inserts.
Limitations	Alerts appear in the app (toasts and lists); no e-mail or SMS notifications.

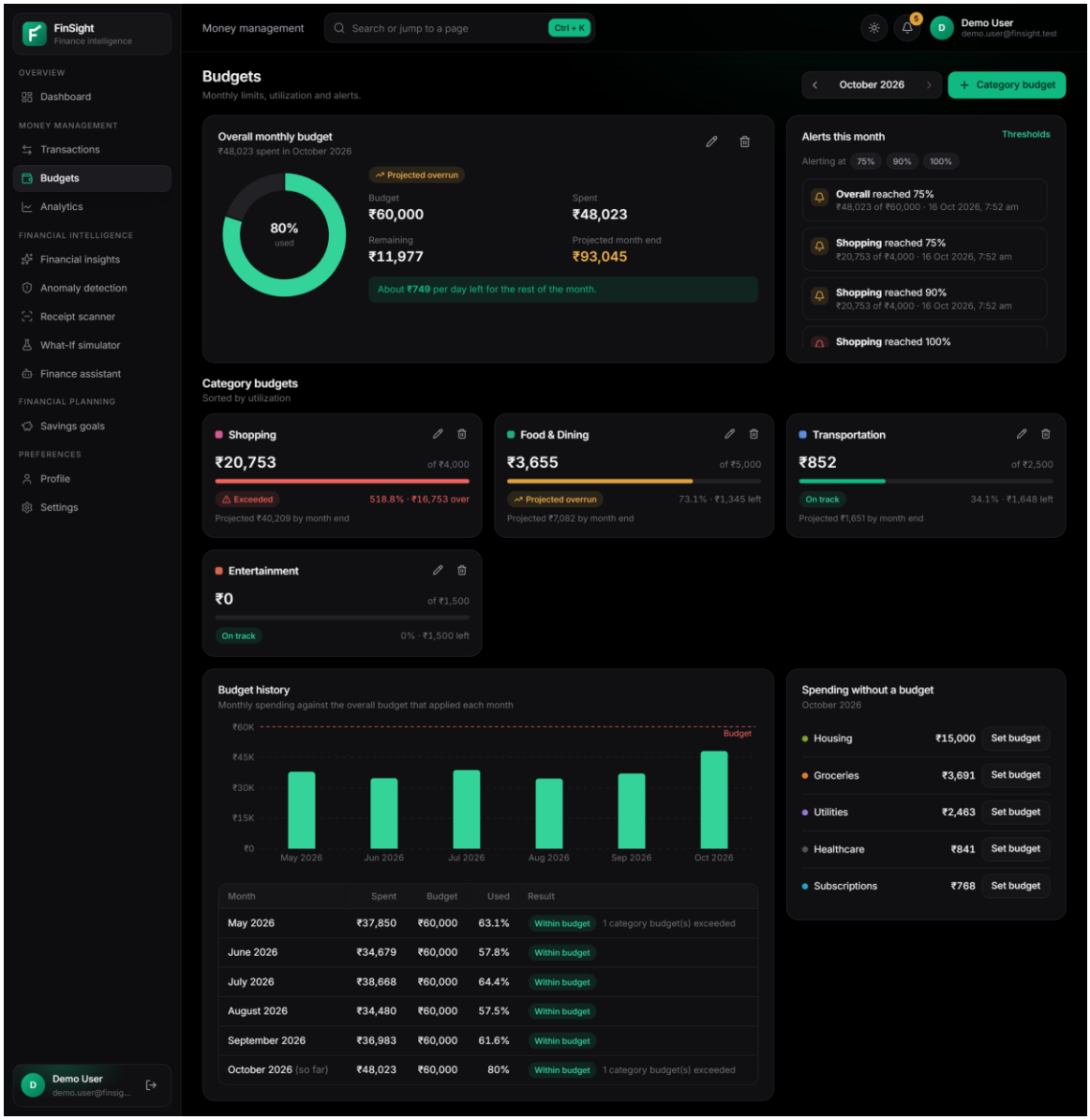


Figure 10: Budgets page: overall budget, alerts, category budgets and history

4.5 Financial Dashboard

The dashboard summarises the selected month in one view. Table 9 specifies the module and Figure 11 shows its user interface.

Table 9: Module specification — Financial Dashboard

Aspect	Description
Purpose	Give an at-a-glance overview: total spent, change versus a comparable period, daily average, goal savings, category split, budget status, trend, recent activity, insights and items needing attention.
Inputs	Selected month; the user's expenses, budgets, goals and alerts.
Processing	One composite request (GET /dashboard) aggregates the month; trend data can switch to six- or twelve-month history.
Outputs	KPI cards, donut chart, budget progress, area chart, activity list.
Technologies	services/analytics.py; React Dashboard page with Recharts.
Endpoints	GET /dashboard?month=YYYY-MM; GET /goals; GET /budgets/history.
Error handling	Empty-account onboarding state; error state with retry.
Security	Computed only from the user's data; no synthesised values.
Limitations	Comparisons need at least one earlier month of data.

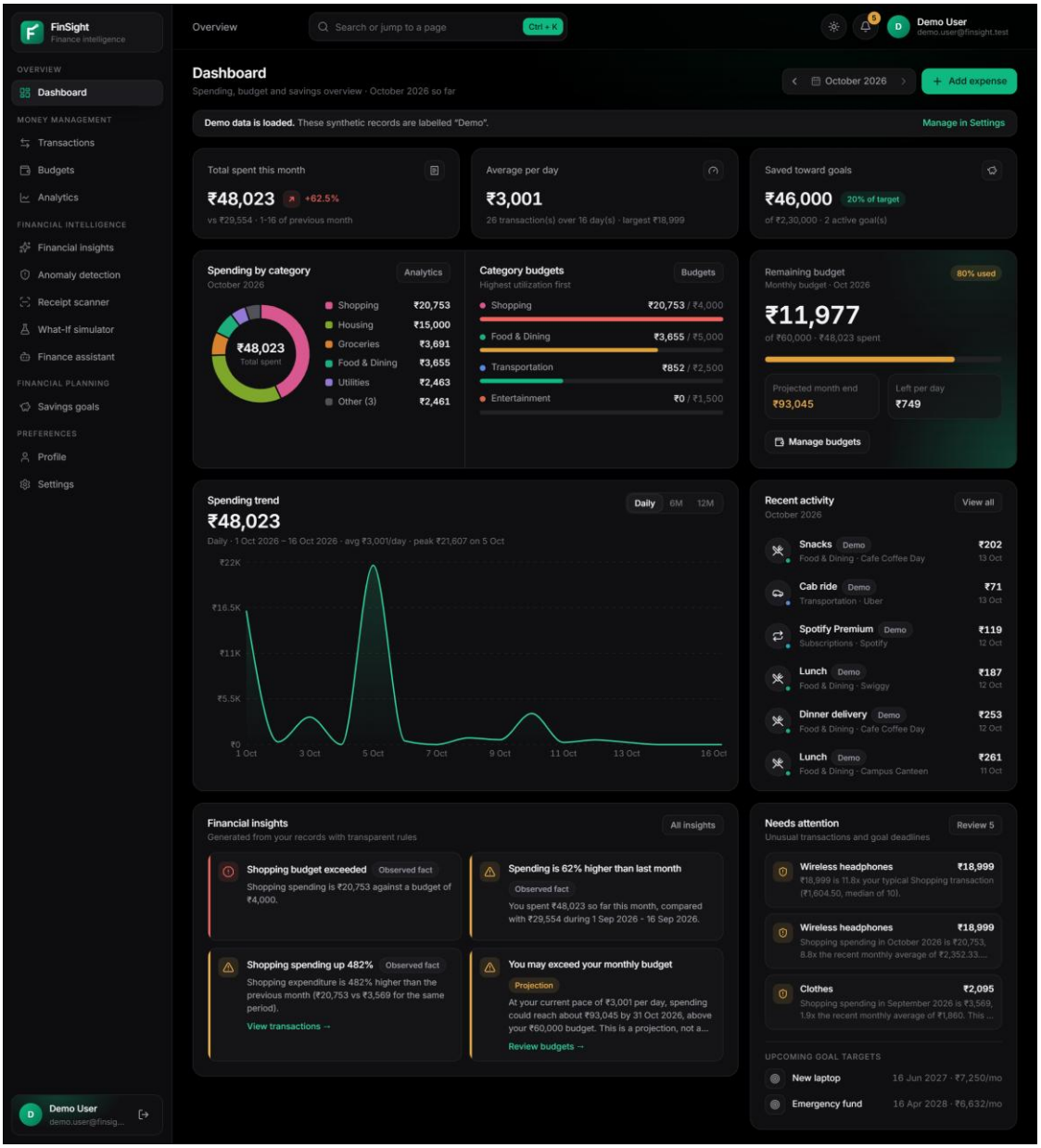


Figure 11: Dashboard in the dark theme (synthetic demo data, labelled "Demo")

The same layout in the light theme is shown in Figure 12; both themes share one set of design tokens.

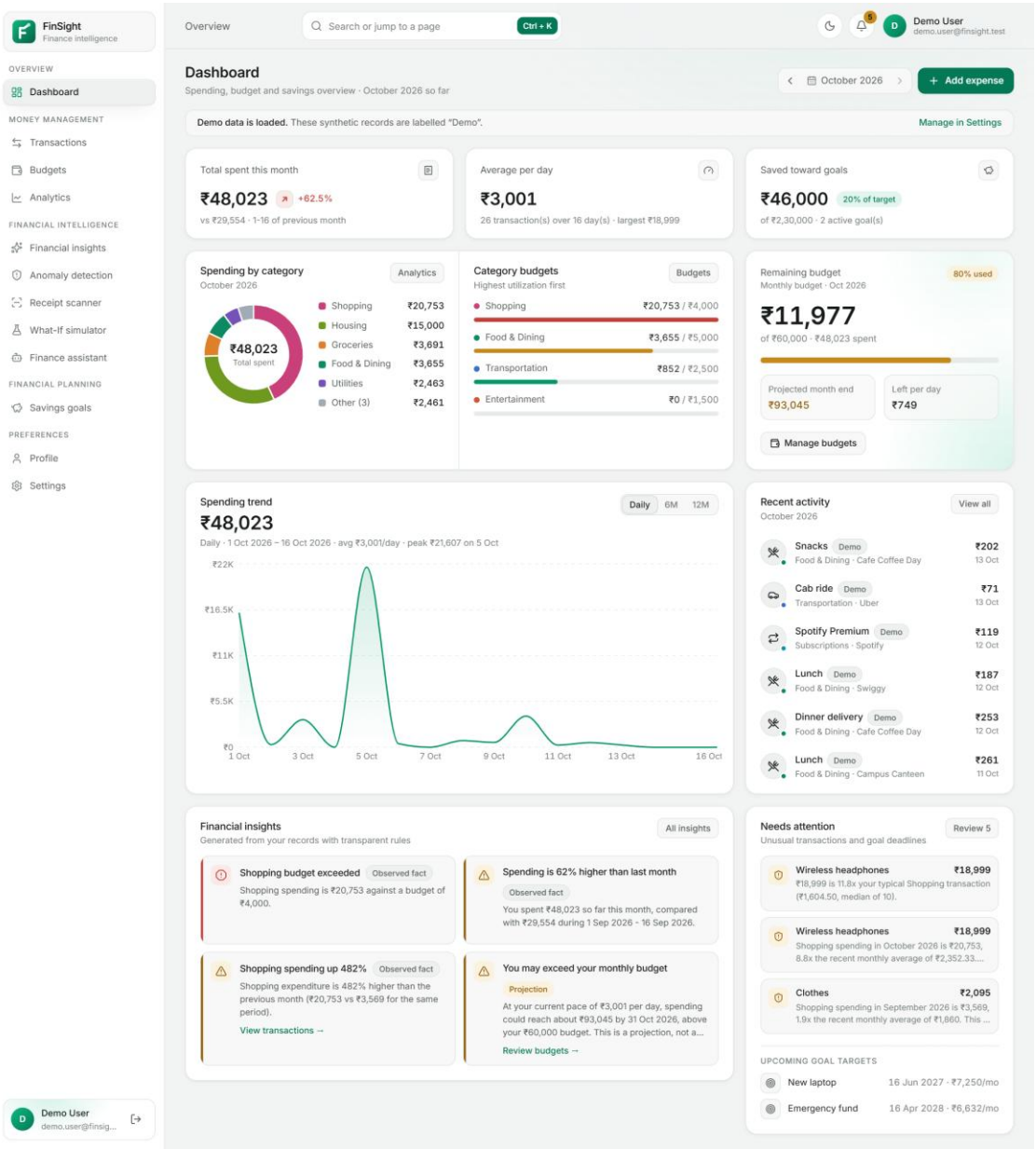


Figure 12: Dashboard in the light theme

4.6 Spending Analytics

The analytics page explores longer periods than the dashboard. Table 10 specifies the module and Figure 13 shows its user interface.

Table 10: Module specification — Spending Analytics

Aspect	Description
Purpose	Analyse spending over preset (30 days, this month, 3, 6, 12 months) or custom ranges.
Inputs	Start and end dates.
Processing	Totals, transaction count, daily and per-transaction averages, category shares, payment-method shares, top merchants, monthly stacked totals and daily series (ranges up to 120 days).
Outputs	KPI cards, stacked monthly bars with a table view, category donut and bars, daily trend, merchant table.
Technologies	services/analytics.py; Analytics page.
Endpoints	GET /analytics?start=&end=.
Error handling	Invalid ranges rejected in the UI and API.
Security	User-scoped queries.
Limitations	No income, so no savings rate.

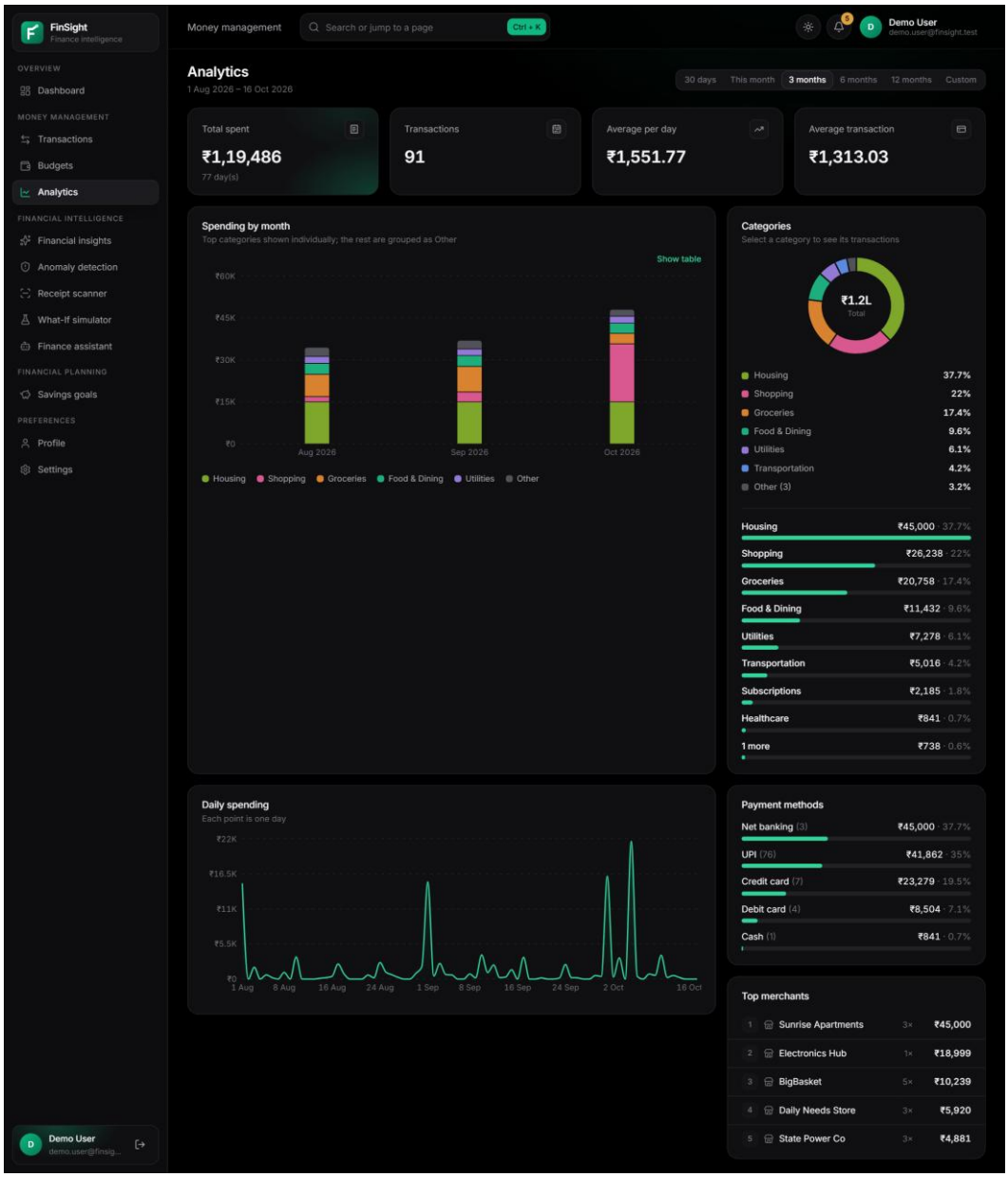


Figure 13: Analytics page (three-month range)

4.7 Personal Finance Intelligence Engine

The engine converts records into explained statements. Table 11 specifies the module and Figure 14 shows its user interface.

Table 11: Module specification — Personal Finance Intelligence Engine

Aspect	Description
Purpose	Generate insights about month-over-month change, category changes, concentration, averages, projections, recurring payments, goals and savings opportunities.
Inputs	Seven months of expenses, budgets, goals and preferences.
Processing	Deterministic rules (Section 7.4); every insight is labelled fact, estimate, projection, recommendation or data gap and carries period, formula, values and transactions.
Outputs	Ordered insight list with severity and optional action link; recurring-payment list.
Technologies	services/insights.py, services/recurring.py; InsightCard component.
Endpoints	GET /insights?month=YYYY-MM (also embedded in /dashboard).
Error handling	Insufficient history produces data-gap insights instead of claims.
Security	Derived on request; never stored, so never stale.
Limitations	Rule coverage is fixed; not personalised beyond the rules.

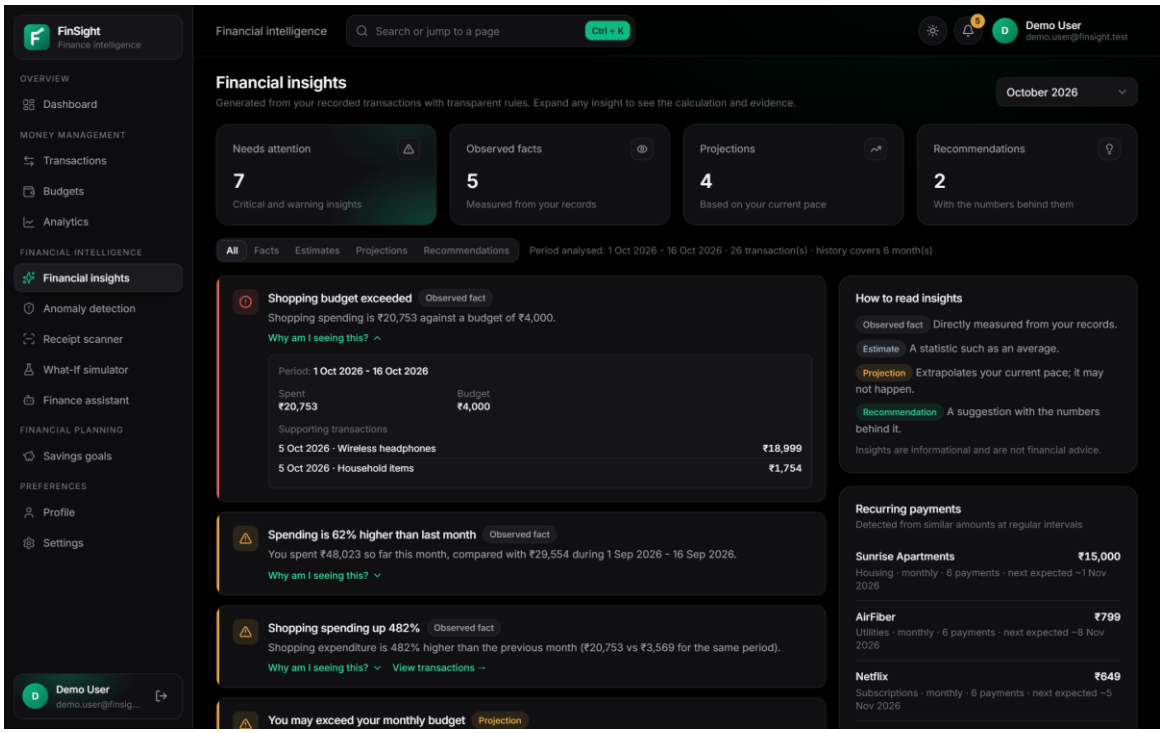


Figure 14: Financial insights with an expanded “Why am I seeing this?” evidence panel

4.8 Financial Anomaly Detection

This module highlights transactions that are unusual for the user. Table 12 specifies the module and Figure 15 shows its user interface.

Table 12: Module specification — Financial Anomaly Detection

Aspect	Description
Purpose	Flag unusually large amounts, possible duplicates, repeated similar transactions and category spikes for review.
Inputs	The new or edited expense and the user's last 365 days of history; sensitivity preset.
Processing	Modified z-score and ratio rules per category, overall large-amount rule, duplicate and repetition rules, category spike rule (Section 7.6); deduplication keys prevent re-creating reviewed alerts.
Outputs	Alerts with rule, severity, reason, comparison values and status (open, reviewed, legitimate, dismissed).
Technologies	services/anomalies.py; Anomaly detection page; header bell count.
Endpoints	GET /anomalies; POST /anomalies/scan; POST /anomalies/{id}/review.
Error handling	Small datasets are not judged statistically.
Security	Detection never modifies or deletes transactions and never labels fraud.
Limitations	Needs history; thresholds are conventions, not calibrated on labelled data.

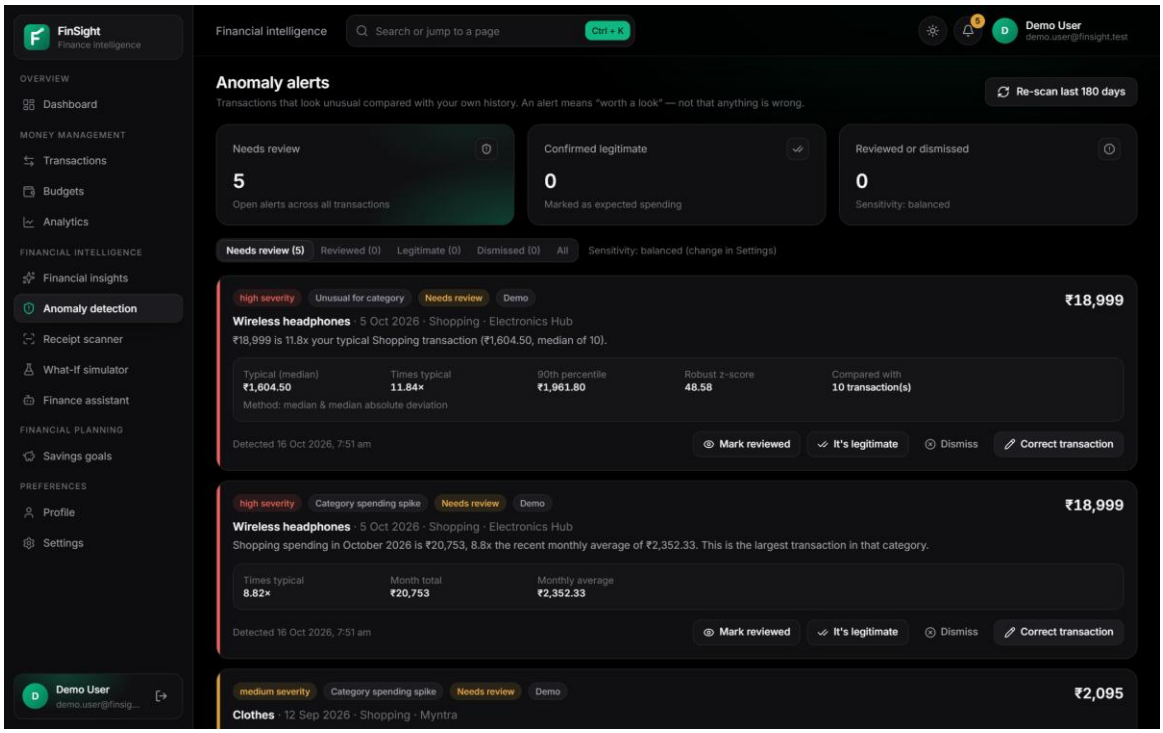


Figure 15: Anomaly alerts with reasons, comparison values and review actions

4.9 Smart Receipt Scanner

The scanner reduces manual entry while keeping the user in control. Table 13 specifies the module and Figure 16 shows its user interface.

Table 13: Module specification — Smart Receipt Scanner

Aspect	Description
Purpose	Read a receipt photo and pre-fill an expense for review.
Inputs	JPEG, PNG or WebP image up to 5 MB.
Processing	Validation (magic bytes, decode, 40-megapixel cap) → pre-processing → Tesseract (--oem 3 --psm 4) → rule-based parser with per-field confidence → human review → confirmation.
Outputs	Extracted fields with confidence and warnings; on confirmation, a new expense linked to the receipt.
Technologies	Pillow, pytesseract, Tesseract 5, services/receipts/*; Receipt scanner page.
Endpoints	GET /receipts/status; POST /receipts; POST /receipts/upload-url; POST /receipts/{id}/process; POST /receipts/{id}/confirm.
Error handling	Unsupported or spoofed types, oversize and corrupt images rejected; unknown fields left blank; currency mismatch warned.
Security	Server-generated storage keys; client file names ignored; private storage.
Limitations	Printed English receipts work best; no PDFs; heuristic parsing.

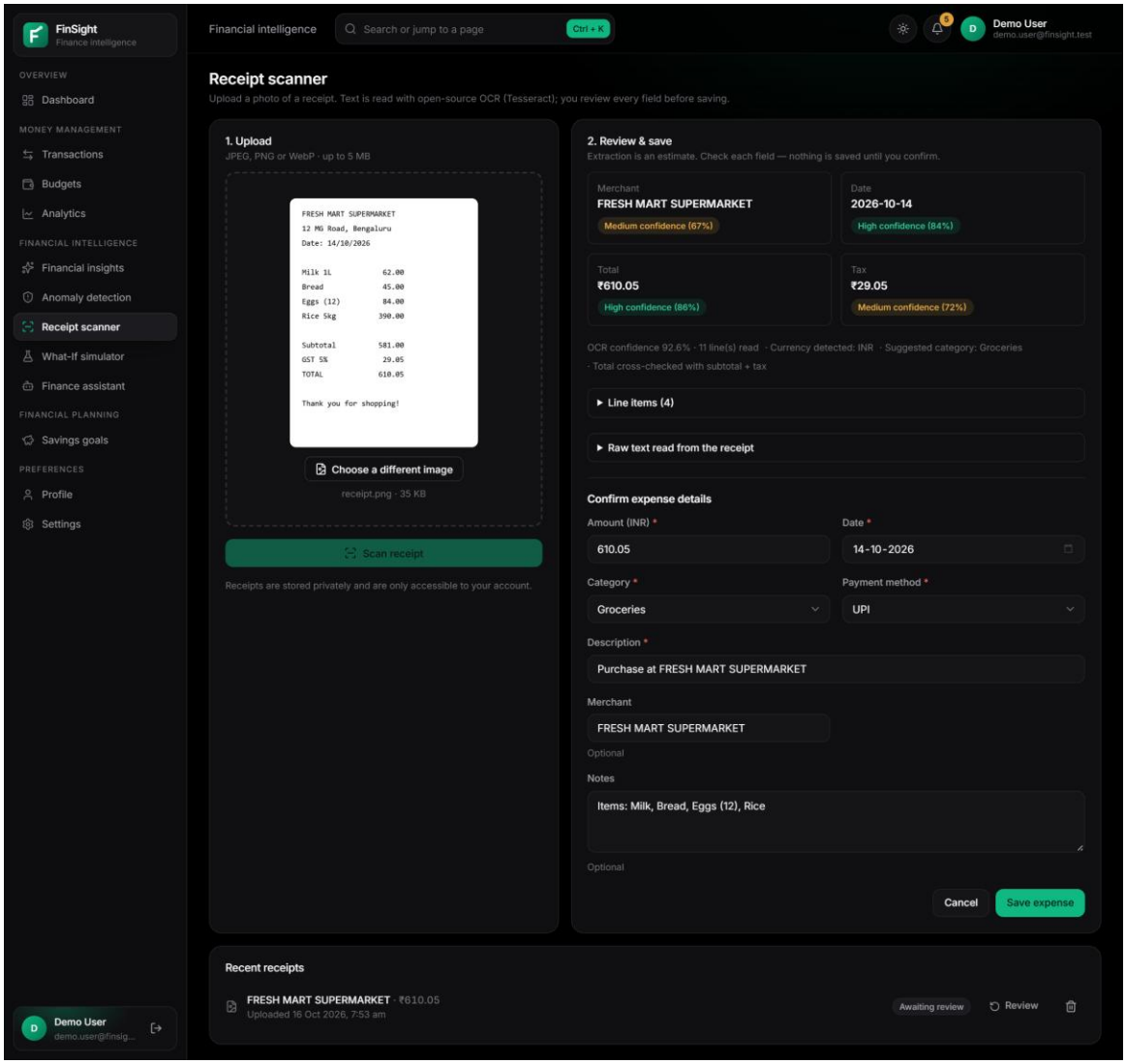


Figure 16: Receipt scanner after genuine Tesseract OCR of a generated sample receipt: extracted fields with confidence and the review form

4.10 What-If Financial Simulator

The simulator answers “what would happen if …” without touching real data. Table 14 specifies the module and Figure 17 shows its user interface.

Table 14: Module specification — What-If Financial Simulator

Aspect	Description
Purpose	Compare up to four hypothetical scenarios of category spending changes, and compute the saving needed for a goal.
Inputs	Baseline (last month, average of 3 months, or this month's projection); per-category adjustments (reduce by %, by amount, or set amount); optional goal.
Processing	Section 7.8; results recalculate automatically after a 300 ms pause.
Outputs	Current versus new monthly spending, monthly and annual savings (annual = monthly × 12), budget and goal impact, written explanation.
Technologies	services/simulator.py; Simulator page.
Endpoints	POST /simulations; POST /simulations/required-savings.
Error handling	Invalid percentages and duplicate categories rejected; reductions capped at current spending with a warning.
Security	No side effects (verified by tests).
Limitations	Assumes the baseline pattern continues; no income.

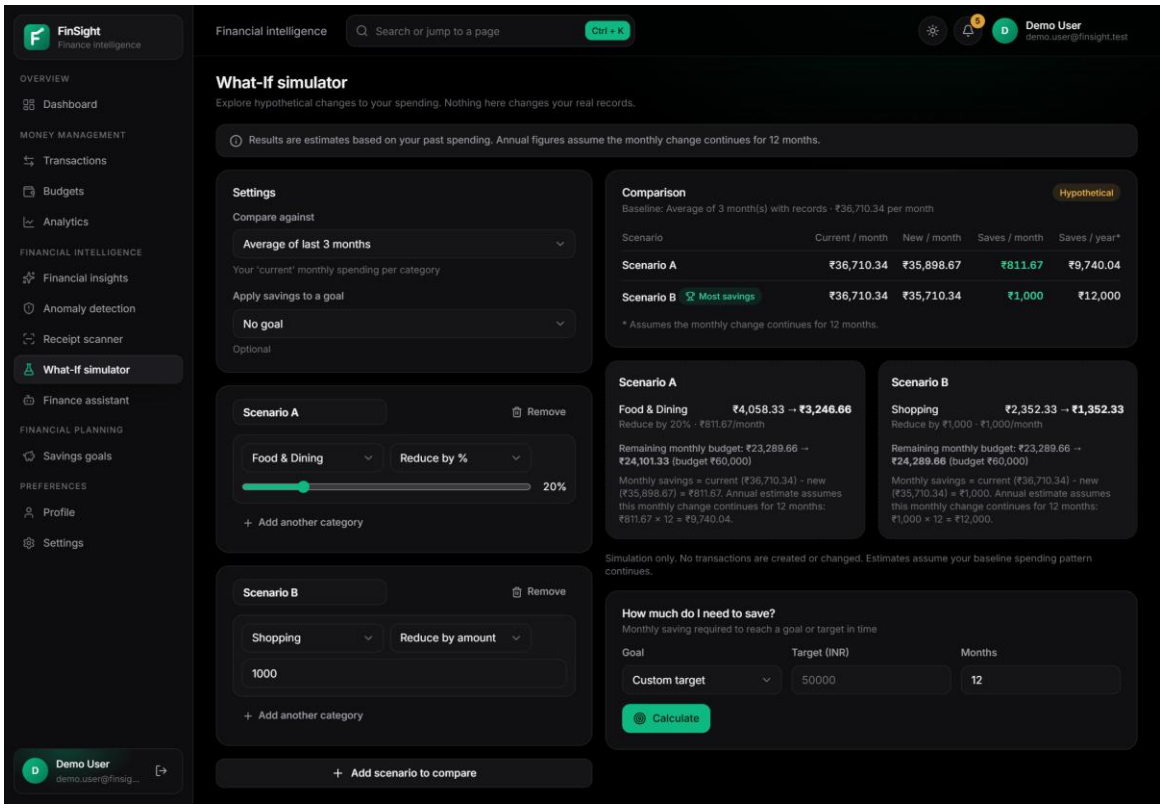


Figure 17: What-If simulator comparing two scenarios (results marked “Hypothetical”)

4.11 Conversational Finance Assistant

The assistant answers common questions in plain language. Table 15 specifies the module and Figure 18 shows its user interface.

Table 15: Module specification — Conversational Finance Assistant

Aspect	Description
Purpose	Answer questions such as “How much did I spend on food last month?” from the user's records.
Inputs	Question text (up to 500 characters).
Processing	Normalisation → date resolution → entity extraction → scoring over 14 intents → deterministic handler (Section 7.9). No large language model is used.
Outputs	Answer text, structured data (transactions, breakdowns, comparisons), resolved period, follow-up suggestions, disclaimer.
Technologies	services/assistant/nlu.py and engine.py; Assistant page.
Endpoints	POST /assistant/query.
Error handling	Ambiguous dates trigger a clarifying question; unsupported topics are explained.
Security	User text only selects an intent; it is never executed; data isolation tested.
Limitations	Fixed intents and vocabulary; English only; conversation kept only in the browser tab.

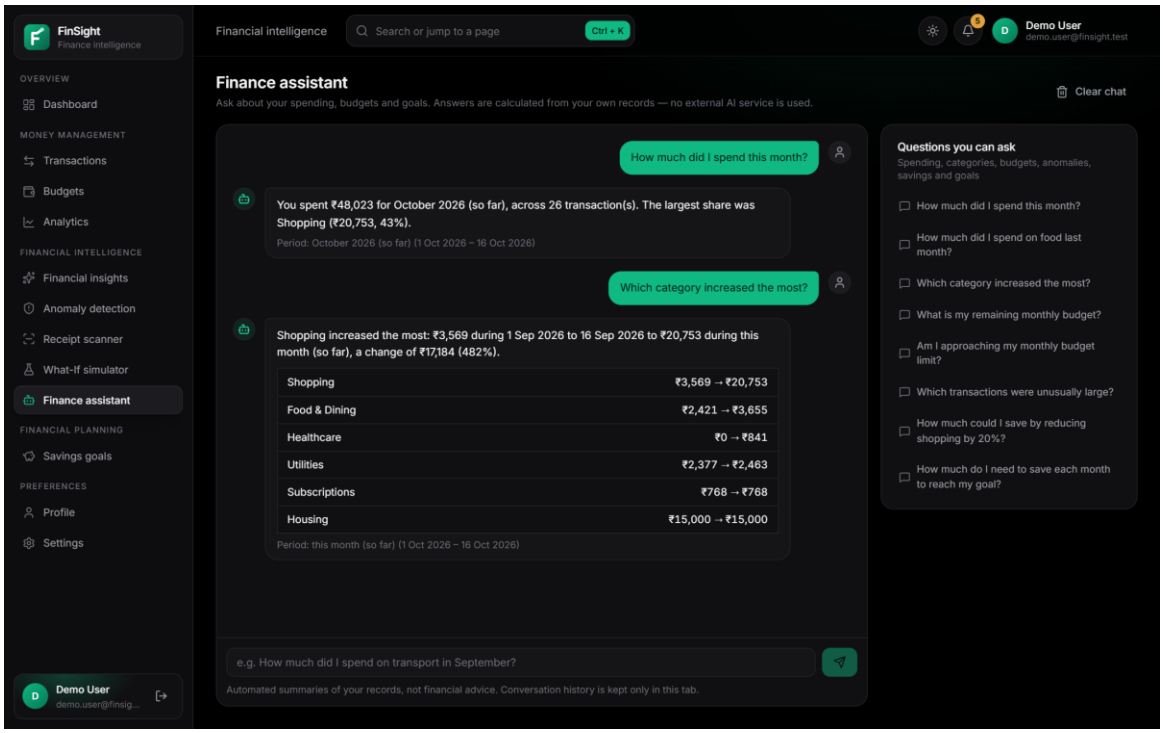


Figure 18: Finance assistant answering two questions with tabular evidence

4.12 Savings Goals

Goals connect spending decisions to targets. Table 16 specifies the module and Figure 19 shows its user interface.

Table 16: Module specification — Savings Goals

Aspect	Description
Purpose	Track named savings targets, contributions and withdrawals.
Inputs	Name, target amount, optional target date and starting balance; contributions (positive) and withdrawals (negative).
Processing	Remaining amount, progress %, required monthly saving (remaining ÷ months left), 90-day contribution pace, status (on track, behind, overdue, completed, not started, archived) and projected completion.
Outputs	Goal cards, contribution history, totals.
Technologies	services/goals.py; Goals page; dashboard KPI.
Endpoints	GET/POST /goals; GET/PUT/DELETE /goals/{id}; POST /goals/{id}/contributions; DELETE /goals/{id}/contributions/{cid}.
Error handling	Over-withdrawal and invalid amounts rejected.
Security	Expenses are never counted as savings.
Limitations	Contributions are recorded manually; no bank link.

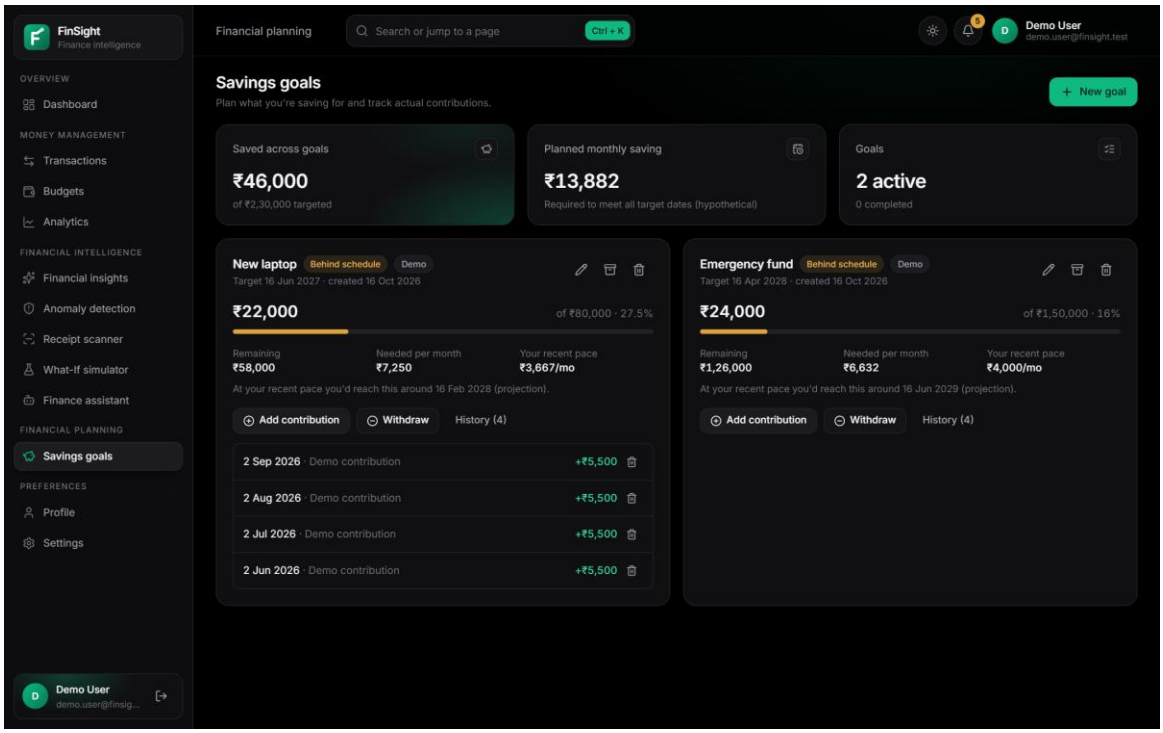


Figure 19: Savings goals with progress, required monthly saving and contribution history

4.13 Receipt Storage

Receipt images are stored privately. Locally they live in backend/data/receipts, outside any static web directory, and are streamed only to their owner with Cache-Control: private, no-store. In the AWS configuration the browser requests a presigned POST (valid 5 minutes, restricted to 1 byte–5 MB and a fixed content type with server-side encryption), uploads directly to the private bucket, and later displays the image through a presigned GET URL valid for 2 minutes. Storage keys have the form <user_id>/<32-hex-id>.<ext>, are generated by the server and validated against a strict pattern, so client file names and path traversal cannot influence storage. Figure 20 shows the upload panel and its privacy note.

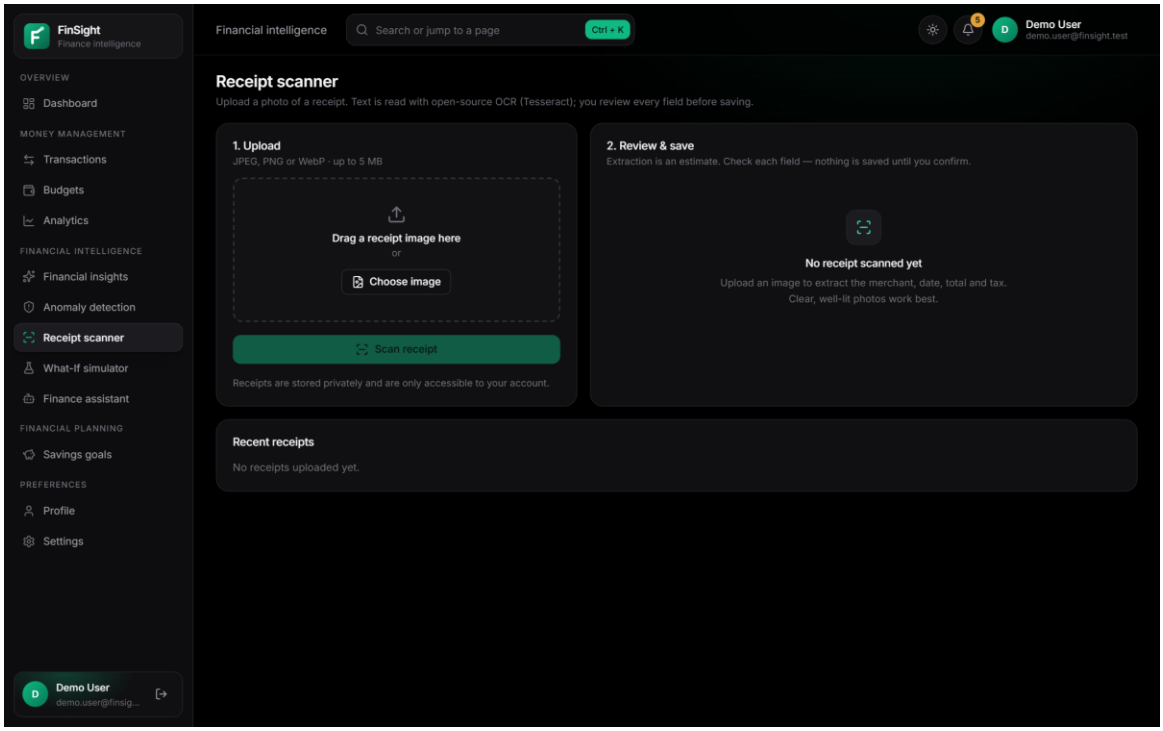


Figure 20: Receipt upload panel (local storage mode)

4.14 Backend API

The backend exposes 48 REST endpoints under /api/v1, summarised in Table 17 (derived from the route modules in backend/app/api/routes and app/main.py). Requests and responses are JSON (multipart for direct receipt upload); amounts are integer minor units in fields ending _minor; errors use one envelope with a code, message and optional field errors.

Table 17: API summary (paths relative to /api/v1)

Method(s)	Path(s)	Token required	Purpose
GET	/health, /config	No	Health check; public configuration (auth mode, Cognito ids, upload mode)
POST	/auth/register, /auth/login	No	Local-mode registration and sign-in (route group disabled in Cognito mode)
POST	/auth/change-password	Yes	Change password (local mode)
GET, POST	/expenses	Yes	List with search/filters/sort/pagination; create
GET, PUT, DELETE	/expenses/{id}	Yes	Read, update, delete one expense
GET, POST	/budgets	Yes	Month overview; create budget
GET	/budgets/history	Yes	Monthly history (6 or 12 months)
PUT, DELETE	/budgets/{id}	Yes	Update or remove a budget (from the current month)
GET	/dashboard, /analytics, /insights	Yes	Composite dashboard, range analytics, insights
GET	/anomalies	Yes	Alerts by status with counts

Method(s)	Path(s)	Token required	Purpose
POST	/anomalies/scan, /anomalies/{id}/review	Yes	Re-scan 180 days; set review state
GET	/receipts/status, /receipts, /receipts/{id}, /receipts/{id}/image	Yes	OCR availability; list; details; image (stream or presigned URL)
POST	/receipts, /receipts/upload-url	Yes	Direct upload (local) or presigned POST (S3)
POST	/receipts/{id}/process, /receipts/{id}/confirm	Yes	Run OCR; confirm reviewed fields as an expense
DELETE	/receipts/{id}	Yes	Delete receipt and image
GET, POST	/goals	Yes	List goals with summary; create goal
GET, PUT, DELETE	/goals/{id}	Yes	Read, update/archive, delete goal
POST, DELETE	/goals/{id}/contributions[/{cid}]	Yes	Add contribution or withdrawal; remove entry
POST	/simulations, /simulations/required-savings	Yes	Run scenarios; saving required for a goal or target
POST	/assistant/query	Yes	Answer a natural-language question
GET	/me, /reference	Yes	User and preferences; categories, payment methods, currencies
PUT	/preferences	Yes	Update currency, time zone, thresholds, sensitivity, display name
POST, DELETE	/categories, /categories/{id}	Yes	Add or remove a custom category
POST, DELETE	/demo-data	Yes	Load or remove labelled demo data (disabled in production)
POST	/account/delete-data	Yes	Delete all of the user's data (local mode also deletes the account)

4.15 Local Database (SQLite)

Locally, persistence uses SQLite in write-ahead-logging mode. Each table stores indexed lookup columns plus a JSON column with the full record (Table 18). Every query includes WHERE user_id = ?, updates are guarded so a row cannot change owner, and uniqueness for alert events and anomaly deduplication is enforced with constraints and INSERT OR IGNORE.

Table 18: SQLite schema

Table	Key / index columns	Purpose
users	id (PK), email (UNIQUE)	Local-mode accounts (scrypt hash)
preferences	user_id (PK)	Currency, time zone, thresholds, sensitivity, custom categories
expenses	id (PK); INDEX (user_id, date)	Expense records
budgets	id (PK); INDEX (user_id)	Versioned budgets
budget_alert_events	PK (user_id, month, budget_key, threshold)	One alert per threshold per month
goals	id (PK); INDEX (user_id)	Goals with embedded contributions
anomaly_alerts	id (PK); UNIQUE (user_id, dedup_key)	Alerts and review state
receipts	id (PK); INDEX (user_id)	Receipt metadata and extracted fields

4.16 Cloud Database (Amazon DynamoDB)

The DynamoDB repository uses a single on-demand table with partition key PK = USER#<uid> and sort keys such as EXPENSE#<id>, BUDGET#<id> and GOAL#<id>. A global secondary index GSI1 (USER#<uid>#EXPENSE, <date>#<id>) serves date-range queries. Uniqueness uses conditional writes (alert events) and TransactWriteItems (anomaly deduplication); no Scan is used and the IAM policy does not grant it. The repository passed the full API test suite against moto, a local mock of AWS services, and runs on the deployed on-demand table (Figure 49), where the live expense, budget and isolation tests passed.

4.17 AWS Authentication and Authorization

Figure 21 shows the Cognito flow as deployed. The browser signs in directly with Cognito, sends the access token to API Gateway, whose JWT authorizer validates issuer and client before Lambda runs; the backend validates the token again and scopes every query to the token's subject. The backend also calls Cognito GetUser with the token, so an access token revoked by sign-out is rejected. This defence in depth was verified on the live stack: requests without a token or with a forged token received 401, another user's resources returned 404, and a token used after sign-out received 401 (token_revoked).

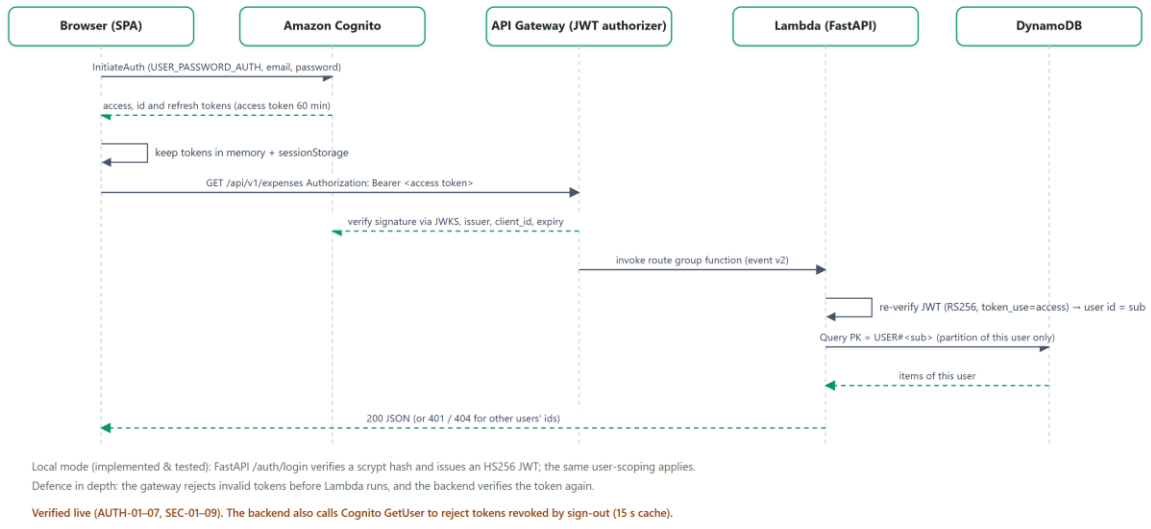


Figure 21: Authentication and authorization sequence (Cognito mode, as deployed)

4.18 Cloud Monitoring and Error Handling

The API returns one error envelope (code, message, field errors) and never exposes stack traces; unexpected exceptions return a generic 500 message. Request logs contain only method, path, status, duration and a request id — no bodies, query strings, tokens or receipt text. In the deployment, each of the nine Lambda functions has a CloudWatch log group with 14-day retention, and an alarm fires when the HTTP API returns five or more

5xx responses in five minutes. The frontend shows explicit error, empty and service-unavailable states instead of substituting data.

4.19 Module Interaction and Data Flow

Figure 22 shows how modules depend on each other: a confirmed receipt creates an expense; every expense change triggers the budget threshold check and anomaly re-evaluation; analytics, insights, the simulator and the assistant read expenses, budgets and goals; and every module persists through the user-scoped repository. Figure 23 presents the same system as a level-1 data flow diagram.

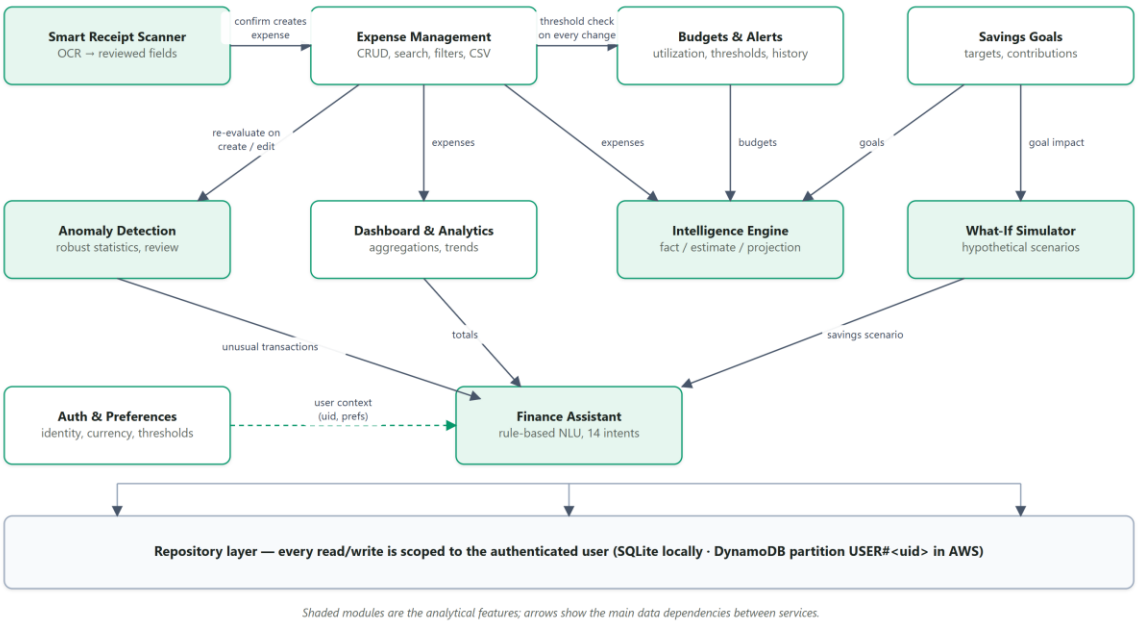


Figure 22: Module interaction diagram

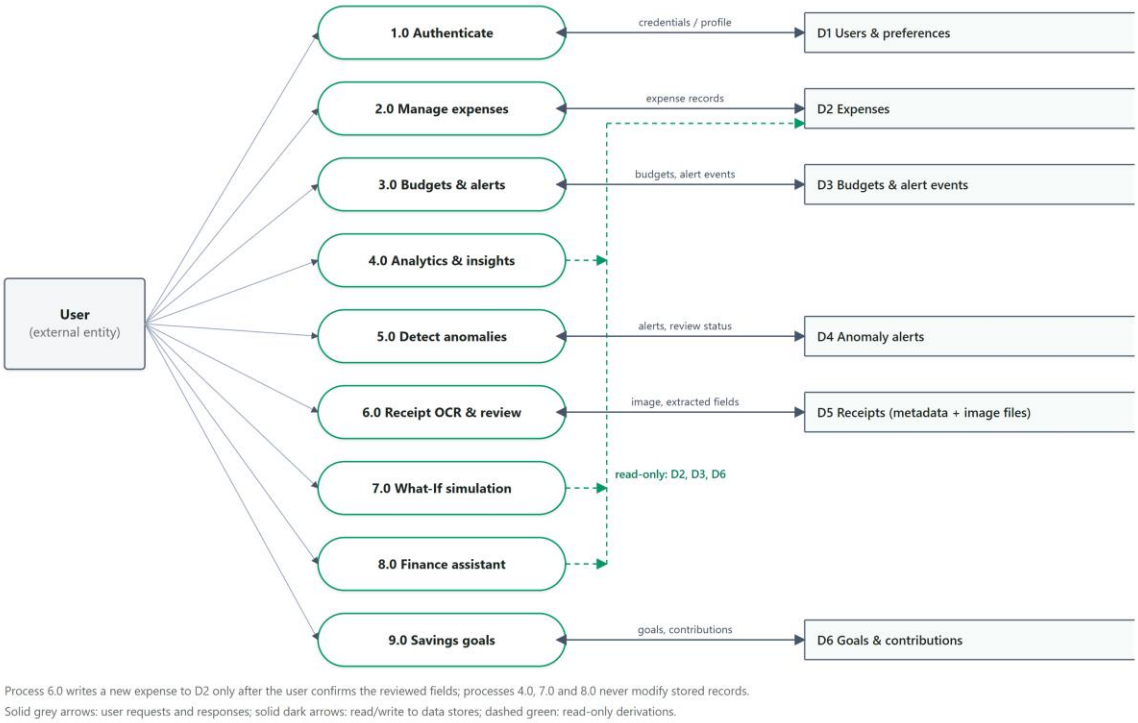


Figure 23: Level-1 data flow diagram

4.20 Data Model and Entity Relationship Diagram

Figure 24 is derived from backend/app/domain/models.py. Every entity carries a user_id; goal contributions are embedded in the goal record; insights are derived on request and are not stored.

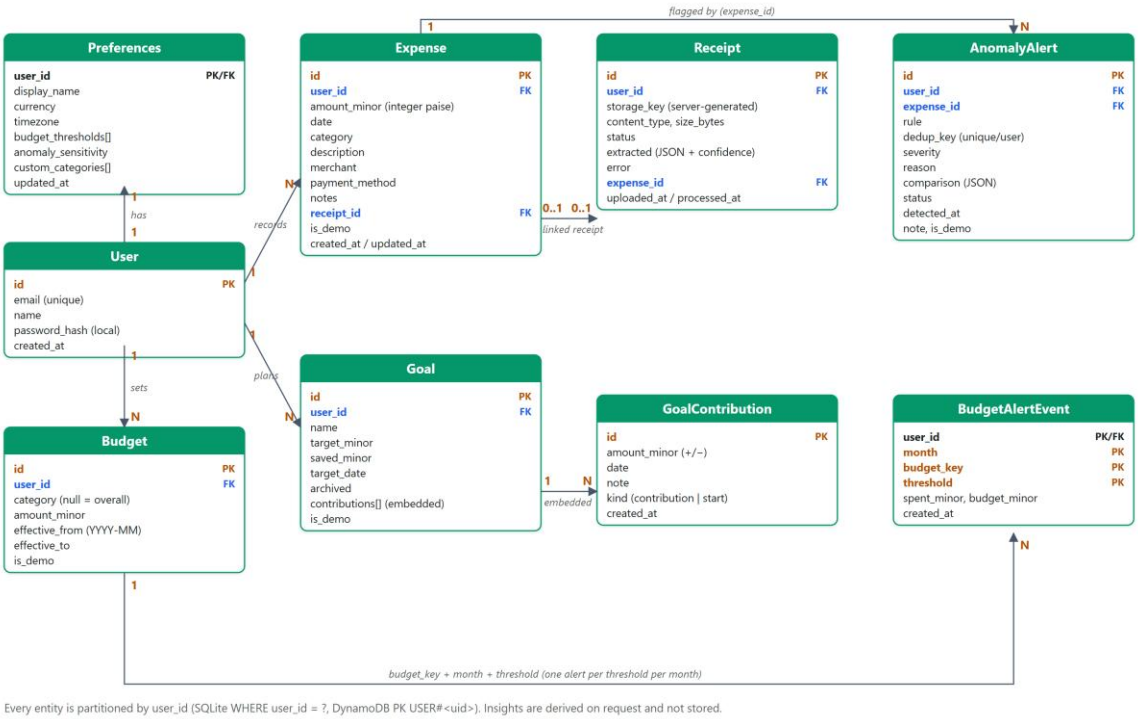


Figure 24: Entity relationship diagram of the logical data model

5. SYSTEM DESIGN AND USER WORKFLOW

5.1 Primary User Workflow

Figure 25 shows the main journey through FinSight, and Table 19 maps each step to the screen that implements it. Steps after sign-in are served by authenticated, user-scoped API calls.

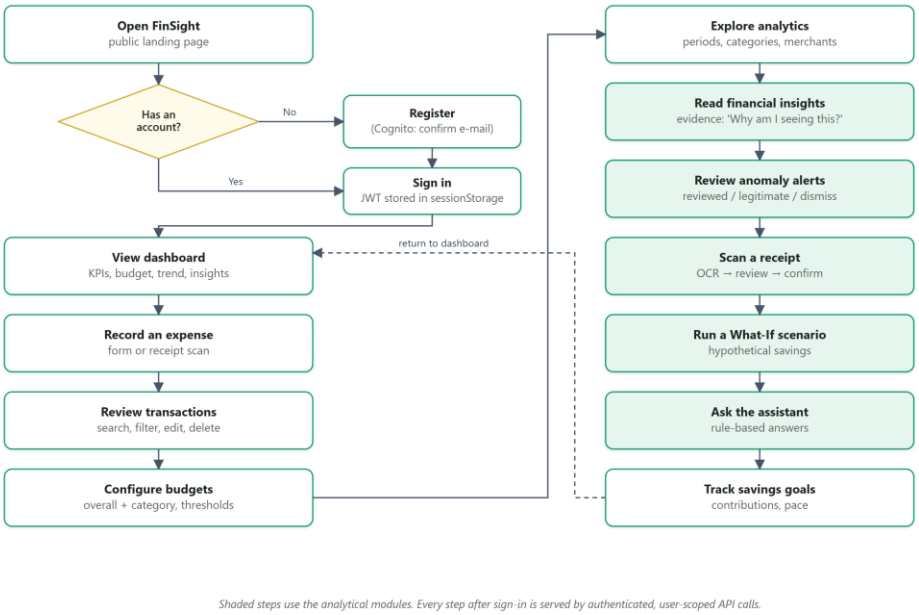


Figure 25: Primary user workflow

Table 19: User workflow steps and corresponding screens

Step	User action	Screen / evidence
1	Open FinSight (public landing page)	Figure 26
2	Register or sign in	Figure 2
3	Review the dashboard	Figures 11 and 12
4	Record an expense (form or receipt)	Figure 6
5	Review, search, filter, edit or delete expenses	Figures 5, 9 and 8
6	Configure budgets and thresholds	Figure 10
7	Explore analytics	Figure 13
8	Read financial insights and their evidence	Figure 14
9	Review anomaly alerts	Figure 15
10	Scan a receipt and confirm extracted data	Figures 20 and 16
11	Explore a hypothetical scenario	Figure 17
12	Ask the finance assistant	Figure 18
13	Track savings goals	Figure 19

5.2 Public Landing Page

The landing page (Figure 26) introduces the product without requiring an account; its example insight is marked as an illustration and contains no user data. The footer credits

the author (“Created by Aakar Gupta”), and an information button in the header opens an accessible About the author dialog (Figures 40 and 41); Settings links to the same dialog.

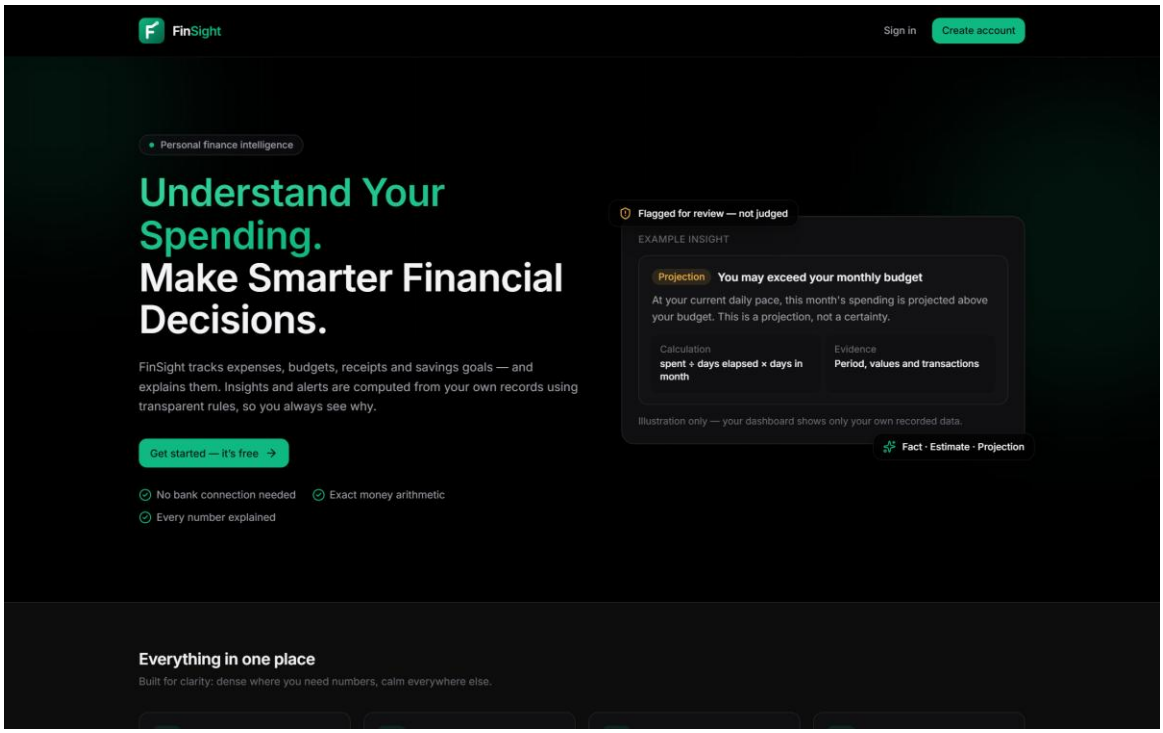


Figure 26: Public landing page

5.3 Responsive Design

Below 1024 px the sidebar becomes a drawer and dashboard cards stack (Figure 27). Scripted checks at 390 px and 1440 px found no horizontal overflow on the tested pages in either theme.

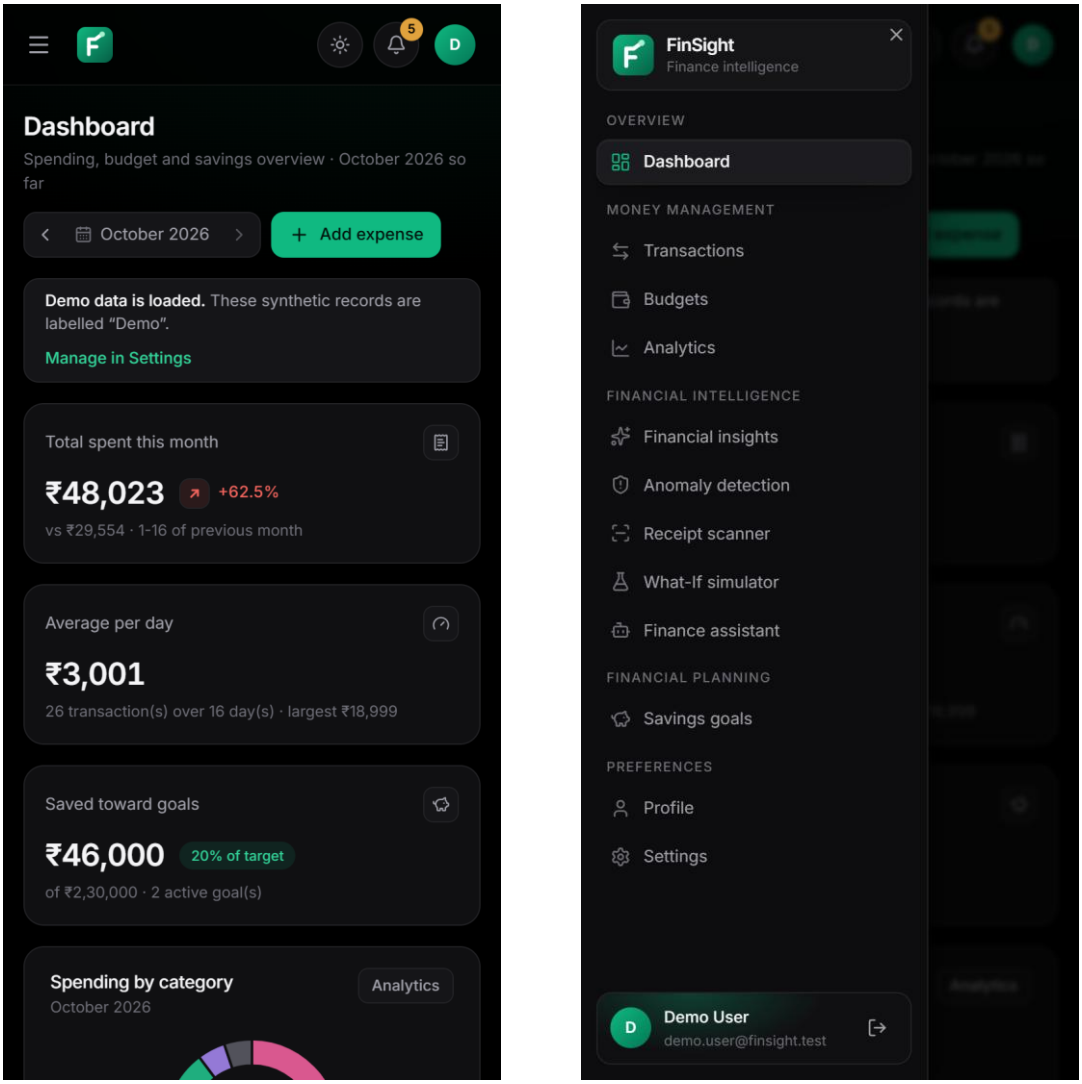


Figure 27: Mobile layout at 390 px: dashboard (left) and navigation drawer (right)

5.4 Requirements Traceability

Table 20: Requirements traceability matrix

Objective	Module(s)	Repository evidence	Verification
O1 Expenses	Expense management	services/expenses.py, routes/expenses.py, pages/Transactions.tsx	test_expenses.py (6), browser workflow
O2 Budgets	Budgets	services/budgets.py	test_budgets.py (10)
O3 Dashboard & analytics	Dashboard, analytics	services/analytics.py, pages/Dashboard.tsx, Analytics.tsx	API tests; browser checks
O4 Insights	Intelligence engine	services/insights.py, recurring.py	test_insights.py (8)
O5 Anomalies	Anomaly detection	services/anomalies.py	test_anomalies.py (13)
O6 Receipt OCR	Receipt scanner	services/receipts/ (ocr, parser, storage)	test_receipts.py (24) incl. real OCR
O7 Simulation	What-If simulator	services/simulator.py	test_simulator_and_goals .py (12)
O8 Assistant	Assistant	services/assistant/nlu.py, engine.py	test_assistant.py (19)
O9 Goals	Savings goals	services/goals.py	test_simulator_and_goals .py
O10 Security	Auth, all modules	auth/tokens.py, api/deps.py, repositories/*	test_security.py (11); live SEC and ISO tests
O11 Serverless AWS	AWS deployment	infra/template.yaml, handlers/, infra/web/handler.py, dynamodb_repo.py	cfn-lint; sam validate; AWS-path tests (32); deployed; live tests 64 of 64 pass

6. IMPLEMENTATION DETAILS

The implementation comprises about 6,300 lines of Python in 59 application modules, about 1,500 lines of backend tests and about 5,900 lines of TypeScript/TSX in 42 frontend files, plus a 490-line SAM template (counts measured on the repository at the time of writing).

6.1 Frontend Architecture

The frontend is a React 19 single-page application written in TypeScript and built with Vite. React Router defines three public routes (landing, sign-in, registration) and fourteen authenticated routes rendered inside a shared application shell (grouped sidebar, header with quick search, anomaly-alert bell, theme toggle and profile). Pages are lazy-loaded. All server communication goes through one API client (lib/api.ts) that adds the bearer token, builds query strings and converts error envelopes into typed errors; it never substitutes placeholder data when a request fails. Styling uses Tailwind CSS with design tokens for a dark theme (default) and a light theme.

6.2 Backend Architecture

The FastAPI application factory (app/main.py) can build the complete API or only selected route groups. Routes handle HTTP concerns; services contain all business logic; repositories implement persistence. Settings are read from environment variables, with no secrets in source code.

6.3 REST API Design

Endpoints are versioned under /api/v1, nouns identify resources and HTTP verbs express actions. Validation uses Pydantic models that reject unknown fields (so a client cannot submit a user_id). Lists return totals computed over all matching items, not only the current page.

6.4 Database Design

A Repository interface defines user-scoped operations (CRUD by id, date-range queries, conditional inserts, delete-all). SQLiteRepository and DynamoDBRepository implement it identically, which is verified by running the same API test suite against both (DynamoDB through moto).

6.5 Authentication

In local mode, registration hashes passwords with scrypt and login issues an HS256 JWT signed with a secret of at least 48 random bytes; production with local auth refuses to start without an explicit secret. In Cognito mode the frontend uses the Cognito Identity

Provider API (SignUp, ConfirmSignUp, InitiateAuth with USER_PASSWORD_AUTH, refresh, GlobalSignOut) and the backend verifies RS256 access tokens against the pool's JWKS. After that check it calls Cognito GetUser with the token, which fails for tokens revoked by sign-out; a positive result is cached for 15 s per token in a warm container, and if Cognito throttles or is unreachable the check is skipped with a warning log so that users are not locked out.

6.6 Authorization

Authorization is ownership-based: a FastAPI dependency verifies the token and builds a user context, and every repository call is scoped by that context's user id. Requests for another user's resource return 404, which does not reveal whether the resource exists.

6.7 Expense Management

Amounts are parsed from strings into integer paise without floating point. After a create, update or delete, the service runs the threshold check and anomaly re-evaluation and returns their effects so the frontend can show notifications.

6.8 Financial Analytics

Aggregations (totals, category and payment-method shares, daily and monthly series, top merchants) are computed in the service layer after a single date-range query, which keeps results identical across both databases.

6.9 Financial Intelligence

The insights service evaluates a fixed set of rules over seven months of data and returns structured insights with kind, severity, message, action and evidence. Recurring payments are detected separately and reused by the insight rules.

6.10 Anomaly Detection

Detection runs when an expense is created or edited and on demand (re-scan of 180 days). Alerts carry a deduplication key so a reviewed alert is not recreated; editing a transaction re-evaluates its alerts and the month's category spike.

6.11 Receipt OCR

Uploads are validated by magic bytes and full decoding; images are pre-processed with Pillow and passed to Tesseract through pytesseract with a timeout. The parser is a separate pure module with its own tests, so extraction rules can be tested without OCR.

6.12 What-If Simulation

The simulator computes a per-category baseline and applies adjustments with Decimal arithmetic. It reads data but never writes; a test asserts that running simulations leaves all records unchanged.

6.13 Rule-Based Finance Assistant

The assistant is a deterministic pipeline (normalisation, date resolution, entity extraction, regex-scored intent selection, handler). Handlers call the same services as the rest of the application, scoped to the user.

6.14 Savings Goals

Goal metrics are computed on read from the stored target, date and contributions, so status always reflects the current date. A starting balance is stored as a contribution of kind starting_balance and excluded from the contribution pace.

6.15 AWS Integration

Each API Lambda function wraps a subset of route groups with Mangum, so the same FastAPI code serves API Gateway HTTP API events; configuration switches DATA_BACKEND to dynamodb, RECEIPT_STORAGE to s3 and AUTH_MODE to cognito. The receipts function is packaged as a container image because Tesseract is a native binary. A separate standard-library handler (infra/web/handler.py) serves the built SPA when the stack is deployed with FrontendHosting=apigateway, including the index.html fallback for client-side routes. The S3 client is pinned to the regional endpoint so that presigned uploads to new buckets are not redirected.

The handler factory used by every function is short; its essential logic is:

```
def build_handler(*groups):
        @lru_cache(maxsize=1)
        def _adapter():  # built once per warm container
                return Mangum(create_app(groups=groups), lifespan="off",
                api_gateway_base_path="/")
        def handler(event, context):
                return _adapter()(event, context)
        return handler

# app/handlers/expenses.py:  handler = build_handler("expenses")
```

6.16 Error Handling

A central exception handler maps domain errors (validation, not found, conflict, unauthorized, rate-limited) to HTTP status codes and a uniform envelope; unexpected

errors return a generic message. The frontend renders error states with retry, and a dedicated screen when the backend is unreachable.

6.17 Logging and Monitoring

A request middleware logs method, path, status, duration and a request id, and adds security headers. In AWS these logs go to CloudWatch log groups with 14-day retention; an alarm watches API 5xx responses.

6.18 Security Considerations

Security is enforced in the backend and repeated at the edge in AWS: token verification, ownership scoping, strict validation, safe file handling, CORS restricted to configured origins, no credentials in the browser, least-privilege IAM and private storage (Chapter 10).

6.19 Local Execution versus AWS Deployment

Table 21: Local execution compared with the AWS deployment

Concern	Local (verified)	AWS (deployed 16 October 2026, verified live)
Web hosting	Vite dev server / static build	WebFunction on the HTTP API, same origin as the API (CloudFront + private S3 defined, pending account verification)
API runtime	Uvicorn running the full FastAPI app	API Gateway HTTP API → 8 API Lambda functions (Mangum)
Database	SQLite file	DynamoDB single table, on-demand
Receipts	Private local folder	Private S3 bucket, presigned POST/GET
Authentication	scrypt + HS256 JWT issued by the backend	Cognito user pool; JWT authorizer + backend JWKS check
OCR	Tesseract installed on the machine	Tesseract inside the receipts container image
Logs	Console	CloudWatch Logs (9 groups, 14 days) and a 5xx alarm

6.20 User Interface Animations

The final iteration added transitions and animations without a new dependency: CSS keyframes plus a small hook module (lib/motion.ts) that counts figures up and decides when charts animate. Pages rise in on navigation, dashboard figures count up, charts draw in only when their data change, progress bars grow, and dialogs, the mobile drawer, validation and assistant messages and the receipt steps have short entrances. Only opacity and transform are animated. With prefers-reduced-motion all motion is disabled and screen readers hear only final values. Four unit tests cover this, and a browser run across all routes recorded no console errors (docs/PROJECT_DOCUMENTATION.md, Section 5.6).

7. ALGORITHMS AND FINANCIAL INTELLIGENCE

All algorithms in FinSight are deterministic rules and classical statistics; no machine-learning model and no large language model is used. Examples in this chapter use FinSight's synthetic demo data and are labelled as examples, not measurements of real users. In the complexity statements, n is the number of the user's expenses in the queried period.

7.1 Exact Money Arithmetic

Purpose. Avoid rounding errors in financial values.

Inputs. User-entered amount strings such as “1,234.50”; currency exponent (2 for INR).

Processing steps.

- Strip grouping separators and currency symbols; reject more than two decimal places.
- Convert whole and fractional parts with integer arithmetic into minor units (paise).
- Compute ratios (percentages, projections) with Decimal and round only for display.

Outputs. Integer minor units stored and transmitted; formatted strings for display.

Time complexity. O(L) in the length of the input string.

Limitations. One currency per account; no exchange-rate conversion.

Example. “1234.5” becomes 123450 paise; 0.1 + 0.2 issues of binary floating point cannot occur.

7.2 Spending Aggregation and Period Comparison

Purpose. Summarise spending and compare it fairly with an earlier period.

Inputs. Expenses in the selected month or range; today's date in the user's time zone.

Processing steps.

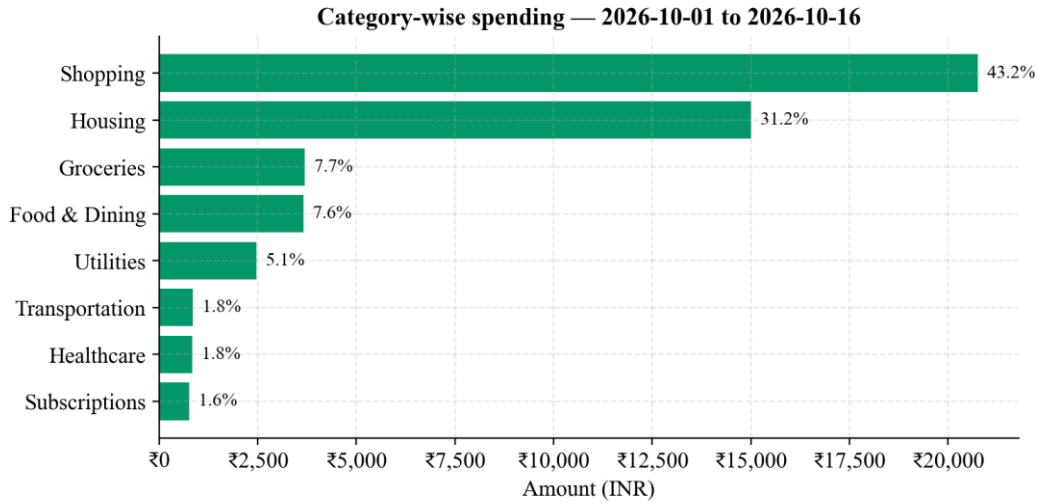
- Query the user's expenses by date range (indexed in both databases).
- Accumulate totals by category, payment method, day, month and merchant in one pass.
- For the current, incomplete month compare with the same day range of the previous month (for example 1–16 October against 1–16 September).
- Change % = (current − previous) ÷ previous × 100; undefined when the previous total is zero.

Outputs. Totals, shares, series, top merchants and a signed change percentage.

Time complexity. O(n) aggregation plus O(m log m) to sort m groups.

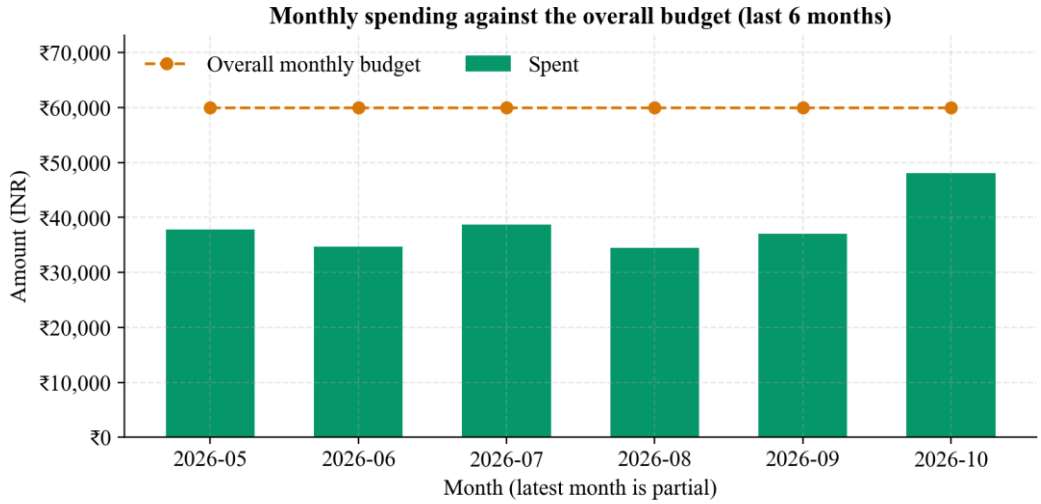
Limitations. Comparisons are meaningful only when the earlier period has data.

Example. In the demo data, ₹48,023 spent in 1–16 October is +62.5% versus ₹29,554 in 1–16 September (Figure 11). Figure 28 shows the category split and Figure 29 six months against the budget.



Example generated from FinSight's synthetic demo data (not real user data).

Figure 28: Example: category-wise spending for the current month (synthetic demo data)



Example generated from FinSight's synthetic demo data (not real user data).

Figure 29: Example: monthly spending against the overall budget (synthetic demo data)

7.3 Budget Utilization, Thresholds and Projection

Purpose. Report budget status early and alert once per threshold.

Inputs. Budgets effective in the month, the month's spending, thresholds, days elapsed.

Processing steps.

- Select the budget version effective for the month (versions have effective_from and effective_to months).

- Utilization = spent ÷ budget × 100 (undefined for a zero budget); status: ok, warning (≥ first threshold), at limit (= 100%), exceeded (> 100%).
- For the current month, after at least 3 days: projected = spent ÷ days elapsed × days in month; projected overrun when projected > budget and not already exceeded.
- For each threshold crossed, insert an alert event only if (user, month, budget, threshold) does not exist.

Outputs. Status, utilization, remaining amount, daily allowance, projection and new alert events.

Time complexity. O(n + b) for n expenses and b budgets.

Limitations. Linear projection ignores planned one-off expenses; alerts are in-app only.

Example. Figure 30 shows the demo month: the Shopping budget is at 519% (exceeded) while the overall budget is at 80%.

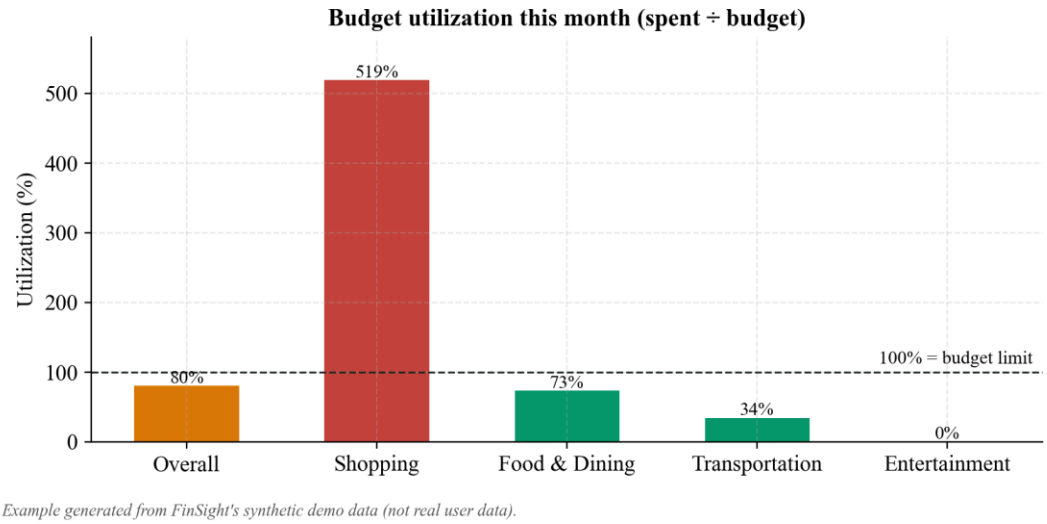


Figure 30: Example: budget utilization by budget (synthetic demo data)

7.4 Insight Generation Rules

Purpose. Produce statements that are labelled and verifiable.

Inputs. Seven months of expenses, budgets and goals; recurring payments.

Processing steps.

- Month-over-month total change (fact), using the like-for-like range from Section 7.2.
- Category change reported only if |change| ≥ 20% and the absolute change is at least 5% of the previous period's total (avoids trivial percentages).
- Top category share; concentration warning when one category is ≥ 50% with at least 5 transactions.
- Daily average (estimate) and average monthly spending over prior months with records (requires ≥ 2 months).

- Month-end projection (after ≥ 5 days) compared with the overall and category budgets; exceeded budgets are facts, possible overruns are projections, plus a daily-allowance recommendation.
- Goals: required monthly saving compared with the 90-day contribution pace.
- Recommendation: for the largest discretionary increase, the saving from a 15% reduction of its recent typical month.
- When data is insufficient, emit a data-gap insight instead of a claim.

Outputs. A list of insights, each with kind, severity, message, optional action link and evidence (period, calculation, values, transactions).

Time complexity. O(n log n) dominated by grouping and sorting.

Limitations. Fixed rules; thresholds are design choices, not learned values.

Example. "Shopping spending up 482%" is labelled an observed fact and lists the supporting transactions (Figure 14).

7.5 Recurring Payment Detection

Purpose. Recognise subscriptions, rent and bills.

Inputs. Expenses grouped by normalised merchant (or description) and category.

Processing steps.

- Keep groups with at least two transactions whose amounts are all within ±15% of the group median.
- Compute intervals between consecutive dates; classify as weekly if every interval is 6–8 days or monthly if every interval is 26–35 days.
- Estimate the next expected date (last date + mean interval) and mark the series active unless the payment is overdue by more than half an interval plus 3 days.
- Report the monthly equivalent (weekly × 52/12) for ranking.

Outputs. Recurring series with cadence, typical amount, occurrences, next expected date and confidence.

Time complexity. O(n log n) for grouping and sorting.

Limitations. Irregular bills or changing amounts are missed; two occurrences give low confidence.

Example. Demo data yields Sunrise Apartments (₹15,000 monthly), AirFiber, Netflix and a newly detected Spotify subscription.

7.6 Anomaly Detection

Purpose. Flag transactions that are unusual relative to the user's own history.

Inputs. The target expense, the user's last 365 days of expenses and the sensitivity preset (thresholds t, r, s).

Processing steps.

- Category rule (needs ≥ 5 earlier transactions in the category): compute the median m and MAD; if MAD > 0, flag when the modified z-score $0.6745\ (x - m) / MAD \geq t$ and $x \geq r \times m$; if MAD = 0, apply only $x \geq r \times m$.
- Overall rule for thin category history (needs ≥ 10 transactions overall): flag when $x \geq \max(5 \times$ overall median, $1.5 \times$ 95th percentile) and no similar amount already exists in the category.
- Duplicate rule: same amount and merchant/description within one day (only the later entry is flagged); repetition rule: three or more at one merchant on one day.
- Category spike: the month's category total ≥ s × the average of earlier months with spending (needs ≥ 2 months).
- Create an alert only if its deduplication key is new; severity increases for repeated large transactions.

Outputs. Alerts with rule, severity, reason and comparison values (median, MAD, z, ratio, percentile, sample size).

Time complexity. O(k log k) per evaluated expense for k transactions in its category (median and MAD), O(n log n) for a full re-scan.

Limitations. Requires history; thresholds are conventions [6]; alerts indicate unusual spending, not fraud [4].

Example. For the demo Shopping category (11 transactions, median ₹1,637, MAD ₹279), the ₹18,999 headphones purchase has a modified z-score of about 42 (Figure 32) and is flagged; the presets are conservative (5.0, 3.0, 2.25), balanced (3.5, 2.5, 1.75) and sensitive (3.0, 2.0, 1.5).

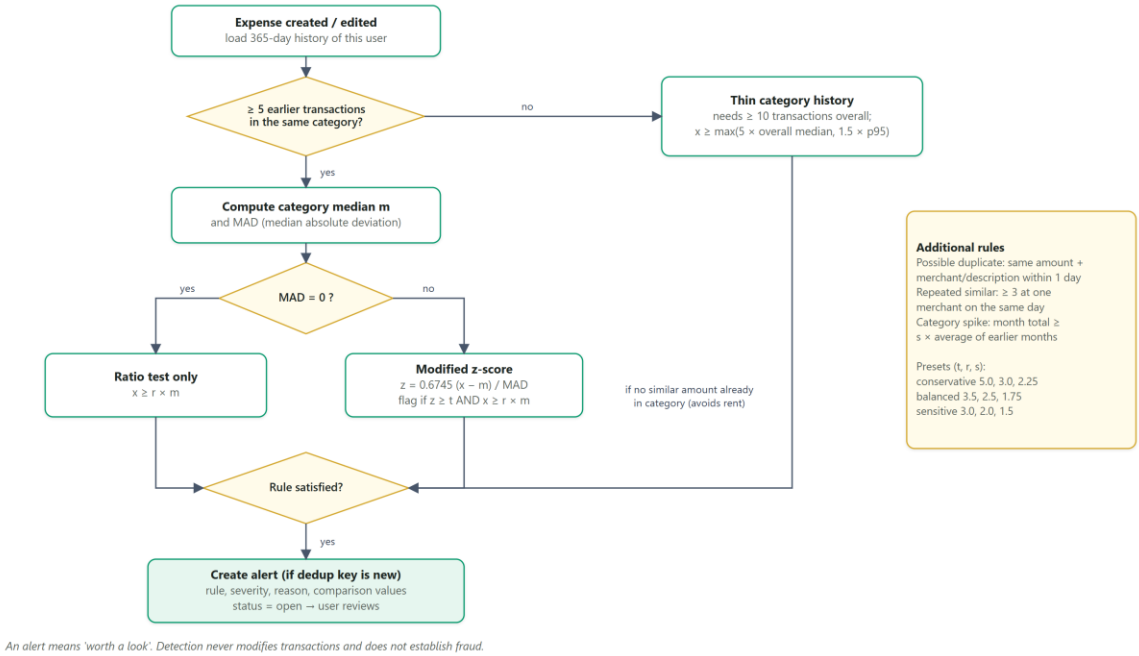
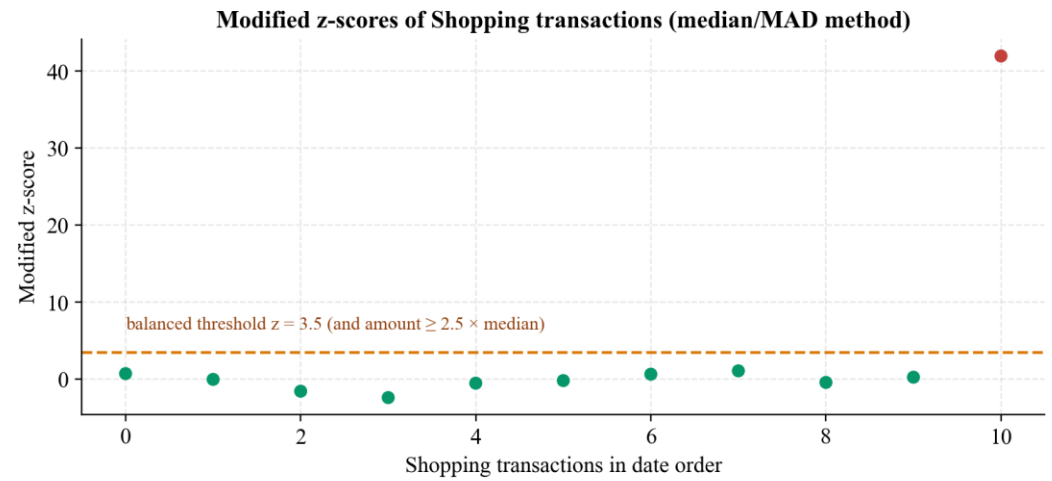


Figure 31: Anomaly detection flow for a new or edited expense



Example generated from FinSight's synthetic demo data (not real user data).

Figure 32: Example: modified z-scores of Shopping transactions (synthetic demo data)

The decision flow is summarised in Figure 31.

7.7 OCR Extraction and Receipt Parsing

Purpose. Turn a receipt image into reviewable expense fields.

Inputs. A validated image; the account currency.

Processing steps.

- Pre-process: apply EXIF rotation, convert to grayscale, scale to an OCR-friendly width, auto-contrast and sharpen.

- Run Tesseract with --oem 3 --psm 4 (single column of variable-size text) and collect per-line confidence.
- Total: rank candidate lines by keyword priority (grand total > net/amount payable > total > amount), excluding subtotal, tax totals, item counts, change and savings; take the amount on the line or the next line; cross-check against subtotal + tax or the sum of line items.
- Date: ISO, day-first numeric (default for INR receipts, with an ambiguity warning) and month-name formats; ignore future dates.
- Merchant: first meaningful header line, skipping invoice, GSTIN and phone lines; tax from CGST/SGST/IGST/VAT lines; currency, line items and a suggested category by keyword rules.
- Normalise amounts (Indian and European grouping, "/-" suffixes, common OCR digit confusions) and ignore dates, times, phone numbers and percentages.
- Attach a confidence score to every field and list warnings; the user reviews and confirms.

Outputs. Merchant, date, total, tax, currency, line items and suggested category, each with confidence; raw text for inspection.

Time complexity. Linear in the number of recognised lines and tokens after OCR; OCR time depends on image size.

Limitations. Heuristic rules; printed English receipts work best; accuracy on real-world receipts was not measured.

Example. For a generated sample receipt, OCR confidence was 92.6% over 11 lines, and the merchant, date, total (₹610.05, cross-checked with subtotal + tax) and tax were extracted (Figure 16). Figure 33 summarises the pipeline.

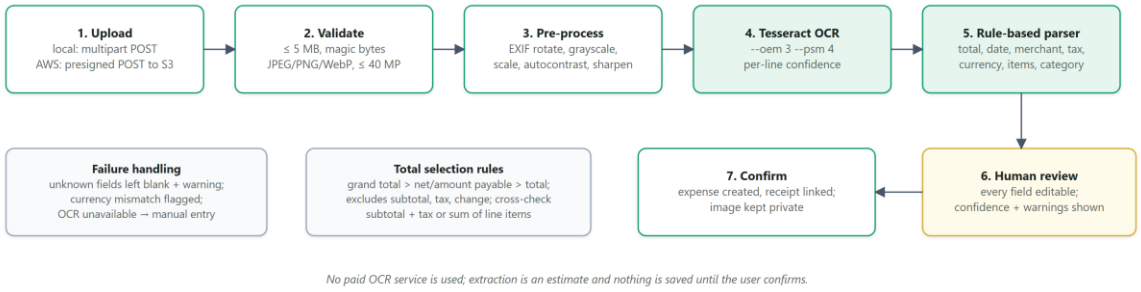


Figure 33: Smart receipt scanner pipeline

7.8 What-If Simulation

Purpose. Estimate the effect of hypothetical spending changes.

Inputs. Baseline mode, scenarios (up to four) with per-category adjustments, optional goal.

Processing steps.

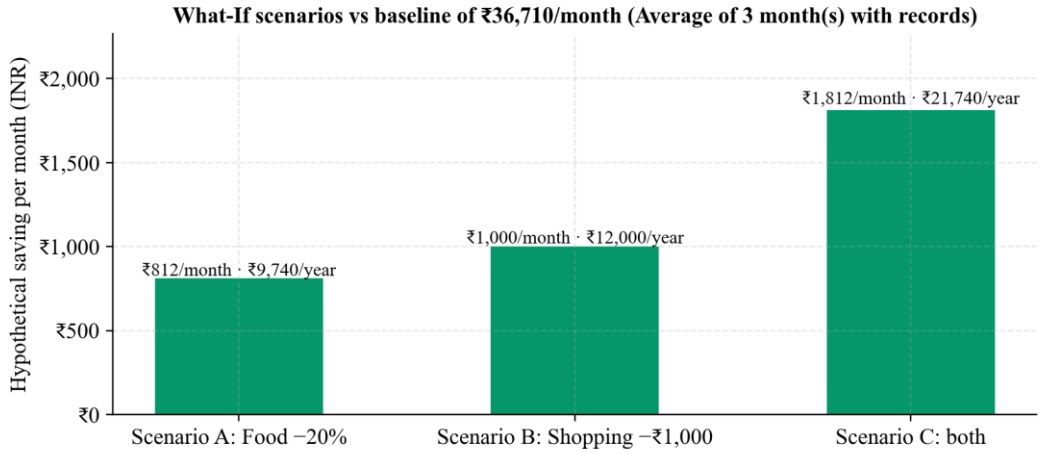
- Baseline per category: last month, the average of the last three months with records, or this month's projection.
- Apply each adjustment: reduce by p% (0–100), reduce by a fixed amount (capped at current spending with a warning) or set a target amount.
- Monthly saving = current − new; annual saving = monthly × 12 (stated as an assumption).
- Budget impact: remaining budget before and after; goal impact: months to reach the goal if the saving is added to the recent contribution pace.

Outputs. Per-scenario totals, savings, warnings, budget and goal impact and a written explanation; the best scenario is highlighted.

Time complexity. O(c × s) for c categories and s scenarios after computing the baseline in O(n).

Limitations. Assumes the baseline continues; hypothetical results are not forecasts.

Example. Against a ₹36,710 monthly baseline, reducing Food & Dining by 20% saves about ₹812 per month and reducing Shopping by ₹1,000 saves ₹1,000 (Figure 34).



Example generated from FinSight's synthetic demo data (not real user data).

Figure 34: Example: hypothetical monthly savings of three What-If scenarios (synthetic demo data)

7.9 Rule-Based Natural-Language Assistant

Purpose. Answer common questions without an external language model.

Inputs. Question text; the user's records.

Processing steps.

- Normalise text (case, punctuation, numbers).

- Resolve dates: today, yesterday, this/last week, month or year, last N days or months, named months with year inference, ranges (“between X and Y”, “since X”, “on X”); vague words such as “recently” trigger a clarifying question.
- Extract entities: categories with synonyms (for example “eating out”, “Netflix”), merchants found in the user's records, percentages, amounts and goal names.
- Score 14 intents with regular expressions: total spent, transaction count, top category, category change, average spending, largest transactions, list transactions, unusual transactions, budget remaining, budget status, simulate saving, goal requirement, help and unsupported income.
- Run the intent's handler using user-scoped service calls and return text, data, period, suggestions and a disclaimer.

Outputs. A structured reply; the user text is never executed.

Time complexity. O(P + h) for P patterns and handler cost h (usually O(n)).

Limitations. Fixed vocabulary and intents; English only.

Example. “Which category increased the most?” returns Shopping (₹3,569 → ₹20,753 for 1–16 September versus 1–16 October) with a comparison table (Figure 18).

7.10 Savings Goal Metrics

Purpose. Show whether a goal is on schedule.

Inputs. Target amount and date, contributions, today's date.

Processing steps.

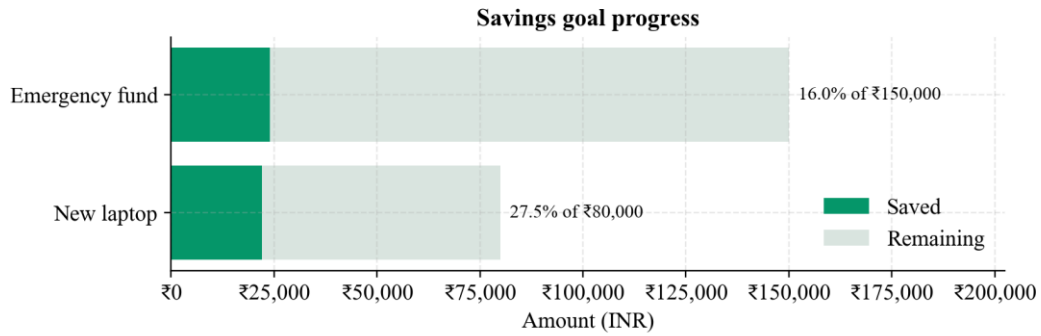
- Remaining = max(target − saved, 0); progress = saved ÷ target.
- Required monthly saving = remaining ÷ months until the target date (at least one month).
- Recent pace = sum of contributions in the last 90 days converted to a monthly rate (starting balances excluded).
- Status: completed, overdue, behind (pace below requirement), on track or not started; projected completion from the pace.

Outputs. Goal metrics and status.

Time complexity. O(g) for g contributions per goal.

Limitations. Relies on manually recorded contributions.

Example. The demo “New laptop” goal is 27.5% complete and needs about ₹7,250 per month against a recent pace of about ₹3,667 per month, so it is behind schedule (Figure 35).



Example generated from FinSight's synthetic demo data (not real user data).

Figure 35: Example: savings goal progress (synthetic demo data)

8. TESTING AND EVALUATION

8.1 Testing Strategy

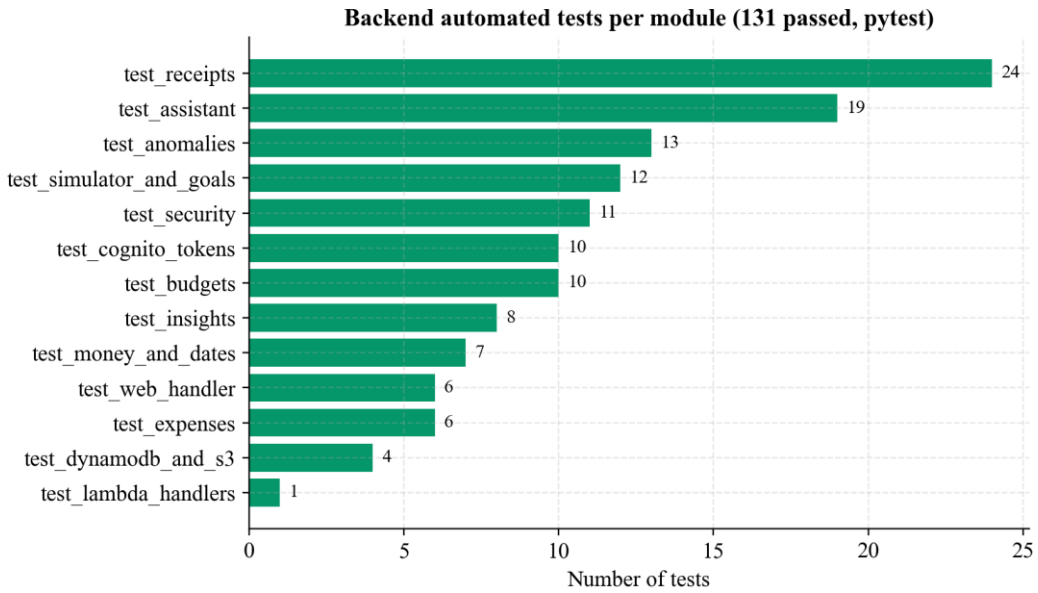
Testing was carried out at three levels. Automated unit and integration tests (pytest for the backend, Vitest for frontend utilities) and static checks (TypeScript, production build, cfn-lint and sam validate for the template) run on the repository; the AWS code paths are also exercised against moto, a library that emulates AWS APIs locally. Scripted browser workflows check every page of the locally running application. After the stack was deployed to AWS on 16 October 2026, a live API suite (infra/live_test.py) and a live browser suite (headless Chrome against the deployed URL) were run against the real Cognito user pool, API Gateway, Lambda functions, DynamoDB table and S3 bucket. All tests use synthetic data and synthetic test users; test credentials are kept only in git-ignored local files.

8.2 Automated Test Results

Table 22 lists the checks executed on the final repository. Figure 36 shows how the 131 backend tests are distributed across modules. Seventeen backend tests were added during deployment (Cognito display-name handling and token revocation, the static web handler and the regional S3 endpoint), and the frontend suite grew from 13 to 20 tests with the animation utilities (Section 6.20) and the About the author dialog.

Table 22: Automated and scripted test results

Category	Command	Result (executed)
Backend unit + integration tests	python -m pytest	131 passed, 0 failed
AWS code-path tests (subset)	pytest test_dynamodb_and_s3.py test_lambda_handlers.py test_security.py test_cognito_tokens.py test_web_handler.py	32 passed
Frontend unit tests	npm test (Vitest)	20 passed (4 files)
Type checking	npm run typecheck (tsc -b)	Passed, no errors
Production build	npm run build (Vite)	Passed
Infrastructure template validation	cfn-lint infra/template.yaml; sam validate --lint	No errors
OCR engine availability	tesseract --version	Tesseract 5.4.0 locally; 5.5.0 in the Lambda container image
Scripted browser workflows, local (headless Chrome)	25 checks; later 36 captures across all routes	25/25 passed; the 36-capture run had 0 console errors and 0 failed requests
Live AWS API, Cognito, S3 and OCR tests	infra/live_test.py on the deployed stack	64 passed, 0 failed, 0 limitations (64 checks)
Live browser tests on AWS	headless Chrome on the live URL	27/27 passed; 0 console errors; 0 failed requests



Source: pytest --collect-only and a full pytest run on the project repository (all 131 passed, 2026-10-16).

Figure 36: Backend automated tests per module (all 131 passed)

8.3 Representative Test Cases

Table 23: Representative test cases

Test ID	Module	Description	Expected result	Actual result	Status	Evidence
TC01	Money	Parse “1,234.50” and reject 3 decimals	123450 paise; error for excess precision	As expected	Pass	test_money_and_dates.py
TC02	Expenses	Create, update, delete; invalid amount and future date	Validation errors; consistent totals	As expected	Pass	test_expenses.py
TC03	Expenses	Search, filters, sorting, pagination	Correct subset and totals over all matches	As expected	Pass	test_expenses.py
TC04	Budgets	Threshold crossing creates one alert per threshold per month	No duplicate alerts	As expected	Pass	test_budgets.py
TC05	Budgets	Projected overrun vs exceeded	Distinct statuses	As expected	Pass	test_budgets.py
TC06	Insights	Insufficient history	Data-gap insight, no claims	As expected	Pass	test_insights.py
TC07	Insights	New recurring subscription detected	Recurring insight with evidence	As expected	Pass	test_insights.py
TC08	Anomalies	Repeated rent not flagged; large purchase flagged	No false positive; alert with comparison data	As expected	Pass	test_anomalies.py
TC09	Anomalies	Identical subscriptions (MAD = 0)	Not flagged	As expected	Pass	test_anomalies.py
TC10	Receipts	Real OCR end-to-end upload → process → confirm	Fields extracted; expense linked; no double confirm	As expected	Pass	test_receipts.py
TC11	Receipts	Spoofed type, oversize, corrupt image, path traversal	Rejected safely	As expected	Pass	test_receipts.py
TC12	Simulator	Scenarios have no side effects	Records unchanged	As expected	Pass	test_simulator_and_goals.py
TC13	Assistant	Ambiguous date; other user's data	Clarifying question; no leakage	As expected	Pass	test_assistant.py

Test ID	Module	Description	Expected result	Actual result	Status	Evidence
TC14	Security	Cross-user read/update/delete; client user_id; alg = none	404 / 422 / 401	As expected	Pass	test_security.py
TC15	AWS paths	Full API on DynamoDB (moto); S3 presigned policy; regional endpoint	Same behaviour as SQLite; size/type conditions	As expected	Pass	test_dynamodb_and_s3.py
TC16	Lambda	Handler serves its route group only; static web handler	200 health; 401; 404 for other groups; SPA fallback	As expected	Pass	test_lambda_handlers.py, test_web_handler.py
TC17	UI	Add, edit, delete expense; filters; budget edit; goal + contribution; OCR; settings persistence; theme; mobile drawer	Workflows complete	As expected	Pass	Scripted browser run
TC18	UI	Backend unavailable	Service-unavailable screen with retry	As expected	Pass	Scripted browser run
TC19	AWS live	Cognito sign-up and sign-in; no token; forged token	Tokens issued; 401 at the JWT authorizer	As expected	Pass	AUTH-01–05, SEC-01–09
TC20	AWS live	Browser → presigned POST → private S3 → OCR Lambda → confirm	Total 610.05 extracted; unsigned GET 403; expense linked	As expected (OCR confidence 93.3%)	Pass	RCT-01–10, UI-08
TC21	AWS live	User B reads or edits user A's expense, receipt and goal	404 for every request; A's data intact	As expected	Pass	ISO-01–03
TC22	AWS live	Access token reused after GlobalSignOut	200 before sign-out; 401 token_revoked after	401 token_revoked about 17 s after sign-out	Pass	AUTH-07

8.4 Live Testing on AWS

The live suites were run on 16 October 2026 against stack finance-intelligence-prod in ap-south-1 at https://zkht2b4ib3.execute-api.ap-south-1.amazonaws.com. Table 24 summarises the 64 API, authentication, storage and OCR checks by area; the full table with the expected and actual result of every check is kept in docs/AWS_LIVE_TEST_RESULTS.md, with the raw results in docs/evidence/09_testing/. Two synthetic users were created with Cognito SignUp and confirmed by an administrator call; the second user, used only for the isolation tests, was deleted afterwards.

Table 24: Live AWS test results by area (64 passed, 0 failed)

Area	Test IDs	What was checked on the live stack	Result
Frontend delivery	FE-01–06	SPA over HTTPS, deep-link fallback, security headers, no plain-HTTP listener, immutable caching of hashed assets, no localhost URLs in the bundle	6/6 pass
Public endpoints	API-01–03	Health reports the DynamoDB back-end; public configuration reports Cognito and presigned uploads; no OpenAPI docs in production	3/3 pass
Edge authorization	SEC-01–09	Eight private routes without a token and one forged token all rejected with 401 by the JWT authorizer	9/9 pass
Cognito and identity	AUTH-01–07, PRF-01, SEC-10	Sign-up, sign-in, wrong password, refresh, identity from the token, display name, revocation after GlobalSignOut (401 token_revoked), preferences, client-supplied user_id rejected (422)	9/9 pass
Expenses on DynamoDB	EXP-01–08	Create, read, update, search, filter and sort, delete, validation envelope, unknown id (404)	8/8 pass
Budgets	BUD-01–04	Overall and category budgets, warning at 89.9%, exceeded after update, duplicate rejected (409)	4/4 pass
Analytics and intelligence	ANA-01–05	Dashboard totals, analytics, six insights with evidence, anomaly list and re-scan	5/5 pass
Assistant, goals, simulator	AST-01, GOL-01–02, SIM-01	Assistant answer from live data, goal and contribution (15%), What-If result	4/4 pass
Receipts: S3 and OCR container	RCT-01–10	Presigned POST, CORS, 6 MB upload refused by S3, OCR in Lambda (₹610.05, 93.3%, 3.2 s), presigned GET, unsigned GET 403, confirm, PDF refused	10/10 pass
Cross-user isolation	ISO-01–03	Six requests by user B for user A's data returned 404; B's list empty; A's data unchanged	3/3 pass
CORS and clean-up	COR-01–02, CLN-01	One origin for site and API; foreign origin not allowed; isolation user removed	3/3 pass

The browser suite signed in through Cognito, created an expense through the user interface, uploaded a receipt from the browser directly to the private S3 bucket and read it with the OCR function, checked the assistant, goals and profile pages, switched and reloaded the theme, verified that the layout has no horizontal overflow at 390 px, signed out and confirmed that protected routes redirect to the sign-in page. Six further checks (UI-22–27) cover the author credit: the footer text, the About the author dialog opened from the information button, its statement that CloudFront is planned but not deployed, closing with Escape, and its fit at 390 px. Figures 37–41 are screenshots from that run; they show only the synthetic test user's data, read from DynamoDB, or public pages.

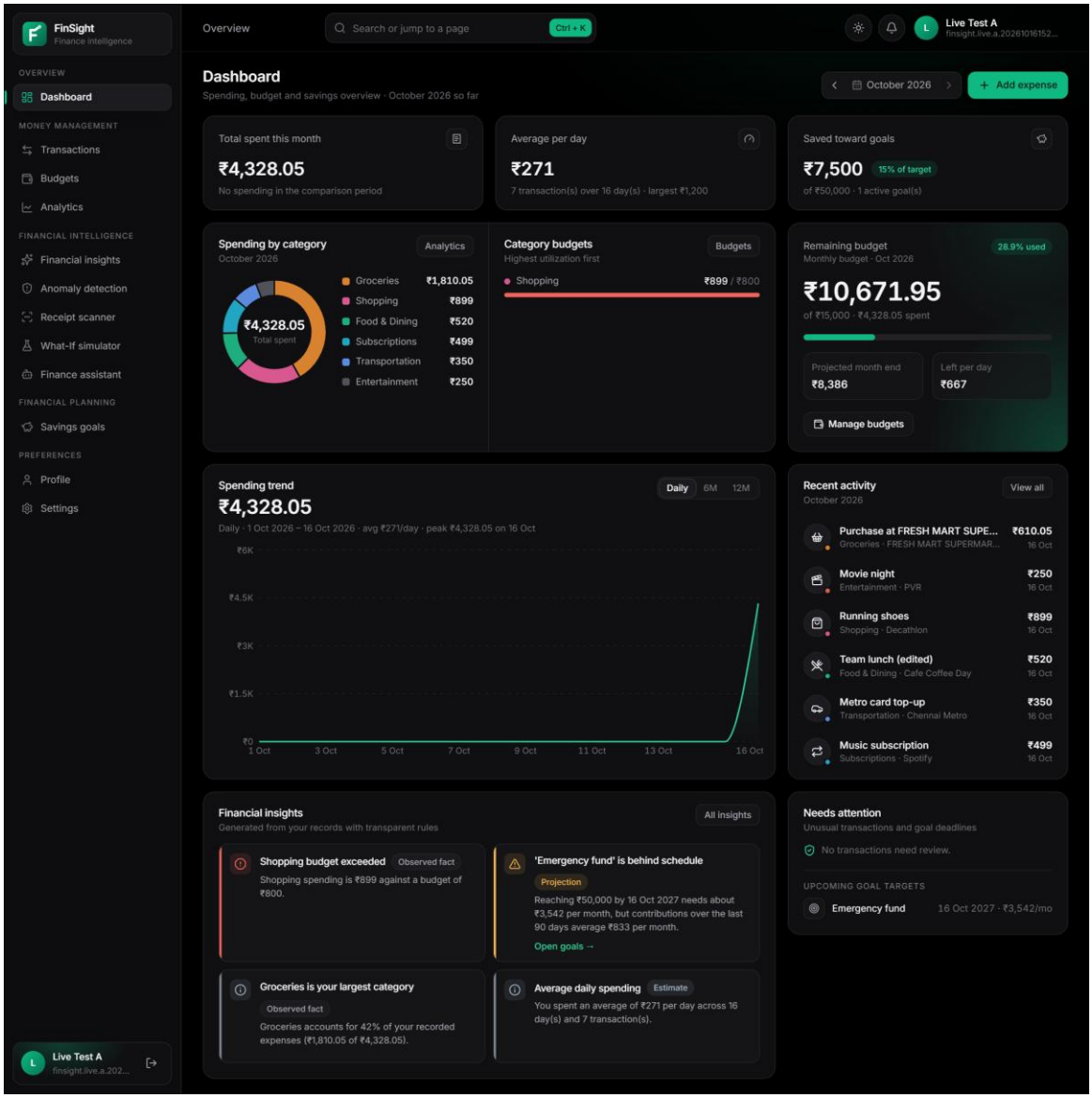


Figure 37: Live dashboard on AWS after Cognito sign-in (synthetic test user; data from DynamoDB)

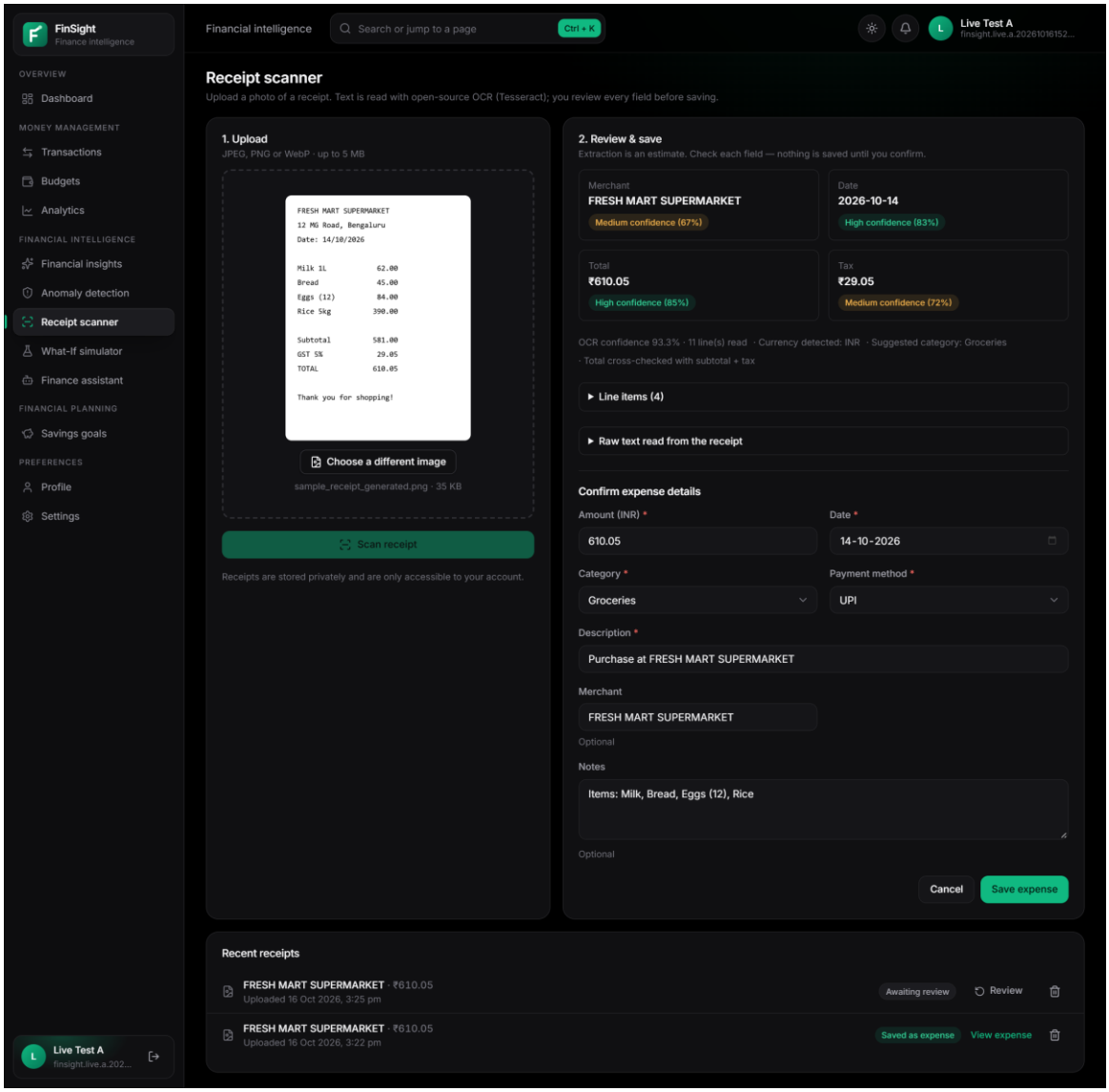


Figure 38: Live receipt scanner: image uploaded to the private S3 bucket and read by the Tesseract container function, with fields awaiting review

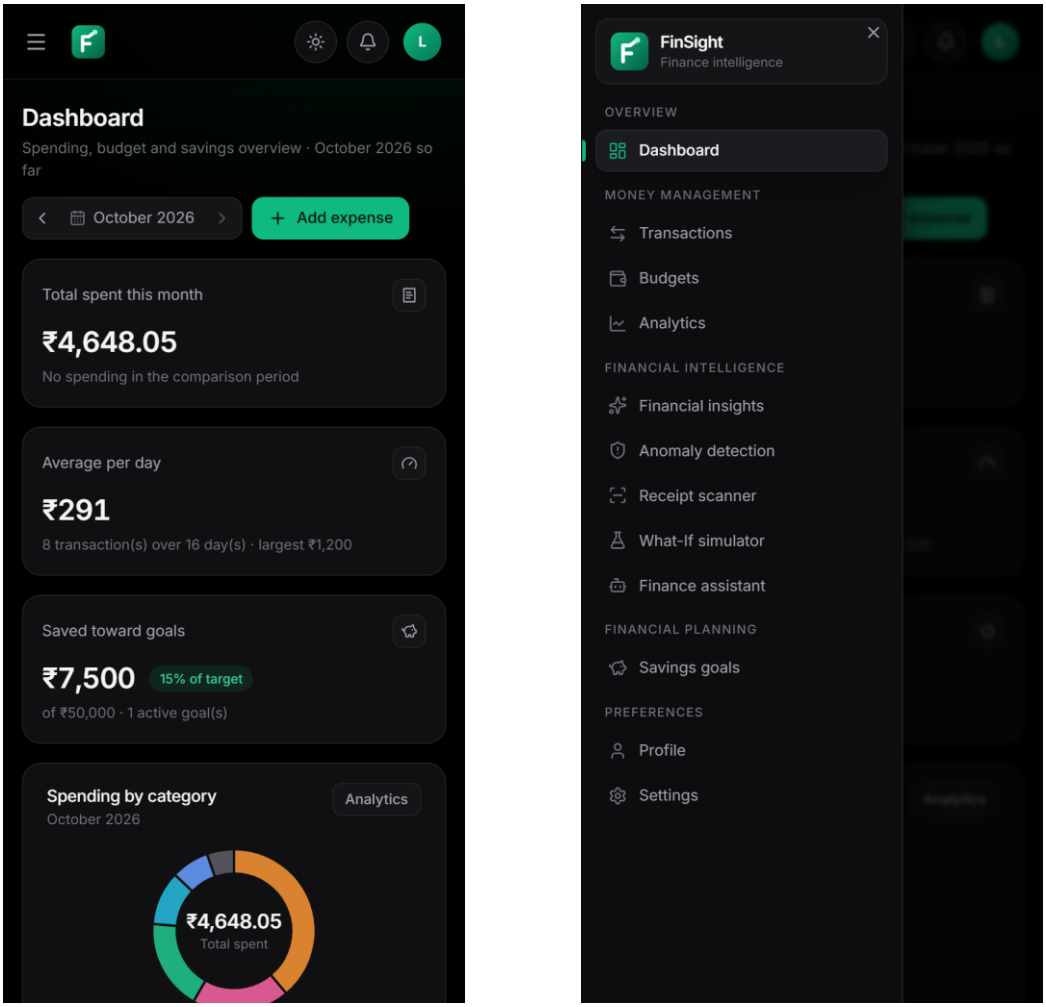


Figure 39: Live application at 390 px: dashboard (left) and navigation drawer (right)

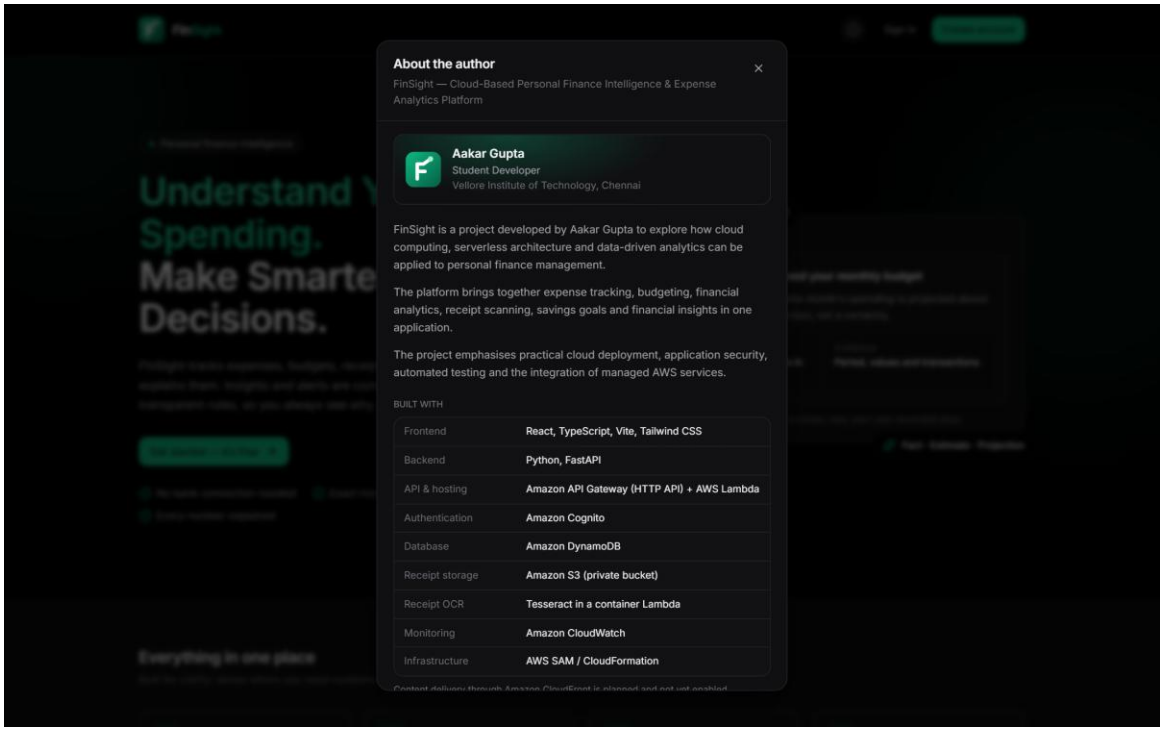


Figure 40: About the author dialog on the live landing page, opened from the information button (dark theme)

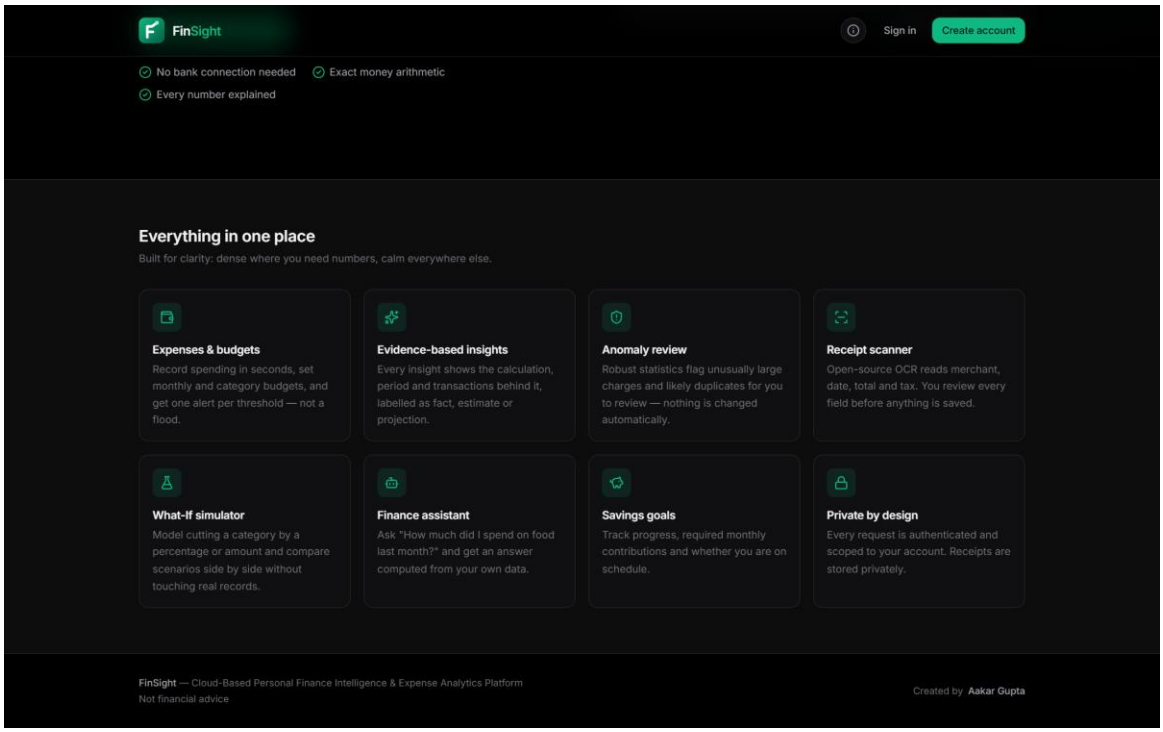


Figure 41: Live landing page footer with the "Created by Aakar Gupta" credit, which opens the same dialog

8.5 Problems Found During Deployment and Their Fixes

Deploying and testing on real infrastructure exposed problems that the local and emulated tests could not show. Table 25 lists them with their causes and fixes; each fix was verified by a re-run, and the two code fixes are covered by new regression tests.

Table 25: Problems found during deployment and live testing

Stage	Symptom	Cause	Fix
Deployment attempts 1–2	FrontendDistribution CREATE_FAILED; stack rolled back	New accounts must be verified by AWS Support before creating CloudFront resources	New FrontendHosting parameter; deployed with apigateway (WebFunction serves the SPA); see Section 9.2
Deployment 3	/health returned 500: Runtime.ImportModuleError, cannot import ArgsKwargs from pydantic_core	SAM's Python builder skips *.so files when copying source, so the vendored native wheels were dropped	Stage requirements.txt instead, so the builder installs manylinux cp313 wheels; imports verified in a Linux cp313 container; code-only update 1
Live run 2	Presigned POST to the receipts bucket answered 307 (RCT-03/04)	boto3 signed against the global S3 endpoint; new buckets outside us-east-1 redirect for some hours	S3 client pinned to the regional virtual-hosted endpoint; regression test test_s3_presigned_urls_use_regional_endpoint; code-only update 2
Live runs (AUTH-07)	An access token still worked after GlobalSignOut until it expired	API Gateway JWT authorizers check only signature, issuer, audience and expiry	Backend checks each token with Cognito GetUser (15 s cache, fail-open with a warning log); AUTH-07 now 401 token_revoked about 17 s after sign-out; code-only update 3
Live runs 1–3 (test harness)	Script aborted; one browser check timed out	Console encoding, transient network resets on the test machine, a fixed 4 s wait for a lazy-loaded page	UTF-8 output, retries for transport errors only, wait for rendered content; 21/21 on re-run

8.6 Evaluation and Limitations

The results show correct logic on synthetic data, correct AWS code paths against emulated services, and correct end-to-end behaviour of the deployed system: Cognito authentication, the gateway authorizer, user isolation on DynamoDB, private S3 storage, OCR inside the Lambda container and token revocation after sign-out. All 64 live API checks and all 27 browser checks passed. Revocation has a deliberate caveat: a positive check is cached for 15 s per token in each warm container, and if Cognito throttles or is unreachable the check is skipped (with a warning log) and the token is accepted on its signature and expiry.

Observed timings are individual measurements, not a performance evaluation: cold starts took about 3 s (512 MB) and the first OCR container start about 15 s; warm requests took 15–130 ms. No load test, OCR accuracy study or user study was performed; e-mail confirmation codes, CloudFront delivery and browsers other than Chrome were not tested.

9. AWS ARCHITECTURE AND DEPLOYMENT

Deployment status. FinSight was deployed to AWS on 16 October 2026 with AWS SAM: stack finance-intelligence-prod in region ap-south-1 (Mumbai), account 9672****3724 (masked), 63 resources, status CREATE_COMPLETE at 13:29 IST followed by three code-only updates. The application is live at https://zkht2b4ib3.execute-api.ap-south-1.amazonaws.com; the web interface and the API (base path /api/v1) share this origin. The figures in this chapter are generated from the template as deployed or rendered from responses of the live AWS APIs.

9.1 Deployed Architecture

Figure 42 shows the deployed architecture. The browser loads the SPA and calls the API on the same HTTP API endpoint. The routes GET / and GET /{proxy+} go to WebFunction, a 256 MB Python function that serves the built frontend files with security headers and long-term caching of hashed assets, and that has no permission to any other AWS resource. Users authenticate with Cognito; API calls carry the access token to the HTTP API, whose JWT authorizer validates it before routing /api/v1/* to one of eight API functions. The functions read and write one DynamoDB table; the receipts function also reads and writes the private receipts bucket, which browsers reach only through short-lived presigned URLs [17]. Logs and an alarm go to CloudWatch. The design follows the service documentation for Lambda [13] and HTTP API JWT authorizers [15]. In the intended configuration, which remains the template default, CloudFront serves the SPA from a private S3 bucket through origin access control [25]; those resources are drawn dashed because they were not created.

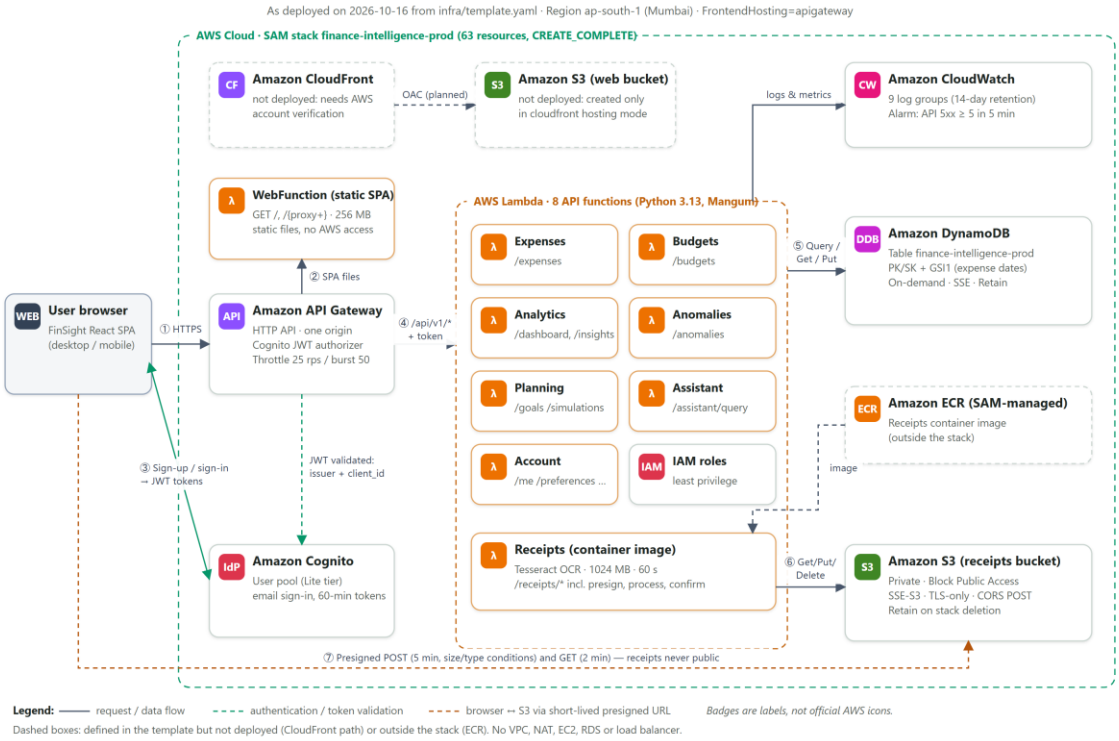


Figure 42: Deployed AWS architecture of FinSight (FrontendHosting=apigateway)

9.2 Frontend Hosting Decision

The first two deployment attempts included the CloudFront distribution and failed on that resource with the message below (docs/evidence/07_aws_deployment/). CloudFormation rolled the stack back automatically; the empty DynamoDB table retained by its deletion policy was then deleted with the owner's approval.

```
CREATE_FAILED  AWS::CloudFront::Distribution  FrontendDistribution
Access denied for operation 'AWS::CloudFront::Distribution': Your account must be verified
before you can add new CloudFront resources. To verify your account, please contact AWS Support ...
ROLLBACK_COMPLETE  finance-intelligence-prod  (attempt 2, 2026-10-16 13:24 IST)
```

Figure 43 shows the same failure in the CloudFormation console: the FrontendDistribution CREATE_FAILED message, the automatic rollback and the deletion of the failed stack before the successful attempt.

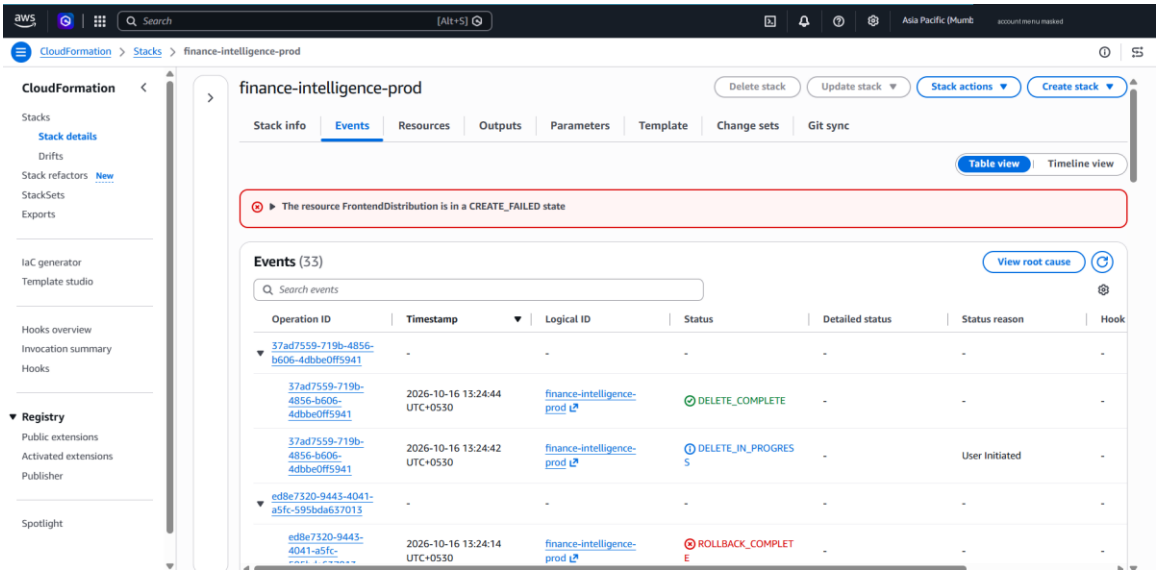


Figure 43: CloudFormation events: FrontendDistribution CREATE_FAILED, rollback and deletion of the failed attempt (AWS Console; account ID masked)

Account verification requires an AWS Support case, which could not be completed within the project timeline. A FrontendHosting parameter was therefore added to the template. With the value apigateway, the web bucket, distribution and origin access control are not created, and WebFunction serves the SPA from the HTTP API instead. The site is still served only over HTTPS; because the interface and the API share one origin, the browser needs no CORS for API calls; and the cost remains a few API requests and short Lambda invocations per page load. The trade-off is the absence of CDN edge caching and of CloudFront's managed security-header policy, which WebFunction replaces with its own headers. Once AWS Support enables CloudFront, setting FrontendHosting=cloudfront, redeploying and running infra/deploy_frontend.py restores the intended path; the web URL will then change.

9.3 Resource Mapping

Table 26: AWS resource mapping and deployment status

AWS service	Purpose	Project component	Deployment status	Cost considerations	Security considerations
AWS Lambda	Backend logic and SPA delivery	7 zip API functions + 1 container (OCR) + WebFunction	Deployed (9 functions, python3.13, x86_64)	Requests and GB-seconds; OCR 1024 MB, 60 s	Per-function IAM roles; WebFunction has no AWS permissions
API Gateway (HTTP API)	Routing, throttling, JWT authorization	HttpApi, 25 routes	Deployed; live-tested	Per-request pricing	Cognito JWT authorizer on private routes; 25 rps / burst 50
Amazon Cognito	Sign-up, sign-in, tokens	UserPool (Lite), client “web”	Deployed; live-tested	Monthly active users above the free tier	No client secret in the SPA; revocation enforced by a backend GetUser check
Amazon DynamoDB	Application data	FinanceTable (PK/SK, GSI1)	Deployed; live-tested	On-demand requests and storage; PITR off	SSE; item-level IAM, no Scan; Retain policy
Amazon S3	Private receipt storage	ReceiptBucket	Deployed; live-tested	Storage per GB-month and requests	Block Public Access, SSE-S3, TLS-only policy, presigned URLs, POST-only CORS
Amazon ECR	OCR container image	SAM-managed repository	Deployed (outside the stack; 3 image versions, newest in use)	About US$0.04 per month for ~0.38 GB	Private repository
Amazon CloudWatch	Logs and alarm	9 log groups, ApiServerErrorsAlarm	Deployed; alarm state OK	Log ingestion and storage; one alarm	14-day retention; no bodies or tokens in logs
AWS IAM	Least-privilege execution roles	SAM roles + TableAccessPolicy	Deployed	No direct charge	Scoped to one table and one bucket
Amazon CloudFront + S3 web bucket	CDN delivery of the SPA	FrontendDistribution, FrontendBucket, FrontendOAC	Not deployed (account verification pending)	Data transfer and requests	OAC, HTTPS redirect, managed security headers
AWS SAM / CloudFormation	Infrastructure as code	template.yaml, samconfig.toml	Stack CREATE_COMPLETE, then 3 code-only updates	SAM artifact bucket storage	Change-set review before execution

9.4 Deployment Workflow

Figure 44 shows the workflow as executed. The tools were AWS CLI 2.38.1, AWS SAM CLI 1.169.0 and Docker (engine 29.8.2) for the OCR image; the CLI used a dedicated profile with short-lived credentials. Before deploying, the account identity, plan and credits were checked, and a second AWS account that was configured as the default CLI profile but not yet activated was detected and not used. The review gate at step 6 ensured that no resource was created or changed until the CloudFormation change set had been inspected and approved.

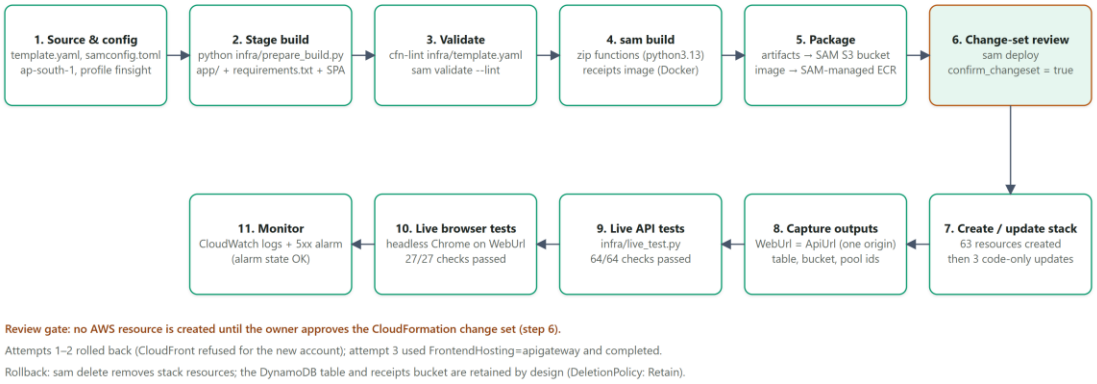


Figure 44: AWS SAM deployment workflow as executed

9.5 Deployed Resources in the AWS Management Console

Figures 45–55 are screenshots of the AWS Management Console taken by the owner on 16 October 2026 between 19:18 and 19:35 IST, after the third code-only update (docs/evidence/manual/aws-console/). For the report, the browser address bar, the console account menu and the Windows taskbar were cropped or covered, and every occurrence of the account ID in ARNs and the receipts-bucket name was blacked out; an OCR pass confirmed that no account ID remains readable (generators/prep_console.py). The API ID, Cognito pool ID and app-client ID are public identifiers that the web application itself receives. The original captures are not masked and are kept only in the evidence folder.

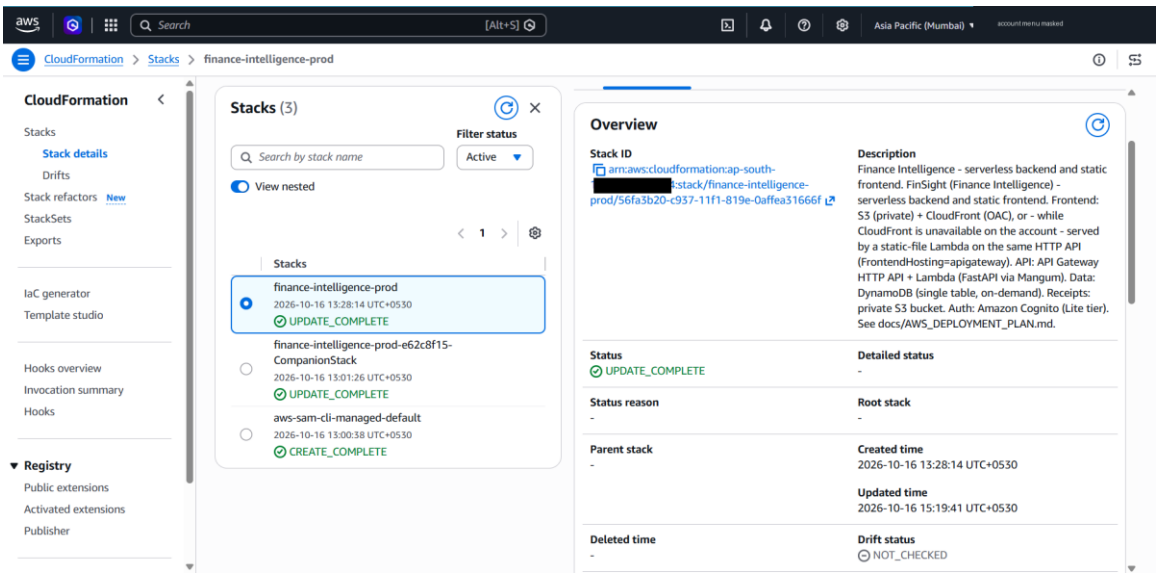


Figure 45: CloudFormation: the three FinSight stacks and the finance-intelligence-prod overview (UPDATE_COMPLETE) (AWS Console; account ID masked)

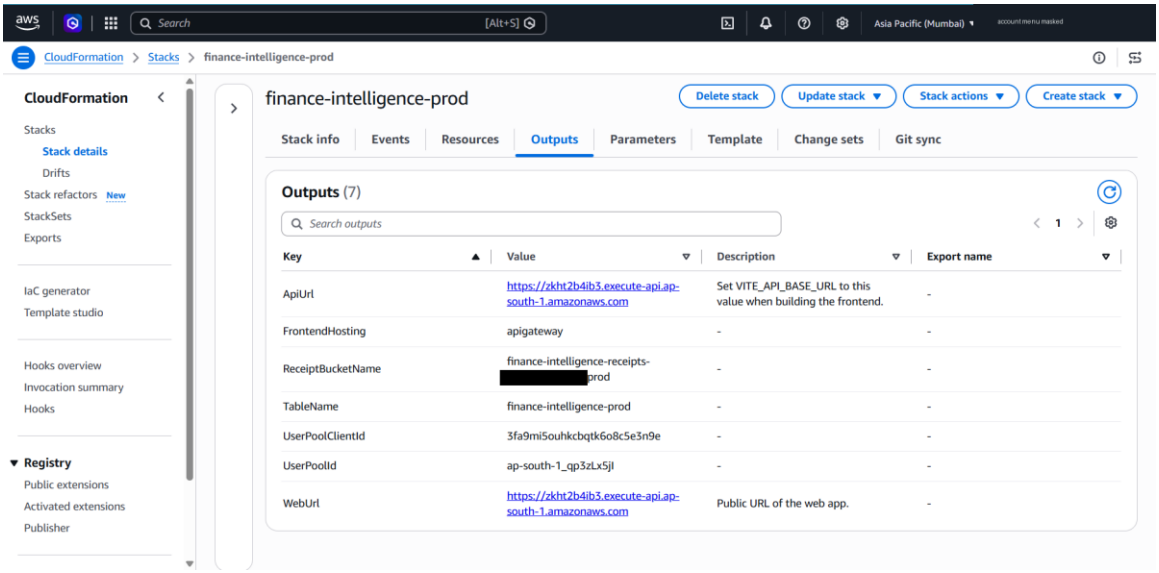


Figure 46: CloudFormation outputs: WebUrl and ApiUrl on one origin, FrontendHosting=apigateway, table, bucket and Cognito identifiers (AWS Console; account ID masked)

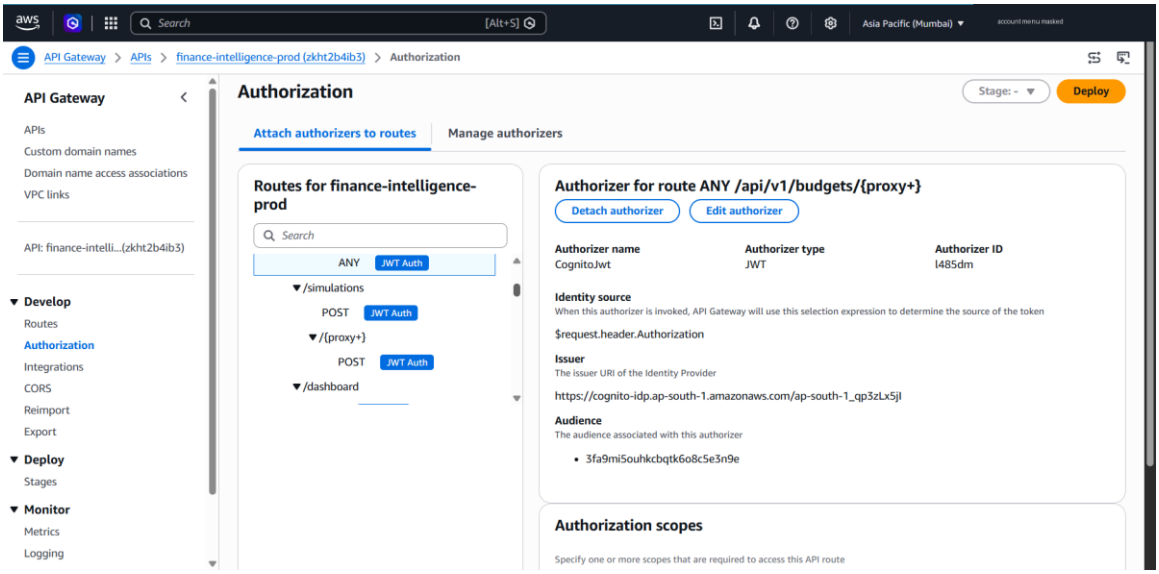


Figure 47: API Gateway: the CognitoJwt authorizer attached to the API routes (JWT Auth) (AWS Console; account ID masked)

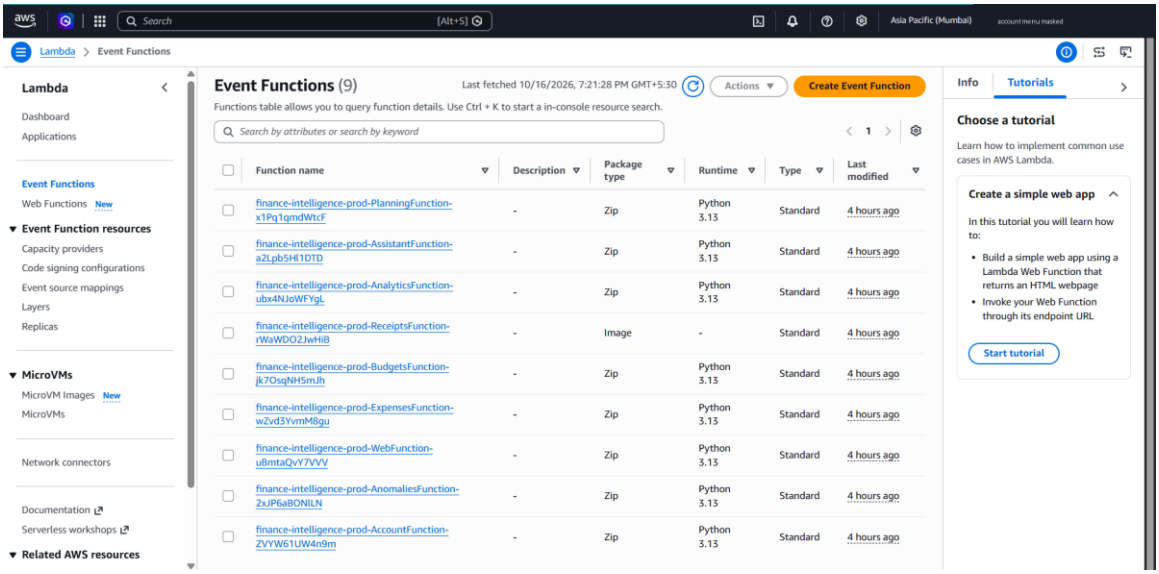


Figure 48: Lambda: the nine FinSight functions (eight zip packages on Python 3.13 and the OCR container image) (AWS Console; account ID masked)

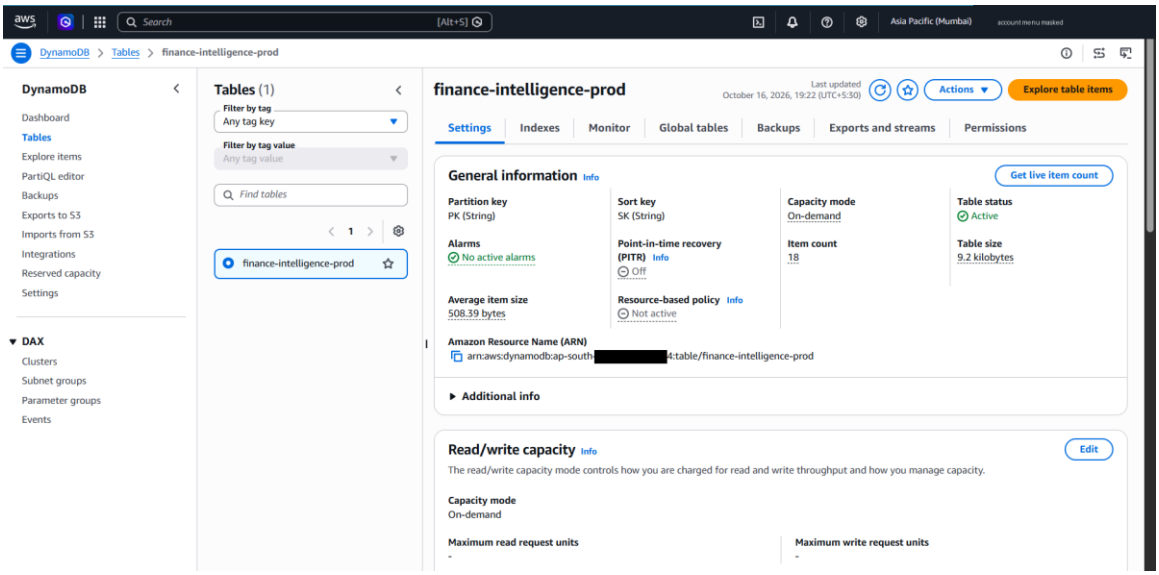


Figure 49: DynamoDB: table finance-intelligence-prod, on-demand capacity, PK/SK keys, point-in-time recovery off (AWS Console; account ID masked)

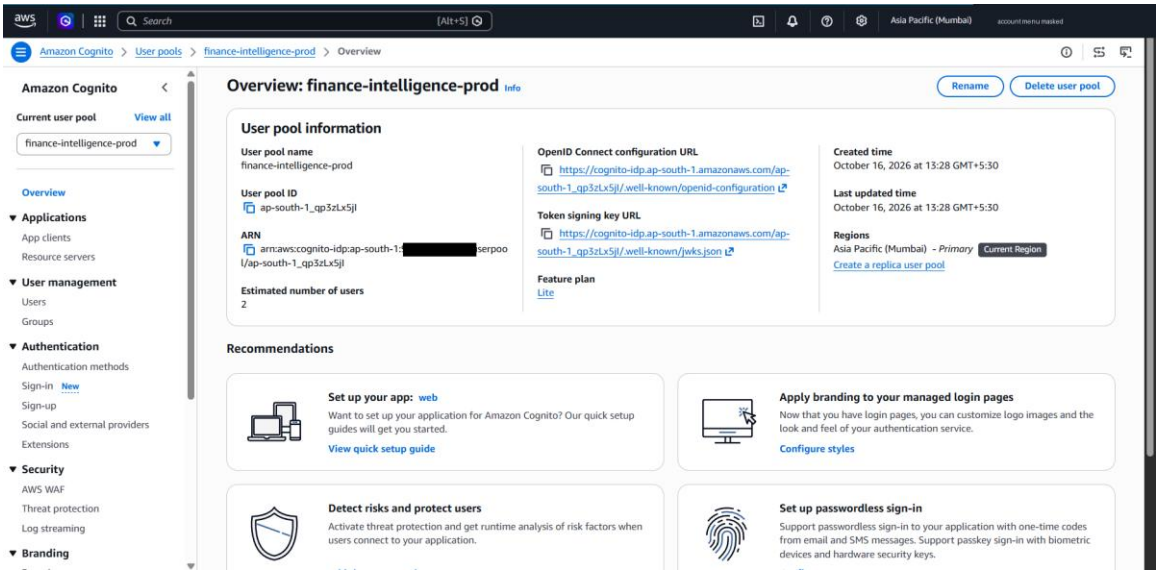


Figure 50: Cognito: user pool finance-intelligence-prod on the Lite feature plan (AWS Console; account ID masked)

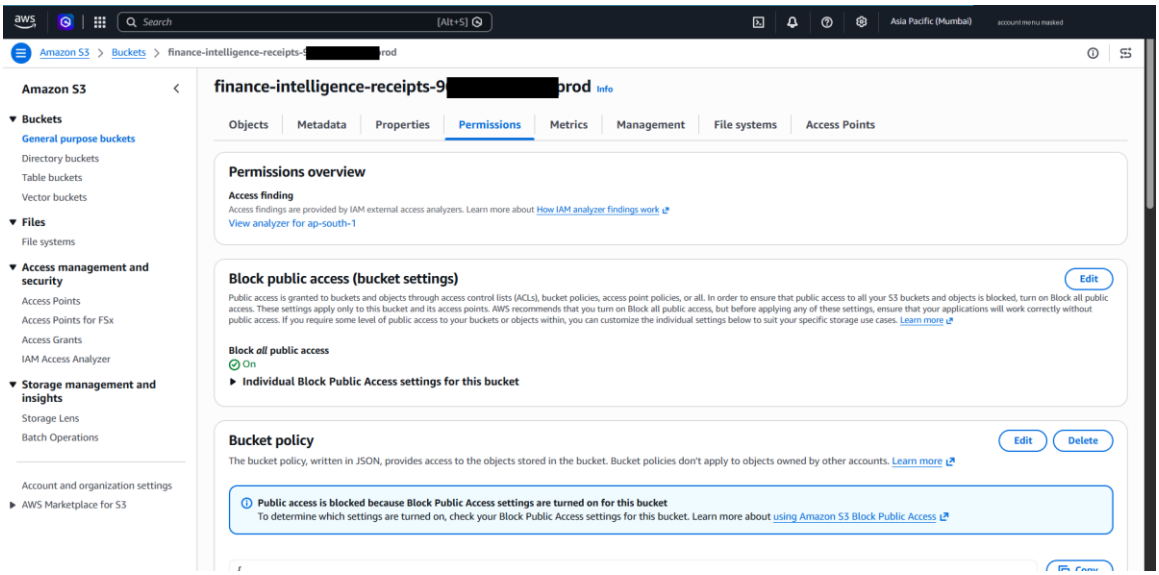


Figure 51: S3: receipts bucket permissions with Block all public access turned on (AWS Console; account ID masked)

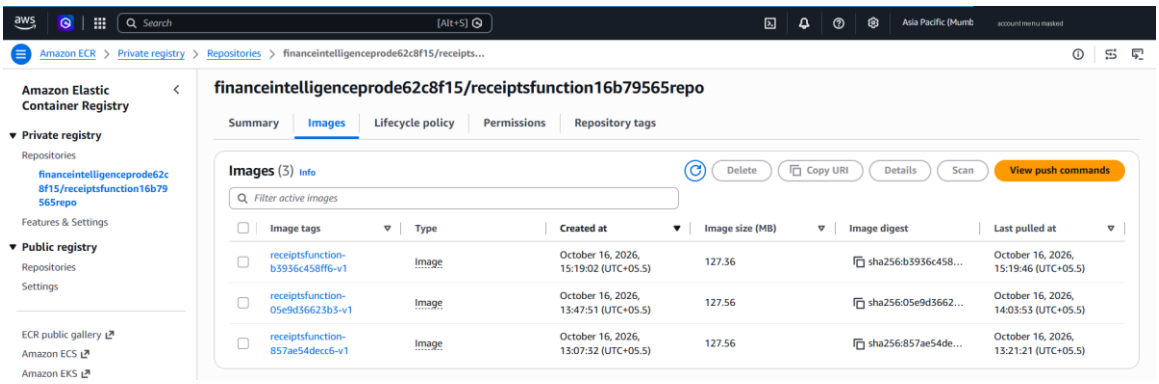


Figure 52: ECR: the three OCR container image versions; the newest is used by the receipts function (AWS Console; account ID masked)

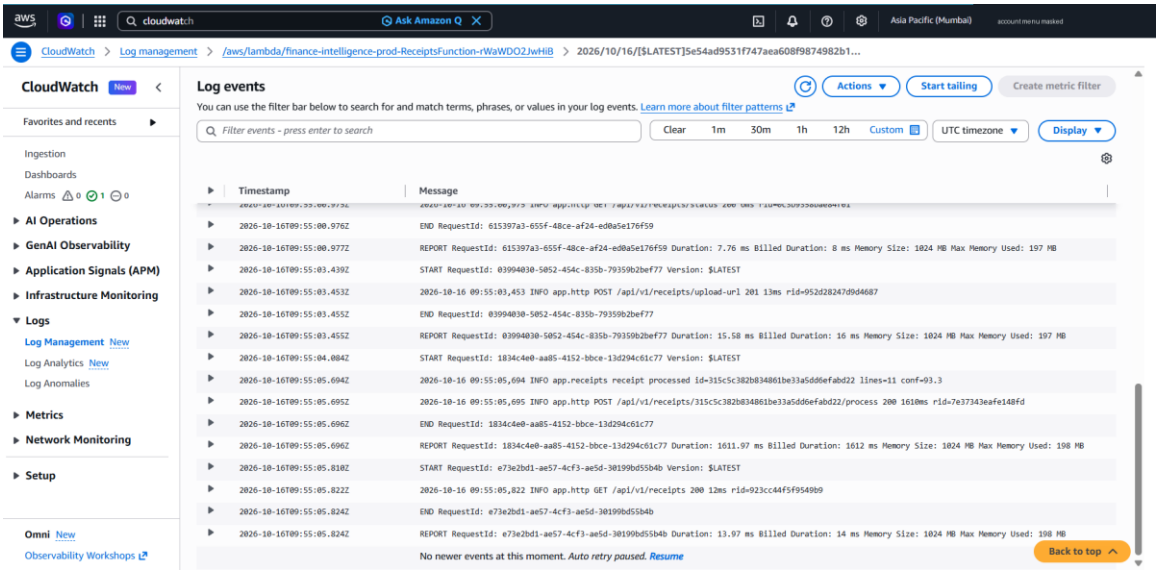


Figure 53: CloudWatch Logs: receipts-function log events with method, path, status and duration only (AWS Console; account ID masked)

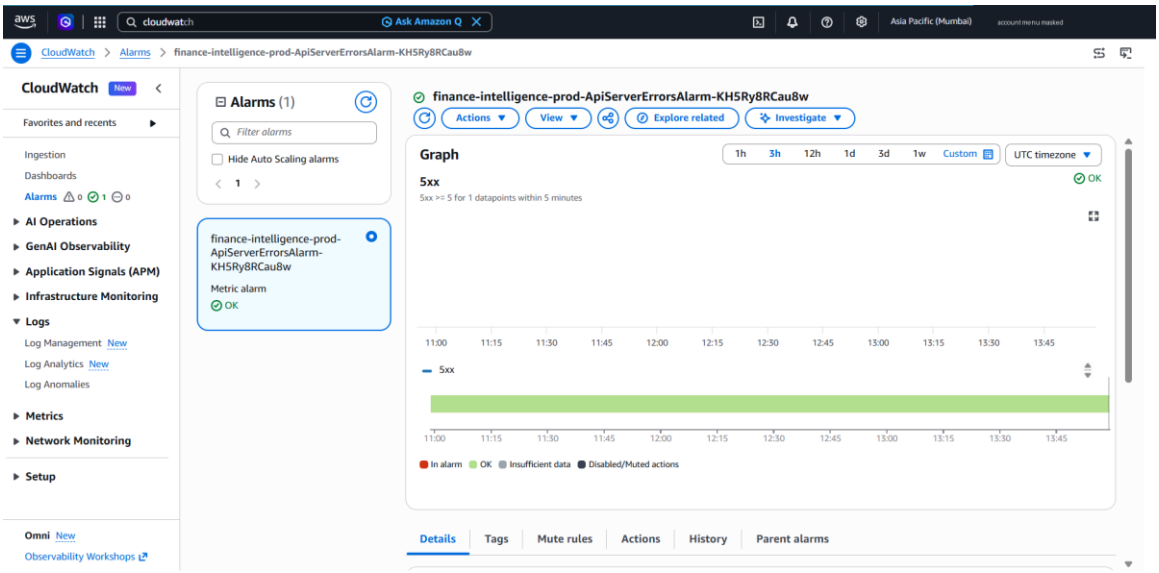


Figure 54: CloudWatch: ApiServerErrorsAlarm (5xx ≥ 5 in 5 minutes) in state OK (AWS Console; account ID masked)

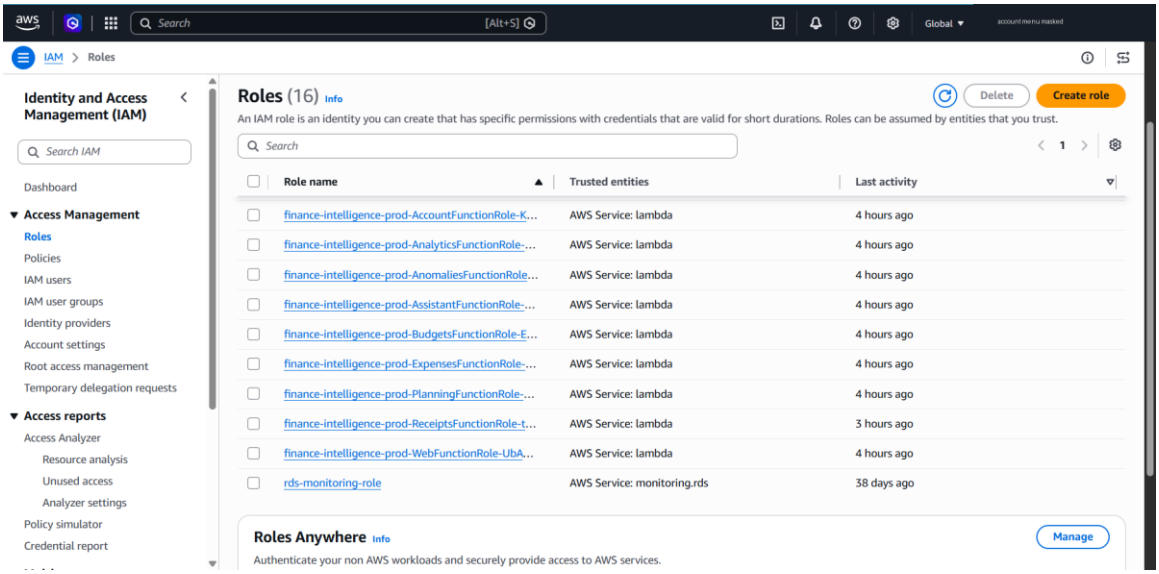
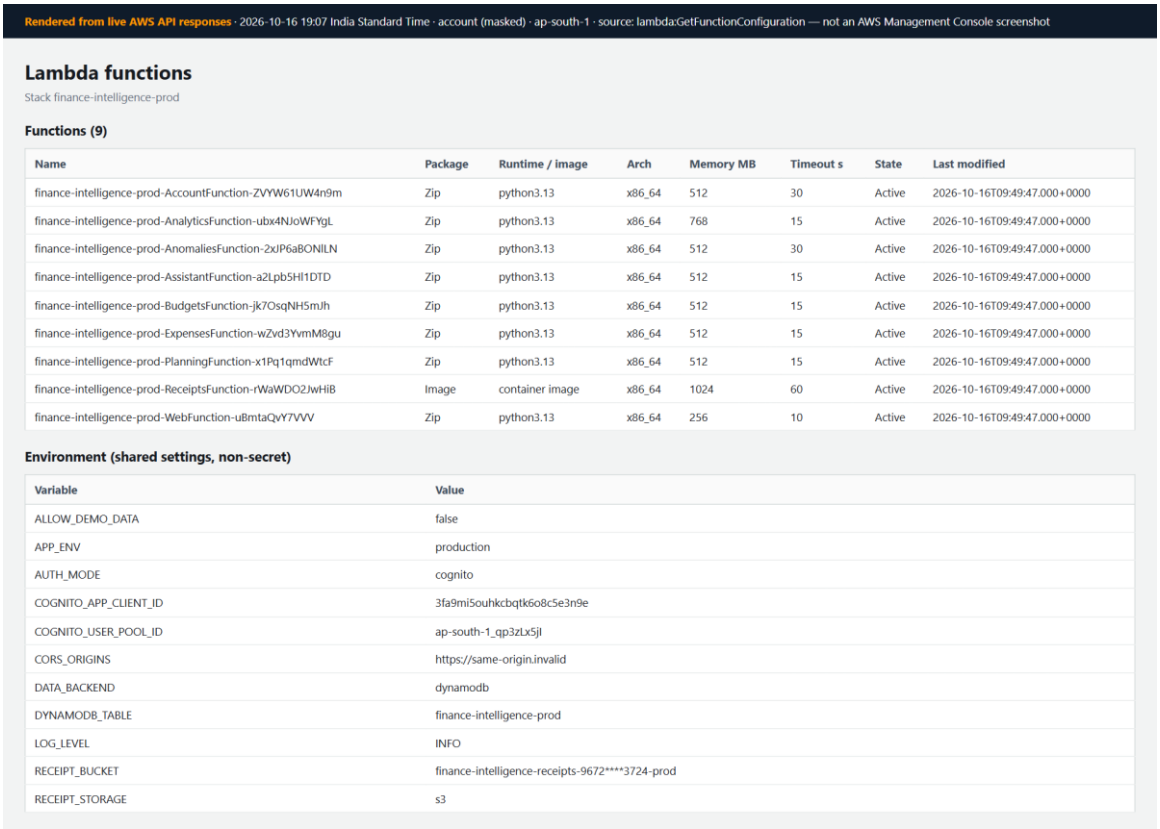


Figure 55: IAM: one execution role per FinSight function, created by the stack (AWS Console; account ID masked)

9.6 Resource Views Rendered from AWS API Responses

Figures 56 and 57 complement the Console screenshots with two details that the Console pages above do not show together: the shared Lambda environment and the full route list with authorization types. They are **not** Console screenshots: each view was rendered from the JSON responses of read-only AWS CLI calls made on 16 October 2026 at 14:26 IST (for example cloudformation describe-stacks and lambda get-function-configuration), and a banner on each image names its API sources. The account ID is masked, and the JSON responses are stored next to the images in docs/evidence/06_aws_console/. The views were captured before the third code-only update, which changed no resource configuration.



Rendered from live AWS API responses · 2026-10-16 19:07 India Standard Time · account (masked) · ap-south-1 · source: lambda:GetFunctionConfiguration — not an AWS Management Console screenshot

Lambda functions

Stack finance-intelligence-prod

Functions (9)

Name	Package	Runtime / image	Arch	Memory MB	Timeout s	State	Last modified
finance-intelligence-prod-AccountFunction-ZVYW61UW4n9m	Zip	python3.13	x86_64	512	30	Active	2026-10-16T09:49:47.000+0000
finance-intelligence-prod-AnalyticsFunction-ubx4NJoWFYgL	Zip	python3.13	x86_64	768	15	Active	2026-10-16T09:49:47.000+0000
finance-intelligence-prod-AnomaliesFunction-2xJP6aBONILN	Zip	python3.13	x86_64	512	30	Active	2026-10-16T09:49:47.000+0000
finance-intelligence-prod-AssistantFunction-a2Lpb5Hl1DTD	Zip	python3.13	x86_64	512	15	Active	2026-10-16T09:49:47.000+0000
finance-intelligence-prod-BudgetsFunction-jk7OsqNH5mJh	Zip	python3.13	x86_64	512	15	Active	2026-10-16T09:49:47.000+0000
finance-intelligence-prod-ExpensesFunction-wZvd3YvmM8gu	Zip	python3.13	x86_64	512	15	Active	2026-10-16T09:49:47.000+0000
finance-intelligence-prod-PlanningFunction-x1Pq1qmdWtcF	Zip	python3.13	x86_64	512	15	Active	2026-10-16T09:49:47.000+0000
finance-intelligence-prod-ReceiptsFunction-rWaWDO2JwHiB	Image	container image	x86_64	1024	60	Active	2026-10-16T09:49:47.000+0000
finance-intelligence-prod-WebFunction-uBmtaQvY7VVV	Zip	python3.13	x86_64	256	10	Active	2026-10-16T09:49:47.000+0000

Environment (shared settings, non-secret)

Variable	Value
ALLOW_DEMO_DATA	false
APP_ENV	production
AUTH_MODE	cognito
COGNITO_APP_CLIENT_ID	3fa9mi5ouhkcbqtk6o8c5e3n9e
COGNITO_USER_POOL_ID	ap-south-1_qp3zLx5jI
CORS_ORIGINS	https://same-origin.invalid
DATA_BACKEND	dynamodb
DYNAMODB_TABLE	finance-intelligence-prod
LOG_LEVEL	INFO
RECEIPT_BUCKET	finance-intelligence-receipts-9672****3724-prod
RECEIPT_STORAGE	s3

Figure 56: Deployed Lambda functions (eight API functions and WebFunction) and their shared non-secret environment (rendered from live AWS API responses — not a Console screenshot)

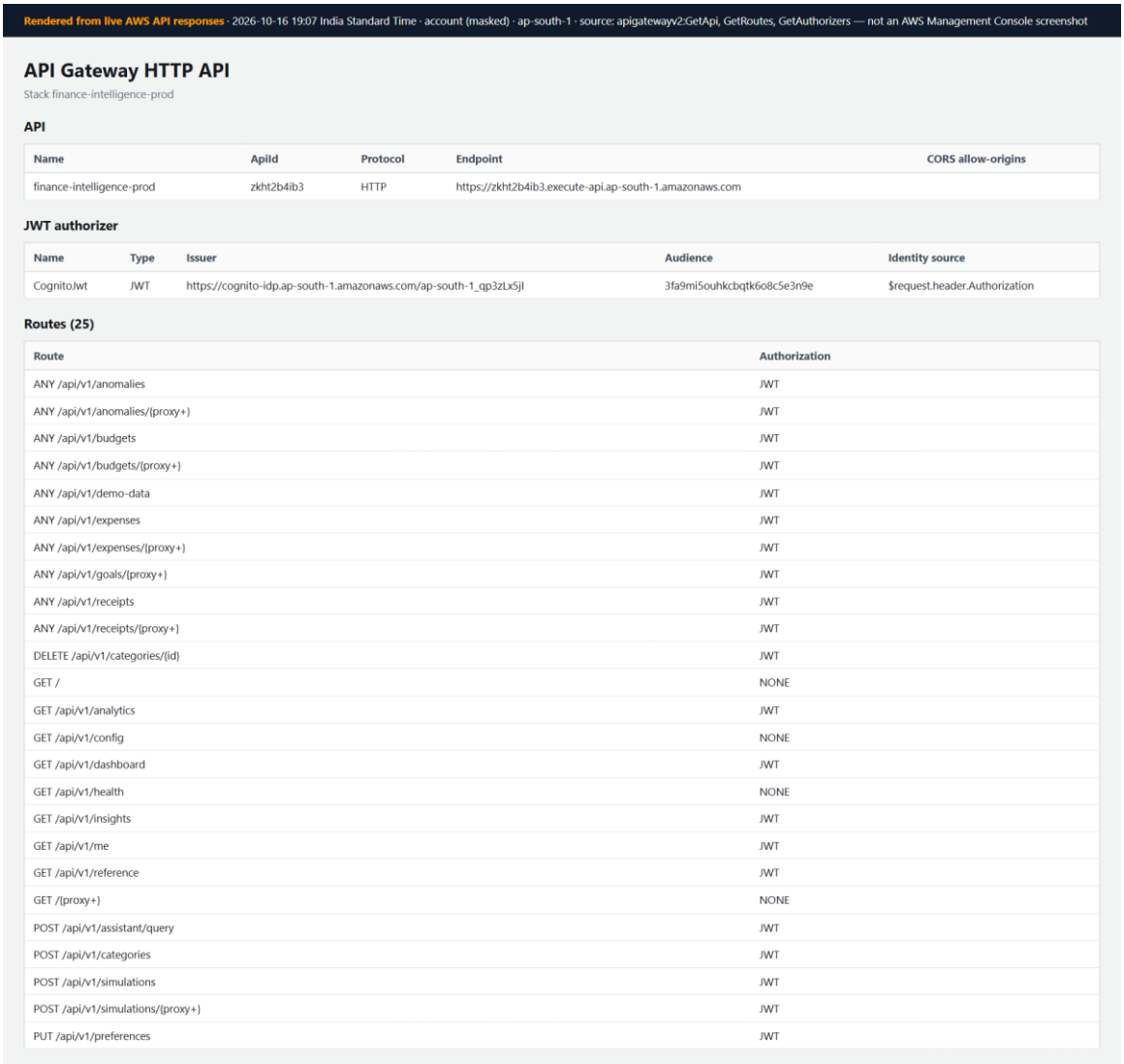


Figure 57: API Gateway HTTP API: Cognito JWT authorizer and the 25 routes with their authorization type (rendered from live AWS API responses — not a Console screenshot)

9.7 Monitoring

Each of the nine functions writes to its own CloudWatch log group with 14-day retention, and the ApiServerErrorsAlarm watches API Gateway 5xx responses (five or more in five minutes); it was in state OK after testing (Figures 53 and 54). Logs contain only method, path, status, duration and a request id; the alarm has no notification target yet. Metrics for the two hours of testing showed, for example, 147 invocations of the account function and 233 of WebFunction (maximum 94 ms). The single function error recorded was the import failure that was fixed by the packaging change in Table 25. The OCR function's longest invocation was about 15 s, its first container start.

9.8 Cost Analysis

The design uses only serverless, pay-per-use services and avoids resources that charge while idle, such as EC2 instances, RDS databases, NAT gateways, load balancers and

public IPv4 addresses. On 16 October 2026 the account was active on the AWS **Free plan** with US$159.91 of promotional credits (AWS added US$20 during the day), and the plan expires on 28 February 2027. Cost Explorer reported US$0.00 month-to-date, but its data lag by up to 24 hours, so the deployment and test day may not yet be included (docs/PROJECT_DOCUMENTATION.md, Section 17). The Billing console's credits page later that evening (Figure 58) showed US$0.06 of credits used (US$0.09 estimated) and US$159.91 estimated remaining. The main cost drivers are:

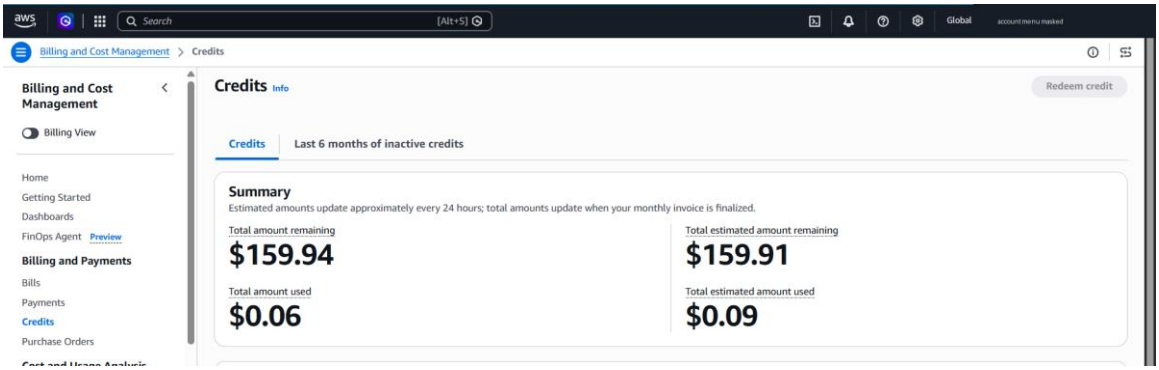


Figure 58: Billing and Cost Management: credits summary on 16 October 2026 (AWS Console; account ID masked)

- **API requests.** API Gateway HTTP APIs charge per request; throttling (25 requests per second, burst 50) bounds runaway usage. Without CloudFront, page loads also pass through the API.
- **Lambda.** Charges depend on the number of invocations and duration × memory; the OCR function (1024 MB, up to 60 s) is the most expensive per call but runs only when a receipt is processed. WebFunction uses 256 MB and responds in milliseconds.
- **Container image.** Three OCR image versions (about 0.38 GB in total) are stored in Amazon ECR, about US$0.04 per month; only the newest is used, and deleting the other two would save about US$0.025 per month.
- **Storage.** DynamoDB and S3 hold kilobytes of synthetic data; on-demand capacity avoids paying for idle throughput, and point-in-time recovery is disabled to avoid extra storage cost.
- **Authentication.** Cognito charges per monthly active user beyond its free tier.
- **Logging.** CloudWatch charges for log ingestion and storage; 14-day retention prevents unbounded growth.

Assuming demonstration use (at most 5,000 API requests, 50 OCR runs and 10 MB of data per month), the estimate is about US$0.05–0.10 per month, drawn from the credits; Lambda stays within its always-free allowance and ECR storage is the largest item. AWS's documentation states that accounts on the Free plan close automatically after six months or when the credits are used, whichever comes first, whereas the Paid plan bills usage beyond credits [21]. An AWS Budgets zero-spend alert exists on the account, but

alerts are notifications, not spending caps. These figures are estimates; no monthly bill had been issued when this report was written.

9.9 Cleanup and Rollback

- Rollback was exercised during deployment: both failed attempts were rolled back automatically by CloudFormation, and the code-only updates can be reverted by redeploying the previous build.
- sam delete --stack-name finance-intelligence-prod removes the functions, API, Cognito pool, log groups and alarm.
- The DynamoDB table and the receipts bucket use DeletionPolicy: Retain to protect data; they must be deleted manually (the bucket emptied first) only after deciding the data is no longer needed.
- The SAM artifact bucket stack and the ECR repository are outside the application stack and must be cleaned up separately. A full cleanup permanently removes all FinSight data and is planned only when the project is finished.

10. SECURITY AND PRIVACY

FinSight handles sensitive personal financial data but does not connect to bank accounts, collect bank credentials or move money. Table 27 lists the controls found in the code and template and states whether each was tested locally, tested on the live AWS deployment, or deployed without a dedicated test.

Table 27: Security controls and their verification status

Control	Implementation	Status
Authentication on private endpoints	FastAPI dependency verifies the bearer token on every private route	Implemented; tested
JWT validation	Local: HS256, required claims, pinned algorithm. Cognito: RS256 via JWKS, issuer, token_use, client_id	Tested locally and live (AUTH-05, SEC-09)
Token revocation	Cognito GetUser after JWKS verification; 15 s cache per token; fail-open with a warning log	Tested locally and live (AUTH-07: 401 token_revoked)
Edge authorization	API Gateway Cognito JWT authorizer	Deployed; live-tested (SEC-01–09)
User-data isolation	User id only from the token; every query scoped; 404 for other users' ids	Tested locally and live (ISO-01–03)
Input validation	Pydantic models reject unknown fields; amount/date/category rules	Implemented; tested
Password handling	scrypt (N = 2^14, r = 8, p = 1) with salt; constant-time comparison; never logged	Implemented; tested
Brute-force protection	5 failed logins per e-mail and IP per 5 minutes → 429; generic error	Implemented (in-memory); tested
Safe uploads	Size limit, magic bytes, full decode, pixel cap, server-generated keys	Implemented; tested
Private receipt storage	Local private folder; S3 Block Public Access, SSE, TLS-only policy	Tested locally, with moto and live (unsigned GET 403)
Presigned URLs	POST 5 min with size/type/encryption conditions; GET 2 min	Tested with moto and live (6 MB upload refused)
CORS	Allowed origins from configuration; wildcard refused in production; same origin on AWS	Tested locally and live (COR-01–02)
Security headers	API: nosniff, X-Frame-Options DENY, no-referrer, no-store. SPA: HSTS, nosniff, frame options, referrer policy from WebFunction	Tested live (FE-03); CloudFront not deployed
Least-privilege IAM	One table and indexes, item-level actions, no Scan; receipts bucket objects only; WebFunction without AWS access	Deployed; reviewed in the template
Log hygiene and retention	No bodies, tokens or receipt text in logs; 14-day retention	Deployed (9 log groups, 14 days)
Error handling	Generic 500 message; uniform envelope; docs disabled in production	Implemented; tested
Secrets management	No secrets in source; .env and local data git-ignored; production requires JWT_SECRET for local auth	Implemented
Dependency audit	npm audit (production dependencies); pip-audit on backend/requirements.txt	0 vulnerabilities; none known (2026-10-16)

These controls align with widely used guidance such as the OWASP Top 10 [19] and NIST SP 800-63B [20], but the project has **not** been independently audited and claims no

compliance or certification. The main remaining risks are listed in Section 3.4: no MFA in local mode, an in-memory rate limiter, tokens in sessionStorage, no malware scanning or web application firewall, an 8-character minimum password length, and a revocation check that is cached for 15 s and fails open. A review of the deployed system (docs/PROJECT_DOCUMENTATION.md, Section 16.4) found no critical or high findings and three medium ones: deployments use the root account (protected by MFA, with no access keys), there is no Content-Security-Policy header, and there is no WAF or CDN layer while CloudFront is blocked. Recommended improvements are an IAM Identity Center administrator for deployments, a Content-Security-Policy, AWS WAF on CloudFront once it is enabled, Cognito MFA, a shared rate-limit store, upload scanning and deleting the Cognito user together with application data.

11. CONCLUSION

This project addressed a common gap in personal finance tools: recording expenses is easy, but understanding them is not. FinSight combines accurate expense management with budgets, analytics and five explainable analytical modules — a financial intelligence engine, robust anomaly detection, open-source receipt OCR with mandatory review, a What-If simulator and a rule-based assistant — all computed from the user's own data without paid AI services.

The objectives in Section 1.4 were met, both locally and on the AWS deployment. Money is handled exactly as integer minor units; budgets project month-end spending and alert once per threshold; every insight states its kind and evidence; anomaly rules follow robust statistical practice and never alter data; receipts are read by Tesseract and confirmed by the user; simulations have no side effects; and every private request is scoped to the authenticated user. These properties are supported by 131 passing backend tests, 20 frontend tests, a clean TypeScript check and production build, a lint-clean infrastructure template and scripted browser workflows.

The application was deployed to AWS in the ap-south-1 region as a serverless stack — an HTTP API with a Cognito JWT authorizer, eight API Lambda functions, a single-table DynamoDB design and private S3 storage — with least-privilege IAM, short log retention and no always-on resources, at an estimated cost of about US$0.05–0.10 per month. On the live system, all 64 API checks and all 27 browser checks passed. Because the business logic is separated from storage and transport, moving to the cloud was a configuration and deployment step rather than a rewrite; the problems that did appear (native-library packaging, a regional S3 redirect and access tokens that survived sign-out) were found only by deploying and testing on real infrastructure, and all three were fixed.

The main limitations are that CloudFront delivery could not be enabled on the new AWS account, so the interface is served through API Gateway without edge caching; that revocation after sign-out takes effect within 15 s and fails open if Cognito is unavailable; and that no load, OCR accuracy or user study was performed. Within these limits, FinSight contributes a working, tested and transparent personal finance application running on a cost-conscious, security-focused serverless deployment.

12. FUTURE WORK

The following items are future work and are not part of the current implementation:

- **CloudFront delivery:** after AWS Support verifies the account, redeploy with FrontendHosting=cloudfront and repeat the live tests on the new URL.
- **Deployment hardening:** use an IAM Identity Center administrator instead of the root sign-in for deployments, add a Content-Security-Policy header and an SNS notification for the 5xx alarm, and delete the two unused OCR images.
- **Automated deployment pipeline:** build, test, lint and deploy through CI/CD with manual approval of change sets.
- **Expense classification:** suggest categories from merchant history with an explainable model, keeping user confirmation.
- **OCR accuracy:** deskewing, adaptive thresholding, PDF receipts and a labelled evaluation set to measure field-level accuracy.
- **Anomaly detection:** seasonality-aware baselines and evaluation of false-positive rates with consenting users.
- **Explainability:** user studies of whether the evidence panels improve understanding [12].
- **Data integrations:** optional statement import (CSV) before any bank-feed integration, with privacy review.
- **Accessibility:** a formal audit with assistive technologies.
- **Monitoring, backup and recovery:** dashboards and alarm notifications, point-in-time recovery for DynamoDB when real data is stored.
- **Security testing:** MFA, penetration testing, dependency scanning and a web application firewall.
- **Performance evaluation:** load and latency testing of the serverless deployment.
- **Localisation and currencies:** multiple languages and currency conversion with dated exchange rates.
- **Financial recommendations:** only after careful validation, with clear disclaimers that FinSight is not financial advice.

REFERENCES

[1] A. Lusardi and O. S. Mitchell, “The economic importance of financial literacy: Theory and evidence,” *Journal of Economic Literature*, vol. 52, no. 1, pp. 5–44, 2014.

[2] J. Kaye, M. McCuistion, R. Gulotta, and D. A. Shamma, “Money talks: Tracking personal finances,” in *Proc. SIGCHI Conf. Human Factors in Computing Systems (CHI ’14)*, 2014, pp. 521–530, doi: 10.1145/2556288.2556975.

[3] V. Chandola, A. Banerjee, and V. Kumar, “Anomaly detection: A survey,” *ACM Computing Surveys*, vol. 41, no. 3, Art. 15, 2009, doi: 10.1145/1541880.1541882.

[4] R. J. Bolton and D. J. Hand, “Statistical fraud detection: A review,” *Statistical Science*, vol. 17, no. 3, pp. 235–255, 2002, doi: 10.1214/ss/1042727940.

[5] C. Leys, C. Ley, O. Klein, P. Bernard, and L. Licata, “Detecting outliers: Do not use standard deviation around the mean, use absolute deviation around the median,” *Journal of Experimental Social Psychology*, vol. 49, no. 4, pp. 764–766, 2013, doi: 10.1016/j.jesp.2013.03.013.

[6] B. Iglewicz and D. C. Hoaglin, *How to Detect and Handle Outliers* (ASQC Basic References in Quality Control, vol. 16). Milwaukee, WI, USA: ASQC Quality Press, 1993.

[7] R. Smith, “An overview of the Tesseract OCR engine,” in *Proc. 9th Int. Conf. Document Analysis and Recognition (ICDAR 2007)*, 2007, pp. 629–633, doi: 10.1109/ICDAR.2007.4376991.

[8] Z. Huang, K. Chen, J. He, X. Bai, D. Karatzas, S. Lu, and C. V. Jawahar, “ICDAR2019 competition on scanned receipt OCR and information extraction,” in *Proc. Int. Conf. Document Analysis and Recognition (ICDAR)*, 2019, doi: 10.1109/ICDAR.2019.00244.

[9] Y. Xu, M. Li, L. Cui, S. Huang, F. Wei, and M. Zhou, “LayoutLM: Pre-training of text and layout for document image understanding,” in *Proc. 26th ACM SIGKDD Int. Conf. Knowledge Discovery & Data Mining (KDD ’20)*, 2020, doi: 10.1145/3394486.3403172.

[10] E. Jonas et al., “Cloud programming simplified: A Berkeley view on serverless computing,” EECS Dept., Univ. California, Berkeley, Tech. Rep. UCB/EECS-2019-3, 2019, arXiv:1902.03383.

[11] J. M. Hellerstein, J. Faleiro, J. E. Gonzalez, J. Schleier-Smith, V. Sreekanti, A. Tumanov, and C. Wu, “Serverless computing: One step forward, two steps back,” in *Proc. Conf. Innovative Data Systems Research (CIDR)*, 2019.

[12] F. Doshi-Velez and B. Kim, “Towards a rigorous science of interpretable machine learning,” arXiv:1702.08608, 2017.

[13] Amazon Web Services, “What is AWS Lambda?,” *AWS Lambda Developer Guide*. [Online]. Available: https://docs.aws.amazon.com/lambda/latest/dg/welcome.html

[14] Amazon Web Services, “What is the AWS Serverless Application Model (AWS SAM)?,” *AWS SAM Developer Guide*. [Online]. Available: https://docs.aws.amazon.com/serverless-application-model/latest/developerguide/what-is-sam.html

[15] Amazon Web Services, “Controlling access to HTTP APIs with JWT authorizers,” *Amazon API Gateway Developer Guide*. [Online]. Available: https://docs.aws.amazon.com/apigateway/latest/developerguide/http-api-jwt-authorizer.html

[16] Amazon Web Services, “NoSQL design for DynamoDB,” *Amazon DynamoDB Developer Guide*. [Online]. Available:

https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/bp-general-nosql-design.html

[17] Amazon Web Services, “Download and upload objects with presigned URLs,” *Amazon S3 User Guide*. [Online]. Available: https://docs.aws.amazon.com/AmazonS3/latest/userguide/using-presigned-url.html

[18] Amazon Web Services, “What is Amazon Cognito?,” *Amazon Cognito Developer Guide*. [Online]. Available: https://docs.aws.amazon.com/cognito/latest/developerguide/what-is-amazon-cognito.html

[19] OWASP Foundation, “OWASP Top 10:2021.” [Online]. Available: https://owasp.org/Top10/

[20] National Institute of Standards and Technology, “Digital Identity Guidelines: Authentication and Lifecycle Management,” *NIST Special Publication 800-63B*. [Online]. Available: https://pages.nist.gov/800-63-3/sp800-63b.html

[21] Amazon Web Services, “Choosing a plan,” *AWS Billing and Cost Management User Guide* (accessed Oct. 2026). [Online]. Available: https://docs.aws.amazon.com/awsaccountbilling/latest/aboutv2/free-tier-plans.html

[22] S. Ramírez, “FastAPI documentation.” [Online]. Available: https://fastapi.tiangolo.com/

[23] Meta Open Source, “React documentation.” [Online]. Available: https://react.dev/

[24] Tesseract OCR contributors, “Tesseract documentation.” [Online]. Available: https://tesseract-ocr.github.io/tessdoc/

[25] Amazon Web Services, “Restricting access to an Amazon S3 origin (origin access control),” *Amazon CloudFront Developer Guide*. [Online]. Available: https://docs.aws.amazon.com/AmazonCloudFront/latest/DeveloperGuide/private-content-restricting-access-to-s3.html